AirOS 路侧感知系统 — 面试 QA 精简版（35 题）

> **这份是什么** 从 `0面试QA总表.md`（83 题）里挑出的**最核心 33 题 + 新增 2 题**，用来考前突击、临场前快速过一遍。 **题号沿用原表**（看到 Q2-3 就是原表的 Q2-3），想深挖某一题，回原表按题号查即可。 **不替代原表**——原表是字典，这份是常用字表。
>
> **挑选标准**：`[必背]` + `★★` 高频 + **诚实题**（容易穿帮的）+ 每个条目的**总览题**。 砍掉的是同类重复题和纯加分储备题——**不是说不考，是优先级低。**
>
> **怎么用**
>
> 1. **每个条目的第 1 题是"目录"**：答案里抛出的每个词，下面都有编号追问，按真实面试被问的顺序铺好了。
> 2. 其余题保留 1 条主追问，用来接住"为什么 / 会怎样"。
> 3. 答案都能**直接念出口**，念完 20~30 秒；念超时说明你加戏了。
>
> 🔴 **四条全局红线（全文通用，先看完再背）**
>
> 1. 全程说「**参与**」，不说「我从零设计」。
> 2. 视频那条说「**拉流和拆包**」，**绝不说解码**——本驱动没有解码环节。
> 3. 部署工具写「独立开发并维护升级」；**口语里主动说用了 AI 辅助**，别等被问出来。
> 4. 交通参数检测那条**另有专属红线**，写在该条开头，**先看完再背题。**
>
> ⏱️ **突击顺序建议**：开场 3 题 → 附录角色题 → 条目二 → 条目四 → 条目一 → 条目三。

目录

- 第 0 部分：开场自述（3 题）
- 条目一：设备抽象与插件化接入（8 题）
- 条目二：多路视频流接入（7 题）
- 条目三：设备与平台运维（5 题）
- 条目四：交通参数检测服务（8 题）
- 附录：通用追问（4 题）

第 0 部分：开场自述（3 题）

Q0.1 你能讲一下 AirOS 和交通流量模块的整体业务流程吗？ [必背]

答：

整套系统部署在路侧 **MEC** 设备上，AirOS 负责设备接入和相关业务处理。以相机为例，接入后通过 FFmpeg **拉流、拆包**，把压缩视频数据封装成 **Protobuf 消息**，发布到 **CyberRT 总线**。下游算法服务订阅后，完成解码、检测、跟踪或融合，输出包含目标 ID、位置、速度等内容的连续目标消息。

交通流量服务以插件形式运行在 **FusionService** 中，通过 DDS 订阅这些目标消息，结合 ROI 工具生成的车道、停止线配置，**逐帧累积、按周期结算**。我负责其中流量和车头时距检测器的实现。统计结果组织成 JSON 消息，由框架发布，再经中继适配进入 AirOS，由交通业务组件处理并上报远端平台。

另外，目标等信息还可以进入 AirOS 的 **V2X 业务分支**，经过消息组织和编码，交给路侧 **RSU**，再无线发送给车载 **OBU**。平台上报和车端广播是两条不同的业务分支。

追问 1：边缘盒子是什么？为什么算力要放在路口，不上云？

> 就是放在路口机柜里的一台小工控机，带 GPU。必须在本地算是因为**延时和带宽**——视频流全量回传带宽扛不住，而车路协同要的是毫秒级响应，绕一圈云端就来不及了。云只用来做管理和长期存储。

追问 2：你说的"接进来"，具体接了哪几类设备？

> 主要是**相机、雷达、信号机**这几类。相机出视频流，雷达出目标点云，信号机给的是路口的灯态相位。它们协议完全不一样，所以才需要那套统一接入接口把它们抹平。

追问 3：为什么用 C++？又为什么跑在容器里？

> C++ 是因为**要压榨边缘设备的算力**，而且要在多线程下稳定跑几个月不重启。跑容器是为了**部署和环境隔离**——几十个路口的硬件批次不同，容器能保证我这份代码在任何一台上跑起来都一样。

追问 4：你们这个系统是分层设计的吗？

> 是。上面应用层、中间服务层、下面硬件抽象层。分层的好处是：加一个新设备不用动上面的代码，按标准实现一个"插件"挂上去就行。

追问 5：「插件化接入」具体是什么意思？

> 驱动编译成独立的动态库，框架启动时扫目录加载，驱动里一行注册宏把自己登记进工厂。所以加一款新设备是**加一个文件 + 改一行配置**，框架代码不用动。 👉 深挖见**条目一 Q1-1**。

追问 6：多路视频流是怎么组织的？

> **一路流 = 一份私有状态 = 一个独立线程**，一个设备管理多路。路跟路之间互不干扰，一路断了不影响别的路。 👉 深挖见**条目二 Q1-1**。

追问 7：运维具体运维什么？部署工具是干嘛的？

> 两块：一是**采 GPU 利用率、显存、温度**这些指标上报，看设备健不健康；二是我独立做的一个**桌面批量部署工具**，把"运维拿 Excel 一台一台 SSH 敲命令"变成"选表 + 点按钮"。 👉 深挖见**条目三 Q1-1 / Q3-1**。

追问 8：交通参数那块，你订阅的到底是什么数据？

> 订阅的是**上游融合感知输出的结构化目标流**——一帧里所有目标的清单，每条带 id、车型、经纬度、速度、车道号。**我从头到尾没见过像素**，解码和检测跟踪是算法团队做的。 👉 深挖见**条目四 Q1-1**。

追问 9：你这两块之间是什么关系？

> 是同一条业务链的**首尾两端**。前面我负责把设备数据接进来，中间解码、目标检测跟踪、多传感器融合是算法团队做的，他们产出结构化结果后我才在后面算交通参数。**中间那一棒不是我做的。**

> 📍 **这一答抛出了整张地图**——被追问时往哪儿走：
>
> - **"插件化接入"** → 条目一 Q1-1
> - **"多路视频流"** → 条目二 Q1-1
> - **"运维 / GPU 指标采集"** → 条目三 Q1-1
> - **"部署工具"** → 条目三 Q3-1
> - **"交通参数检测 / 订阅目标流"** → 条目四 Q1-1
> - **"两块什么关系 / 哪些不是你做的"** → Q0.3、附录 Q1

八股补给：

Q1：边缘计算和云计算，本质区别是什么？

> 就一条：**算力放在离数据多近的地方**。云是"数据送到远方的大机房算"，边缘是"在数据产生的现场就地算"。 选择边缘是因为两个硬约束：**带宽**——几十路视频全量回传，网络扛不住；**延时**——车路协同要毫秒级响应，数据绕一圈云端再回来就来不及了。 云并没有被取代，它负责的是**管理、配置和长期存储**这些不要求实时的事。

Q2：容器和虚拟机有什么区别？

> **虚拟机**是虚拟出一整台"电脑"——包括自己的操作系统内核，所以重、启动慢、占资源多。 **容器**是**共用宿主机的内核**，只把应用和它依赖的库**打包隔离**起来。所以它轻、启动快（秒级）、一台机器上能跑很多个。 代价是**隔离不如虚拟机彻底**（毕竟共用内核），以及**容器内看到的内核就是宿主机的**，所以跨硬件平台不像虚拟机那么自由。

Q3：为什么容器能保证"在哪台机器上跑都一样"？

> 因为它打包的不只是你的程序，还有**它依赖的所有库和系统工具**。同一份镜像在任何一台装了容器运行时的机器上，看到的"环境"是**一模一样的**。 这在我们的场景里特别关键——几十个路口的设备**硬件批次和出厂系统版本都不一样**，靠容器才保证我这份代码在任何一台上行为一致。 一句话：**容器解决的是"在我机器上明明是好的"这个千古难题。**

Q4：进程和线程有什么区别？

> **进程**是操作系统分配资源的最小单位——每个进程有自己独立的内存空间，互相看不见。**线程**是 CPU 调度的最小单位——一个进程里的多个线程**共享同一块内存**。 所以：线程之间传数据极快（直接读写同一份变量），代价是**必须自己管好锁**；进程之间互不可见，安全，但要通信就得走专门的方式（管道、共享内存、消息队列这些）。 我们"每路流一个线程"就是看中线程**共享数据方便**；而"驱动做成动态库"是**在同一个进程里**被加载进来的，不是独立进程。

> 背法：Q0.1 是**全表目录**。开场这 40 秒答完，面试官接下来问的每一句，基本都落在上面这张表里。---

Q0.3 你做的这些，和「交通参数检测」是一回事吗? [必背] ★★ 开场必被问

答： 两块都属于公司的路侧产品线，装在同一台边缘设备上，是同一条业务链的**首尾两端**：我前面负责把设备数据接进来、保证链路稳，后面负责订阅上游的感知结果算流量、排队这些指标。 中间那一棒**不是我做的**——数据接进来要解码、做目标检测跟踪、多传感器融合，那是算法团队的活。他们给我的是**结构化结果**：一条条参与者记录，带类型、经纬度、速度、车道号。

> 🔴 **红线**：绝不能说"我拉的视频流直接给了流量模块"。中间隔着解码和感知融合，一追问就穿帮。

追问 1：那你拉的数据到底发给谁了?

> 发到框架的**消息总线**上了，谁订阅谁收到。具体谁是订阅方属于感知融合那一块，是算法团队的模块，我没参与。我只保证这一棒是稳的——能连上、断了能重连、出问题能从指标上看出来。

追问 2：那为什么两边用的通信方式不一样?

> 不同团队的选型。主框架用一套通信中间件加 protobuf；交通参数检测那边的形态是**算法插件**，跑在一个宿主程序里，宿主给它的是 C 接口加另一套通信库，数据格式是 JSON。两边业务上靠**标准化的感知消息**衔接，通信各走各的。

追问 3：所以到底哪些是你做的?

> 我做的：多路视频流的拉取、设备的插件化接入、GPU 指标采集、部署工具，还有交通参数那边的检测器框架、过线判定、生产消费管道。**没做的**：图像解码、目标检测和跟踪、多传感器融合。

八股补给:

Q1：把一个大系统切成好几个模块，边界该怎么划?

> 判据是**看变化的方向**。会**一起变**的东西放一个模块里；**各自变**的东西拆开。比如"加一款新相机"只应该动驱动那一个文件，如果它还要动框架，那边界就划错了。 另一个判据是**依赖方向要单向**：下游认识上游，上游不认识下游。如果出现"A 要用 B、B 也要用 A"，那基本就是设计出了问题，得抽一层出来打破这个环。

Q2：模块之间怎么传数据才算"解耦"?

> 最低要求是**不直接认识对方**：不 include 对方的头文件、不调对方的函数、不知道对方的存在。中间靠一个**双方都认的东西**衔接——可以是一份消息定义（protobuf / JSON）、一个接口、或者一个消息队列。好处是谁重写都不影响另一边；代价是**多一层转换**（要序列化、要约定格式），以及**调用栈断了，调试时要靠日志去串**。

Q3：同一个进程内的两个模块，和两台机器上的两个服务，通信方式有什么不同?

> 进程内最简单——**直接函数调用**，或者传个对象指针，最快。但两边就要互相依赖，耦合度高。 跨进程或者跨机器，就必须走**通信中间件**：共享内存、消息队列、socket、或者 DDS 这类。它们都能做到"两边不认识"，代价是**多一次序列化 + 一次传输**。 所以选型的问题其实是：**你更在乎快，还是更在乎两边能独立演进。**

📌 ***自己复习：*** `6两个模块-合成一个故事.md` ***+ 图*** `imgs/07-两个模块怎么串成一条链路.svg`

条目一：设备抽象与插件化接入（8 题）

> **简历原话** 设备抽象与插件化接入：为相机等路侧设备定义统一抽象基类接口（初始化、启停、数据下发、状态查询），各厂商驱动通过虚函数重写实现同一套接口；配合工厂模式 + 注册宏完成驱动自注册与动态库加载，新增设备型号只需新增驱动文件、框架代码零改动，接入方式从"改代码"变为"改配置"。
>
> 🔴 **本条两条红线**
>
> 1. 全程说「**参与**」——接口和工厂是 mentor 定的架构，我负责具体模块落地。
> 2. "框架代码零改动"指的是**新增一款设备**不用改框架；**新增一类能力还是要动接口**，被追问要能说清这个边界。
>
> ❗ **"统一"是按设备类型各一套**（相机 / 雷达 / RSU / 信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类。** ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 协议层接入，**没对接任何厂商私有 SDK。**

技术点 1：统一接口与多态（1 题）

Q1-1 你说的「统一接入接口」，具体是怎么设计的? [必背]

答： 就三步。先写一个**抽象基类**，把四个**纯虚函数**列上去——初始化、启动采集、查状态、下发指令；再让每家厂商的驱动**继承**这个基类、把四个函数各自实现出来；

最后框架手里只拿**基类指针**，只管调，不管跑的是哪一份实现。 另外留了一条**回调**给数据上行：设备采到数据就丢给它，至于交给谁由框架定，驱动自己不知道。 能统一成这四个函数，是因为不管哪家设备，一辈子干的事都一样——先配资源、再出数据、中间要能查健康、有的还得能反向下发指令。差别只在每一步具体怎么做。



橙色回环 = 数据上行：驱动只负责把数据交给回调，不关心发给谁

> 看图要点：从上往下是「框架 → 工厂 → 抽象基类 → 具体驱动」，橙色回环是数据上行。上面三层只认**抽象基类**这一个类型，最底下那个驱动是后加的，框架一行都不用改。

> ❗ **别只答"虚函数基类 + 子类重写"**——那只讲了多态，没讲插件化。完整的是**三件套**：
>
> 1. **抽象基类**（纯虚函数）——定能力清单；
> 2. **工厂 + 注册宏**——驱动文件末尾一行注册宏，把"名字 → 构造函数"登记进表，**发生在 main 之前**；
> 3. **动态库加载**——驱动编译成独立 `.so`，启动时扫目录加载。
>
> ❗ **"统一"是按设备类型各一套**（相机、雷达、RSU、信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类**。被追问就诚实说：按类型分别统一，抽公共基类是后续优化点。
>
> ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 的协议层接入，一套代码通吃各家，**没对接任何厂商私有 SDK**。

追问 1：那这个接口在代码里是怎么落地的？

> 就是一个**抽象基类**，四个方法全是**纯虚函数**——基类只声明不实现，谁继承谁就必须自己写。框架手里拿的是基类指针，调的时候自动跑到具体那份实现上，这就是运行时多态。

追问 2：为什么正好是这四个方法？不能多不能少？

> 因为不管哪家设备，**一生的动作都一样**：先配置资源、再开始出数据、中间要能查健康、有的还要能反向下发指令。这四个覆盖了设备全生命周期，差别只在每一步怎么做。 ⚠️ 真遇到某家不支持其中某个，让它**空实现返回不支持**就行，不能为了迁就它去改接口——改接口会波及所有已接入的驱动。

追问 3：为什么用纯虚函数？普通虚函数不行吗？

> 不行。普通虚函数意味着基类**给了默认实现**，子类不写也能编译过——运行时就变成一个"什么都没干"的静默失败，现场查起来极其痛苦。纯虚函数把这件事**逼到编译期**：漏实现就编译不过。 而且基类本来也给不出有意义的默认实现——一台相机怎么初始化，基类怎么会知道。

追问 4：用基类指针销毁对象，要注意什么？

> **基类析构函数必须是虚函数。** 否则通过基类指针 delete 的时候，只会调到基类的析构，子类那部分资源就泄漏了——而且这种泄漏是静默的，跑几天才看出来。 这也是我判断一个"面向接口"的设计成不成熟时最先看的一点：**只要打算用基类指针持有对象，析构就必须虚。**

追问 5：那条回调为什么选择在构造函数里传进来？

> 这叫**构造期注入**。最大的好处是：回调变成对象一旦存在就**必然可用**，不会出现"建好了但还没设回调"这种半初始化状态。如果做成 setter，就多了一个必须记得调用的步骤，忘了就静默丢数据。

追问 6：这套接口最大的问题是什么？重做你会怎么改？

> 两个。一是**接口一旦定死，扩展就要动所有实现**，所以定接口得先想清楚粒度；二是**各类设备各有一套接口，没有公共基类**，同构代码重复了九份。重做的话我会先抽一层真正公共的基类，再往下按类型分化。

> ✅ **一句话版本（面试照背）**
>
> 它的核心确实是抽象基类和虚函数重写——接口层把能力定成纯虚函数，驱动继承下来各自实现，框架只拿基类指针调。但光有这层还不叫插件化：我在接口之外还配了**注册工厂 + 动态库加载**，驱动里一行注册宏把自己登记进工厂，框架侧只认配置里的一个名字。所以新增一个型号，是加一个驱动文件 + 一行注册，框架代码一行都不动——这才是我说的「统一接入」。

八股补给（面试官爱往底层挖的，答不上很伤）：

Q1：什么是多态？有哪几种？

> 同一个调用、跑出不同结果。分两种：**编译期多态**——函数重载、模板，编译时就定死了调哪个；**运行期多态**——虚函数 + 基类指针，运行时才决定调哪份实现。我这块用的是后者。

Q2：只有纯虚函数的类，能 new 出来吗？

> **不能**，这种叫**抽象类**（接口类），只要有一个纯虚函数没实现就不能实例化。它的用途就是当"规格说明书"给别人继承的。

Q3：那这个类里能写普通成员吗？

> 能。可以有普通成员函数、也可以有成员变量（比如我们这个基类里就有一个回调成员）。"全是纯虚函数"只是我们这个场景的选择，不是抽象类的要求。

📌 **自己复习（面试别说）：**

- 接口：`base/device_connect/ipcamera/device_base.h` 的 `IpCameraDevice`，四个纯虚 `Init / Start / GetState / WriteToDevice`，`protected: IpCameraCallBack sender_`，回调由构造函数注入。
- 子类：`standard_ipcamera.h:44` `StandardIpCamera : public IpCameraDevice`，四方法全 override。
- 工厂：`device_factory.h` 单例 + `map<string, CONSTRUCT>`；宏 `V2XOS_IPCAMERA_REG_FACTORY(StandardIpCamera, "standard_ipcamera")`，注册点 `standard_ipcamera.cc:608`。
- 框架取实例：`ipcamera_component.cc:42` `GetUnique(conf_.device(), ...)`；同文件 :31 动态加载 `device/lib/ipcamera/`。
- 同构共 9 套（camera / ipcamera / lidar / radar / rsu / traffic_light / mec / ins / cloud），每类一份 device_base.h + device_factory.h。
- 老的 `modules/camera/ipcamera/include/camera.h`（按 `Vendor` 分海康/大华/仿真）走厂商 SDK 路线，跟 standard_ipcamera 不是一条路，别混讲。

技术点 2：工厂 + 自注册（3 题）

Q2-1 为什么要用工厂模式？直接在框架里判断类型创建对象不行吗？ [必背] ★ 高频

答： 好处就一句话：**加一款新相机，框架不用动。** 不用工厂的话，框架里得写一串 **if-else**：配置里写 A 牌就 new A 牌那个类、写 B 牌就 new B 牌的——每来一款新相机都要打开框架加一个分支、整个重编一遍，而且框架得把各家驱动的**头文件**都 include 进来。 用了工厂，框架手里只有一张 **map**：**key** 是配置里写的设备名，**value** 是"这个对象怎么造"。

框架报个名字，工厂在 map 里一查、new 出来交给它，框架压根不认识背后是哪个**类**。 那这张 map 是谁填的？**驱动自己填的**——每个驱动文件末尾写一行**宏**（`#define` 那套），宏展开后是个静态对象，动态库一加载它就自动把自己的名字和造法塞进 map。所以新相机就是加一个 .cpp 文件，框架一个字不改。 打个比方：map 就像前台的取件码本子——你报码、他查本子、把包裹递给你，你不用知道仓库在哪。

追问：那这不就是把 if-else 挪到工厂里了吗？

> 区别在于**那张 map 是谁填的**。挪进工厂的话，工厂还是得认识所有牌子，加一款照样要改工厂。我们这个是**驱动自己往 map 里塞**——新相机写一个 .cpp、末尾一行宏，工厂完全不知道世界上有哪些设备。所以代价从"改工厂"变成了"加个文件"。

八股补给（这块面试官爱往底层挖，答不上很伤）：

Q1：什么是工厂模式?

> 就是把"new 对象"这件事交给专门一个地方，调用方只说要什么、不自己 new。三种常见写法：**简单工厂**（一个类里写 if-else）、**工厂方法**（每个产品配一个工厂类）、**抽象工厂**（造一整族产品）。我这个是简单工厂 + 一张 map，再加个自注册。

Q2：为什么用 map 存，不直接写 if-else?

> 为了**好扩展**。if-else 是写死的，加一种就要改一次代码、重编一次；map 是运行时查表，加一种只是往表里多塞一条。查找上 map 是红黑树、O(log n)，几十种设备根本不是瓶颈。

Q3：你说的工厂单例是什么意思？为什么用它，怎么实现？

> 单例限制的是**工厂对象只有一份**，不是只能创建一种设备。这个对象维护一张 **map**，把设备名和创建方法放在一起，让驱动注册和框架查找共用同一张表。实现时用入口函数返回**局部静态工厂对象的引用**，每次拿到的都是同一个对象；再把构造函数设为私有、禁止复制和移动，限制外部另建对象。一个工厂仍然能创建多个设备实例。

追问：局部静态对象是不是天然线程安全，map 就不用加锁了？

> C++11 起保证的是**局部静态对象的初始化**，多个线程同时第一次调用，也只会完成一次初始化。这个保证不自动覆盖后续的 map 操作；注册和查询如果并发，仍然需要同步，不能把「单例初始化安全」当成「整个工厂所有操作都安全」。

> ❗ **别把「类」和「对象」说反**：类是定义，对象是按定义创建出来的实例；单例限制的是对象数量。map 是容器，不是函数，里面存的是创建方法；工厂单例是本项目的选择，不是所有工厂模式都必须单例。

> 📌 *自己复习（面试别说）：`base/device_connect/ipcamera/device_factory.cc:24` 的 `Instance()` 返回局部静态对象的引用；`device_factory.h:69` 起是私有构造和禁止复制、移动，`:76` 是注册表。构造函数私有负责限制外部创建，返回引用避免把工厂复制一份；设备实例由 `Produce` 调用表中的创建方法生成。*

Q4：宏（#define）有什么坑？

> 三个：**没类型检查**、**不好调试**（报错报的是展开后的代码）、**容易命名冲突**（所以我们宏里要用 `##` 把类名拼进变量名，不然多个驱动会重复定义）。这是我觉得这套设计里最该被换掉的部分。

Q5：那单例有什么缺点？

> 本质是个全局变量——测试的时候不好替换，依赖关系也被藏起来了。不是什么东西都该做成单例，只有**确实全局唯一的**才值得。

> 📌 *自己复习（面试别说）：工厂是 `device_factory.h` 单例 + `map<string, 构造方式>`；抽象基类 `IpCameraDevice` 四个纯虚方法；宏 `V2XOS_IPCAMERA_REG_FACTORY(...)` 注册点 `standard_ipcamera.cc:608`；框架侧 `GetUnique(conf_.device(), ...)` 只按配置名取，返回基类指针。其余术语（运行时多态 / 编译期依赖 / 自注册 / 静态对象 / 动态库加载）心里有数就行，口播不抛。*

Q2-3 你说的「一行注册宏」，它到底是怎么工作的？ [必背] ★★ 高频

答： 每个驱动文件末尾写一行宏。

这行宏展开之后，其实是定义了一个**静态对象**，它的**构造函数**干的事就是：把自己的设备名和"怎么造自己"塞进工厂那张 map。 关键就在"静态"两个字——C++ 保证静态对象的构造函数会在程序加载时**自动执行**，不需要谁手动去调。所以框架把驱动加载进来，注册就自动完成了，不需要在任何地方维护一份"设备清单"。

追问：但是静态对象的初始化顺序是不确定的，会不会出问题？

> 这个担心是对的，C++ 确实不保证不同 `.cpp` 之间静态对象的初始化顺序。但这里恰好安全，关键在于**工厂是用函数内的局部静态变量实现的单例**——用到的时候才构造，所以不存在"注册跑到一半、工厂还没建好"。 反过来如果工厂是个全局变量，注册可能在它构造之前发生，往一个还没初始化的 map 里塞东西就是**未定义行为**。

八股补给：

Q1：宏展开之后大概长什么样？

> 大致是 `static 某某注册器 g_某某注册器("设备名");`——一个静态对象，构造函数里干正事。那个变量名靠 `##` 把类名拼出来，保证每个驱动不重名。

Q2：为什么非得用宏？写个普通函数不行吗？

> 宏的好处是**一行搞定、不容易写漏**。但代价很实在：**没类型检查**、**报错信息是展开后的代码**、**调试器里看不到原样**。所以宏不算好设计，只是当时"让驱动作者少写几行"这个收益更大。要改我会换成显式注册函数。

Q3：你说的"静态对象"和"静态成员变量"是一回事吗？

> 不是。这里是**文件作用域的静态对象**（全局对象加了 `static`，作用域只限本 `.cpp`）；静态成员变量是类里的、属于类而不属于某个对象。两个概念别混着讲。

追问 1：静态对象到底什么时候构造？动态库呢？

> 可执行文件里的静态对象，在 **main 之前**就构造好了；动态库里的是**加载时才构造**——也就是框架扫描目录、把 `.so` 加载进来的那一刻。所以注册这个动作发生在框架真正开始工作之前，等它去查表时表里已经有东西了。

追问 2：宏里面要用符号拼接生成变量名，为什么？

> 因为宏在**每个驱动文件里都要用一次**，如果不拼接就会生成同名的全局变量，链接期直接报重复定义。拼接上类名之后，每个驱动拿到的是自己那份独一无二的变量名。

追问 3：工厂是单例，多线程同时调用安全吗？

> 要分开看：**工厂对象第一次初始化**由局部静态变量机制保证；后续注册表的读写，仍然要看有没有并发修改。只读查询和一边注册一边查询不是同一种情况，需要同步时就加锁或约束注册时机。当前这份工厂代码里没有给 map 额外加锁，不能说它因为用了单例就自动保证所有调用都线程安全。

追问 4：如果两个驱动注册了同一个名字会怎样？

> 后加载的那个会把先来的**覆盖掉**，运行时的现象是"配了这个名字但行为不对"，而且不报错、很难查。所以更稳妥的做法是**重复注册时直接报错退出**，而不是静默覆盖。

追问 5：宏用起来爽，代价是什么？

> 调试很痛苦——宏展开后的代码和看到的不一样，报错信息也难读。而且它是**编译期**的黑魔法，出了问题只能看预处理后的文件排查。这是我认为这套设计里最该被替换掉的部分，更好的做法是改成显式注册函数。

追问 6：注册宏里的 `static` 到底起什么作用？去掉会怎样？

> 去掉也**照样能注册**——全局变量不加 `static` 一样是静态存储期，加载时照样构造。`static` 管的不是"何时构造"，是**链接属性**：加了就是内部链接，变量名只在本 `.cpp` 可见；不加就是外部链接，符号暴露给整个进程，两个驱动万一撞了同名变量就链接冲突。所以它是**防重名的保险**，不是自注册成立的条件。 自注册真正的地基是"**全局对象**"这个身份——只有全局对象才能在加载时自动构造、自动完成注册；要是写成函数里的局部变量，就得有人去调那个函数才构造，没人调就永远不注册。

> 📌 *自己复习（面试别说）：存储期（何时构造/析构）由"全局对象"身份决定，与 `static` 无关；`static` 只提供内部链接防跨 `.so` 撞名。宏定义在 `device_factory.h:91`。*

追问 7：宏展开、模板实例化、注册进 map，这三件事分别发生在什么时候？★ 自己曾搞混

> 三件事分属**两个阶段**，别揉在一起： **编译期两件**：① 宏展开——预处理，纯文本替换，把一行宏变成静态对象定义；② 模板实例化——编译时，编译器拿具体类名套模板生成注册器的机器码，创建函数里 new 谁在这就**定死**了。这两件只是"**备货**"——把字符串、代码、"加载后执行"的记录（`.init_array`）打进 `.so`，**此刻什么都没注册。 运行期一件**：加载 `.so` 时静态对象构造，构造函数把"名字 → 创建方法"**真正写进 map**——对应关系到这一刻才形成。 一句话：**编译期定死"代码长什么样"，运行期决定"数据是什么、走哪条路"**。map 是运行期的内存对象，编译期它根本不存在，所以不可能提前写进去。好比编译期是印菜谱（哪页哪道菜印刷时定死），运行期是开门营业（点什么、做什么现决定）——**编译期定死"说明书内容"，运行期定死"取哪本说明书"**。

Q2-8 用具体流程说说：新来一款设备，你要做哪些事？ [必背] ★★ 高频场景题

答： 四步，**全程不碰框架代码。** 第一，新建一个 `.cpp`，写一个类**继承**那套统一接口，把那几个纯虚函数实现出来； 第二，文件末尾加一行注册宏，名字填"厂商 + 型号"； 第三，把这个模块编译成独立的 `.so`，丢到框架会去扫描的那个目录下； 第四，改配置，把里面的设备名从老驱动改成新的。 重启之后框架扫描目录、加载 `.so`、注册自动完成，就按新配置创建实例了。**框架代码一行不改，也不用重新编译框架本身。**

追问：这套流程的瓶颈在哪？

> 两个。一是**接口是有限的**，遇到厂商有特殊能力就得回头扩展接口；二是**出问题只能靠日志排查**，没有"已注册清单"的时候比较绕。这两个都是我前面说的改进点。

八股补给：

Q1：编译成 .so 要加哪些选项？

> 两个都要：`-fPIC` 和 `-shared`。`-shared` 告诉链接器产出的是动态库而不是可执行文件；`-fPIC` 生成**位置无关代码**。64 位下少了 `-fPIC` 基本链接不过。

Q2：为什么动态库必须用 -fPIC?

> 因为同一个 `.so` 会被加载到**不同进程的不同地址**，代码里不能写死绝对地址。`-fPIC` 让所有跳转和取址都走**相对偏移**或者 GOT 表，这样放哪儿都能跑。

Q3：静态库（.a）和动态库（.so）怎么选?

> 静态库在**链接期**就被整个拷进可执行文件——程序变大，但部署简单、不依赖外部环境；动态库在**运行期**才加载，多个程序能共用一份，更新也不用重编主程序。我们选动态库，就是为了"加设备不用动框架"这一点。

技术点 3：动态加载（3 题）

Q3-2 加载失败会有什么现象？你怎么排查？ [加分] ★ 有实战感

答： 现象是**设备名查不到**——配置里明明写了这个名字，但创建实例的时候失败，打一条错误日志。 麻烦的是失败原因不会直接暴露，要用 `dlerror()` 才能拿到人话的错误信息。我一般三步排查：用 `ldd` 看这个库是不是缺了某个依赖；确认那个依赖在搜索路径里能找到；把依赖补上，或者在加载目标库之前**先显式地把依赖加载进来**。

追问：你实际遇到过吗?

> 遇到过。有次一个 `.so` 始终加载失败，现象就是它的设备注册不上。后来发现它依赖了某个第三方库，而那个库当时还没被加载。解决办法是加载驱动之前先检查依赖在不在，不在就按顺序先加载。这类问题**编译期完全发现不了**，只能靠启动日志。

八股补给:

Q1：`ldd` 是干什么的?

> 列出这个 `.so`（或可执行程序）依赖哪些库、以及每个库**实际解析到了哪个路径**。输出里出现 `not found` 就是缺依赖，这是最常见的一类问题。

Q2：找不到库的时候，系统会去哪儿找?

> 大致顺序：可执行文件或库里写死的 `RPATH` / `RUNPATH` → 环境变量 `LD_LIBRARY_PATH` → `/etc/ld.so.cache`（由 `ldconfig` 生成）→ 最后才是 `/lib`、`/usr/lib` 这些默认路径。所以现场要么改环境变量，要么把库放到标准路径后跑一次 `ldconfig`。

Q3：为什么这类问题编译期发现不了?

> 因为编译期只检查"你代码里调的函数有没有声明"，而动态库加载是**运行期**的动作——库在不在、依赖全不全，只有真正加载那一刻才知道。这正是"灵活"换来的代价。

Q3-6 从动态加载到启动拉流，整体顺序是什么？ [必背] ★★

答：

框架先扫描驱动目录，按描述文件加载对应的动态库，库里的**静态注册器**随之完成登记。随后读取组件配置，按设备名到工厂查创建方法，创建**具体子类对象，用基类指针持有**。接着通过多态调用初始化接口，读取 RTSP 地址、重连间隔等驱动配置，再调用启动接口，为各路视频流建立拉流线程。线程负责**拉流、拆包、封装消息**，通过之前传入的回调交给组件发布。

追问：动态库一加载，就已经把相机设备对象创建出来了吗？

> 还没有。加载时先构造**注册器对象**，把设备名和创建方法放进工厂的 map；框架按配置中的设备名查表，调用创建方法时，才生成真正的设备对象。

> ❗ **这三处别说反**：① 扫目录后按描述文件选库，不是无差别加载所有 `.so`；② 创建的是子类对象，基类指针负责持有和调用，抽象基类本身不能实例化；③ 驱动配置里的 RTSP 地址等参数由子类初始化接口读取，启动接口再开启拉流线程，本驱动不做图像解码。

> ✅ **一句话流程（自己复习）**

```
扫描驱动包、按描述加载 .so
    → 静态注册器把“设备名 → 创建方法”写进工厂 map
    → 读取组件配置，确定设备名和驱动配置路径
    → 工厂查表，创建子类对象，并传入数据回调
    → 基类指针调用 Init，读取驱动配置并初始化
    → 基类指针调用 Start，启动每路拉流线程
    → 拉流、拆包、封装 → 调用回调 → 组件发布到 CyberRT
```

> 📌 *自己复习（面试别说）：`base/plugin/modules_loader/dynamic_loader.cc:15` 起扫描驱动包，读取 `device_lib_cfg.pb` 和 `so_name`；`library-holder.cc:6` 起实际使用 `dlmopen`，先以 `RTLD_NOLOAD` 检查，再以 `RTLD_NOW` 加载。`middleware/device_service/framework/ipcamera/ipcamera_component.cc:29` 起串起加载、组件配置、工厂创建、`Init`、`Start`；`standard_ipcamera.cc:114` 的 `Start` 遍历各路流，`:148` 的 `StartStream` 建立线程。`dlmopen` 可指定加载命名空间，具体是否新建命名空间由加载器参数决定，不能只看函数名就断言启用了隔离；设备自注册这条链路不需要再用 `dlsym` 查创建函数。*

八股补给：

Q1：加载动态库一共要用到哪几个接口？

> 四个，都声明在头文件 `dlfcn.h` 里：`dlopen`——打开并加载一个 `.so`，返回一个句柄（一个 `void*`），代表"这个库在这个进程里的一份实例"。`dlsym`——**按名字在库里查一个符号的地址**，函数或者全局变量都行。`dlclose`——关掉句柄，引用计数减一。`dlerror`——拿上一次失败的人话错误信息，没有错误就返回空。 还有个容易忘的：**编译的时候要链** `-ldl`，不然会报这几个接口找不到。

Q2：`RTLD_NOW` 和 `RTLD_LAZY` 有什么区别？

> 它俩是 `dlopen` 的第二个参数，决定**什么时候去解析库里那些还没找到的符号。** `RTLD_NOW`：加载时就**全部解析完**，有一个找不到就整个加载失败。启动慢一点点，但**问题在启动那一刻就暴露。** `RTLD_LAZY`：**用到哪个才解析哪个**，加载快，但可能程序跑了一半、调用到某个没解析的函数时才炸。 长期跑的服务基本都选前者——**早点失败，比半夜崩好。**

Q3：`dlsym` 是干什么的？我们这个项目里用到了吗？

> 它是"**按名字在库里找东西**"的接口。编译的时候你根本不知道这个库将来会被加载到哪个地址，所以只能运行期拿着名字去查，查回来一个函数指针再调用。 **设备驱动这块没用到它**——驱动是靠库里那个全局静态对象**自己报到工厂**的，框架不需要知道库里有什么函数。 而**算法插件那块用到了**：那边是显式地按名字去取一个"创建对象的函数"，取到之后才能造对象。两种路子的区别一句话：**一个是库主动报到工厂，一个是框架主动去库里找。**

Q4：动态库加载进来之后，代码和数据是共享的还是每个进程一份？

> **代码段是共享的**：同一个 `.so` 被十个进程加载，物理内存里**只有一份代码**，靠内存映射挂到各自的地址空间——这是动态库省内存的主要原因。 **数据段是每个进程各一份**：库里的全局变量、静态变量，每个进程都有自己的副本，改了互不影响。 所以**别想靠动态库的全局变量做进程间通信**，那玩意儿根本不共享。

Q5：用 `dlerror` 拿错误信息有什么要注意的？

> 它返回的是**最近一次失败的**原因，而且**取一次就清空了**。所以必须**紧跟在失败的那个调用后面立刻取**，中间不能再插入别的动态加载调用，否则拿到的就是别的错误、甚至是空。 另外它返回空**不等于一定成功**，只代表"这次没有新错误"。判断成败还是要以 `dlopen` 的返回值是不是空为准。

Q3-7 组件的 Init 是谁调用的？ [必背] ★

答：是 **mainboard** 调的。mainboard 是运行框架的组件加载进程，启动脚本里每个模块都是一条 `mainboard -d xxx.dag`。它读 dag 配置——里面写着组件库的路径、类名、订阅通道、配置文件路径——然后**加载组件的 `.so`、按类名实例化、调 Init**；之后订阅的通道每来一条消息，就调一次 Proc。 有意思的是，**组件被 mainboard 加载的方式，和驱动被组件加载是同一套**：都是动态库 + 注册宏 + 配置指定类名。组件对 mainboard 是插件，驱动对组件也是插件——同一个模式套了两层。

追问：那从敲启动命令到数据跑起来，整条链是什么？

> 启动脚本拉起 mainboard 并指定 dag → mainboard 按 dag 加载组件库、实例化、调 Init → Init 里加载驱动库（驱动靠静态对象自注册进工厂）→ 按配置里的设备名让工厂造出设备对象 → 调驱动的初始化（读 RTSP 地址等配置）和启动（每路流一个拉流线程）→ 采到数据走回调交给组件发布。**"谁启动谁"的答案也是配置，不是代码。**

> 📌 *自己复习（面试别说）：`script/airos_keep_alive.sh:59` 起每模块一条 `mainboard -d dag/xxx.dag`；`framework/ipcamera/dag/ipcamera.dag` 里写 module_library、class_name、readers、config_file_path；组件注册宏是 `middleware/runtime/src/cyberrt_component.h:36` 的 `CYBER_REGISTER_COMPONENT`，和设备注册宏同一套路。*

技术点 4：改配置不改代码（1 题）

Q4-1 你说的「改配置不改代码」，具体是怎么实现的？ [必背] ★

答：

我们把**驱动选择和运行参数放进配置文件**。框架启动时加载驱动库并完成注册，再根据配置里的设备名，通过工厂创建对应的驱动对象。切换已经支持的驱动、增加同一驱动下的视频流，或者调整地址和重连间隔，就改相应配置，重启服务后读取生效。新增尚未支持的驱动仍然要写代码、编译和部署动态库，只是不需要修改**框架代码**，不能理解成填个名字就能接入任意设备。

追问：你一直说「框架」，那框架具体是什么？

> 框架就是把**通用流程和接口提前组织好的一套代码**，具体模块按约定接进去。在我们这个设备接入场景，它负责加载驱动、读取配置、通过工厂创建对象，再调用统一的初始化和启动接口；驱动负责相机连接、拉流、拆包和重连，数据准备好后通过回调交回组件发布。新增驱动沿用这套流程，就不用把通用部分再写一遍。

> ❗ **两个名字别混**：配置中的「设备名」主要用来选择驱动实现；RTSP 地址用来指定连接哪台相机。对应驱动必须已经实现、部署并注册，不能只在配置里凭空加一个名字。

自己复习：常见四种修改

想改什么	实际要做什么
改重连间隔	改驱动配置中的参数，重启相应服务
同一种驱动再接一路相机	增加 RTSP 地址和对应输出话题等配置
切换已经支持的驱动	确保动态库已部署、加载并注册，再改组件配置中的驱动名
新增尚未支持的驱动	实现接口、添加注册、编译并部署动态库，再改配置；框架代码不用跟着加设备判断

> 📌 *自己复习（面试别说）：配置分三层——进程编排决定启动哪些组件及其配置入口；组件配置决定用哪个驱动、驱动配置在哪里；驱动配置决定 RTSP 地址、输出话题和重连间隔等。对应 `middleware/device_service/framework/ipcamera/dag/ipcamera.dag`、`conf/config.pb.txt`、`conf/ipcamera_config.yaml`；组件 `ipcamera_component.cc:34` 起完成加载、读配置、工厂创建及初始化，驱动 `standard_ipcamera.cc:43` 起解析运行参数。当前流程在初始化时读配置，不代表支持实时热更新。*

八股补给：

Q1：这些配置文件一般用什么格式？

> 常见三种：**JSON、Protobuf 文本配置、YAML**。交通参数模块用 JSON，相机驱动参数用 YAML，组件配置按 Protobuf 定义解析。Protobuf 有明确的字段约定，但配置文件里的拼写、类型和值是否有效，仍要在读取时检查，不是所有错误都能在编译期发现。

Q2：配置读进来之后放哪？

> 看模块的设计，不是所有配置都放全局单例。**相机组件配置保存在组件对象里，驱动参数保存在驱动实例里**；交通流量模块则有自己的全局配置管理，检测器再保存所需的车道信息。当前这些路径主要在初始化时读取，所以改完文件通常要重启相应服务。

Q3：为什么只在启动时读一次?

> 两个原因：一是**没必要**，每次用都去读文件纯浪费；二是**避免状态不一致**——读到一半配置变了，前面的模块用老值、后面的用新值，这种 bug 极难查。真要热更新就得整体切换，不能改一半。

条目二：多路视频流接入（7 题）

> **简历原话** 多路视频流接入：参与多路 RTSP 视频流的拉取与链路调试，每路流由独立线程承载，基于 FFmpeg 完成拉流与拆包并封装为 protobuf 消息统一发布；支持断线自动重连（外层重连、内层读帧的双层循环）与帧间隔统计，便于现场定位链路抖动。
>
> 🔴 **本条两条红线（最容易穿帮）**
>
> 1. **绝不说"解码"**——代码里**没有解码环节**，只说拉流 + 拆包 + 封装。
> 2. **绝不说"我拉的流直接给了流量模块"**——中间隔着解码 + 目标检测跟踪 + 融合，那是算法团队的活。
>
> ⚠️ **背诵优先级**：真正要背的只有三句——① 这模块干嘛的 ② 核心实现（每路一个线程 + 双层循环）③ 我具体参与了什么。

技术点 1：整体流程（1 题）

Q1请介绍一下你项目中相机设备接入的完整流程，数据是怎么从相机到下游的?

A： 我们的系统是部署在路口 MEC 边缘计算设备上的。相机通过 有线网络 接入 MEC，相机机内持续录像，并把原始像素实时压缩成 H.264 码流，但它处于 待机状态，不会主动推流 。

接入流程是：MEC 读取配置表拿到相机的 RTSP 地址 （包含 IP、端口、账号密码和流路径），先与相机建立 TCP 连接 ，在 TCP 上走 RTSP 控制协议 发送 SETUP、PLAY 等指令；相机收到 PLAY 后，把 H.264 码流切块装进 RTP 包 ——包头带 序号和时间戳 ，用于接收端组包和检测丢包——然后按协商的方式经 UDP 或 TCP 传给 MEC；当前驱动没有显式固定传输方式。

MEC 这边 每路流由独立线程承载 ，基于 FFmpeg 完成拉流和拆包——拆开 RTP 拿到 H.264 压缩帧——再 封装成 protobuf 消息 统一发布到 CyberRT 消息总线 。链路可靠性上做了 断线自动重连 ：外层重连、内层读帧的双层循环，外加帧间隔统计，方便现场定位链路抖动。

下游感知服务订阅总线消息后做 解码 ，上 GPU 完成 YUV 转 RGB、目标检测、跟踪和多传感器融合，产出结构化障碍物数据；最后按 V2X 标准编码成 RSM/SPAT 等消息，通过 UDP 发给 RSU ，由 RSU 通过 PC5 无线广播给过往车辆的 OBU ，实现闯红灯预警、绿波车速引导这类车路协同应用。

Q1-1 多路视频流，从启动线程到读包发布，整体流程是什么? [必背] ★★ 总览题

答：

启动时遍历流表，**每路创建一个独立线程**。线程里是**双层循环**：外层检查停止标志，打开 RTSP 输入、探测流信息并找到视频流；内层持续读包，把压缩数据和时间戳等填进 Protobuf 消息，通过回调交给组件发布，并统计发送间隔。**每处理完一包，就释放原包的数据引用**。读包失败后退出内层、关闭输入；如果没有要求停止，就等待一段时间重新连接，本驱动不做图像解码。

按你复述的顺序，对照代码看：

```
Start 遍历 streams_，对每路调用 StartStream
    ↓
StartStream 创建线程，执行 TaskPullRtspStream
    ↓
外层检查“本路停止标志”和“整个驱动停止标志”
    ↓
ConnectRtspStream：open_input → find_stream_info → 找到视频流索引
    ↓ 成功
初始化 AVPacket，进入内层循环
    ↓
av_read_frame 读包，检查返回值及 stream_index
    ↓ 是视频包
创建 CompressedImage，复制压缩数据，填写格式、帧编号、关键帧、时间戳
    ↓
sender_(话题名，消息) → 组件回调 → Send → Writer::Write
    ↓
av_packet_unref：每个成功读出的包都要释放，包括被忽略的非视频包
    ↓
继续读下一包；停止或读取失败时退出内层
    ↓
avformat_close_input 关闭输入；未停止就等待后回到外层重连
```

追问 1：流表里存的是每一帧图像吗？

> 不是，是**每一路流的管理信息**，比如 RTSP 地址、输出话题、连接状态、线程和统计。图像消息在读包循环里另外创建，同一路话题会持续发布很多条消息。 👉 流表和线程组织见 **Q2-6**。

追问 2：你说打开 RTSP 输入，那和 RTP 是什么关系？

> **RTSP 管会话控制，RTP 承载媒体数据**，底层可以采用 UDP，也可以通过 TCP 交错传输。驱动调用 FFmpeg 打开输入、探测流信息，再记下视频流索引，具体协议细节由库处理，不能只凭 RTSP 地址就断定一定走 UDP。 👉 FFmpeg 调用顺序见 **Q2-1**。

追问 3：Protobuf 消息类从哪里来？读到包以后怎么装进去？

> 事先用 **.proto 定义字段**，构建时生成 C++ 消息类；运行时创建消息对象，把压缩数据和其他字段填进去。这一步是在构造内存里的消息对象，**还不是序列化成字节流**。 👉 封装字段见 **Q2-2**。

追问 4：回调什么时候真正触发发送？

> **消息填好后，拉流线程调用保存的回调**，就执行之前绑定的组件方法。组件再调用发布接口，把对象交给 CyberRT；后续通信需要的序列化由中间件处理，不是驱动在回调里手写序列化。 👉 回调注入和发布见 **Q2-3**。

追问 5：数据只是读出来复制，为什么每包还要释放？

> 因为**有没有修改数据，和内存要不要释放，是两件事**。消息里已经保存了副本，原包的数据引用就可以释放；释放不代表把数据还给相机，也不会影响独立的消息副本。 👉 每包释放的位置见 **Q2-1**。

追问 6：断流以后怎么重连？会一直重试吗？

> 读包失败就退出内层，关闭旧输入、等一段时间，再回外层尝试连接。**只要没有收到停止标志，就继续尝试**；不能说线程永远不退出。每路还记录发送间隔，帮助排查连接恢复后的抖动。 👉 重连见 **Q2-4**，发送间隔统计见 **Q2-5**。

> 📍 **简历术语 → 题号**：**拉流、拆包** → Q2-1；**Protobuf 封装** → Q2-2；**回调、CyberRT 发布** → Q2-3；**断线重连、双层循环** → Q2-4；**帧间隔统计** → Q2-5；**每路独立线程** → Q2-6。

> 📌 *自己复习（面试别说）：`standard_ipcamera.cc:114` 的 `Start` 遍历流表，`:148` 的 `StartStream` 建线程，`:225` 的 `TaskPullRtspStream` 执行双层循环；`:373` 打开输入，`:379` 探测流信息，`:390` 起找视频流索引；`:250` 初始化包，`:253` 循环读取，`:267` 创建消息，`:293` 起复制压缩数据，`:324` 调用回调，`:343` 释放包引用，`:348` 关闭输入。`ipcamera_component.cc:97` 起是发布回调；`middleware/runtime/src/air_middleware_component.h:44` 起的 `Send` 获取发布者并调用 `Write`。函数名是代码对照，不用整串塞进口述答案。*

八股补给：

Q1：为什么用智能指针，不用裸指针?

> 三个理由：一是**不用手动 delete**，多线程下不用猜"还有没有人在用"；二是**异常安全**，中间抛异常也能自动释放；三是**所有权写在了类型里**——`shared_ptr` 表示多方共同持有，看代码就知道谁在用。裸指针你得自己记得什么时候能删，多线程下极易出错。

Q2：`shared_ptr` 的引用计数是线程安全的，那还需要加锁吗?

> **需要。** 计数本身的加减是原子的、线程安全，但**它指向的那个对象不是**——多个线程同时读写对象里的成员，照样要加锁。这是个很常见的误解，别以为套了 `shared_ptr` 就万事大吉。

Q3：`shared_ptr` 最大的坑是什么?

> **循环引用**：A 持有 B、B 又持有 A，两边计数都降不到 0，谁也释放不掉，内存泄漏。解法是用 `weak_ptr` 打断这个环——它**不增加计数**，只是"看一眼还在不在"。

技术点 2：六个实现点（6 题）

Q2-1 用 FFmpeg 拉流和拆包，具体是怎么实现的? [必背] ★★

答：

我先让 FFmpeg **打开 RTSP 输入、探测流信息**，再找到视频流的编号。连接准备好后，循环读取压缩媒体包，检查读取结果，只把视频包的数据复制进消息。每个成功读出的包处理完，都要**释放原包的数据引用**，和读包一

样逐包执行；断流时关闭输入，由外层等待后重连。网络协议处理由 FFmpeg 完成，我负责调用接口、筛选和封装，不在驱动里把视频解码成像素。

追问：你说「拆包」，和 TCP 粘包、半包是一回事吗？

> 不是。TCP 的问题是**字节流没有业务消息边界**；RTP 则有自己的视频分片、聚合和传输封装，UDP 报文边界清楚也不代表一份视频数据只占一个报文。这份驱动没有自己写 RTP 重组逻辑，而是用 FFmpeg 读取处理后的媒体包。

自己复习：这份驱动实际调用的 FFmpeg 接口

阶段	函数	作用
初始化	`av_register_all`	旧版格式注册入口
初始化	`avformat_network_init`	初始化网络相关支持
连接准备	`avformat_open_input`	根据 RTSP 地址打开输入
连接准备	`avformat_find_stream_info`	探测媒体流和编码参数
调试	`av_dump_format`	打印输入流信息
开始读包前	`av_init_packet`	初始化包结构中的字段，属于旧接口
内层逐包执行	`av_read_frame`	读取压缩媒体包，不是直接返回像素图像
每个成功读出的包处理后	`av_packet_unref`	解除包对数据缓冲区的引用并重置字段
断流、停止或准备失败	`avformat_close_input`	关闭输入、清理资源

> ❗ **不要背反**：读包和释放都在循环里逐包进行，不是等全部读完才释放。打开、探测等连接准备工作在每次建立连接时执行，重连也会重新执行；不是整个进程一生只调一次。

八股补给：

Q1：FFmpeg 主要分哪几个库？这份代码用了什么？

> `avformat` 主要处理输入格式和协议、打开输入及读取媒体包，`avcodec` 管编解码及包等相关接口，`avutil` 提供基础工具，`swscale` 用于缩放和像素格式转换。这份驱动用到了输入读包和包管理接口，**没有实现把压缩视频解码成像素的流程**，不能凭「调用了某个库」就判断有没有解码。

Q2：打开 RTSP 输入，底层会处理哪些会话步骤？

> 常见流程包含查询能力、获取 **SDP 媒体描述**、协商传输通道和请求播放，具体由 FFmpeg 与服务器交互完成。探测流信息还可能读取一些媒体数据来补充参数，不能把探测函数简单等同于「只解析一次 SDP」。应用侧检查打开和探测的返回结果，再确认是否找到了视频流。

Q3：RTSP、RTP 和 UDP / TCP 怎么区分？

> **RTSP 管会话控制，RTP 承载媒体数据，UDP / TCP 是底层传输方式。**当前打开输入的调用没有显式指定 `rtsp_transport`，实际走哪种要看协商或运行日志。`av_read_frame` 返回 FFmpeg 的压缩媒体包，不是让应用直接拿原始 RTP 报文自己解析。

Q4：`av_read_frame` 读到的是一帧吗？

> 按 FFmpeg 接口对视频包的正常约定，可以理解为**一帧的压缩编码数据**，还不是像素图像。一个编码帧可能涉及多个 NALU、多个 RTP 报文，FFmpeg 会进行相应的输入处理，所以不能拿「一个 NALU 不一定是一帧」来直接推导输出包的边界。还要通过流编号区分视频包和音频包。

Q5：没修改包里的数据，为什么还要调用 `av_packet_unref`？

> **只读也会占着内存，释放和修改无关。**读包后，包持有本地数据缓冲区的引用；压缩数据复制进独立消息后，就可以解除原包引用，没有其他引用时缓冲区才会释放。非视频包即使直接忽略，也要释放，所以这一步放在视频筛选判断之外。

> 📌 ***自己复习：实际调用集中在 `middleware/device_service/modules/ipcamera/standard_ipcamera/standard_ipcamera.cc:196` 起的初始化、`:225` 起的读包任务、`:362` 起的连接函数。`av_register_all`、`av_init_packet` 是本代码使用的旧接口，不说成所有新版项目的必需调用。协议和接口参考：H.264 的 RTP 分片、FFmpeg 4.4 读包接口、AVPacket 引用释放、RTSP 传输配置。***

Q2-2 拿到一个包之后，怎么封装成 protobuf 消息的？ [必背] ★★

答： 分两步，**编译期定图纸、运行期填字段。** 编译期先用一份**描述文件**（.proto）定义这条消息有哪些字段——帧 id、格式、是不是关键帧、曝光时间戳、数据本体、以及一个 header（时间戳、模块名、序列号等），编译时自动生成对应的 C++ 类。 运行期每读到一个包，就 `new` 一个 `CompressedImage` 对象，把字段一个个填进去：帧 id 用每路独立递增的计数器、格式写 `h264`、关键帧检测结果填 `frame_type`、从 SEI 解析出的曝光时间戳填 `exposure_time`，最后把**压缩数据原样拷进 `data` 字段，一个字节都不动。** 消息对象用 `shared_ptr` 管理，便于需要继续使用它的下游保留所有权；但压缩数据填入消息时确实发生了复制，不能把共享指针直接说成整条链路零拷贝。

追问：为什么要分"编译期图纸 + 运行期填字段"两步？

> 因为要**跨进程、跨语言。**描述文件是双方共同认可的一份契约：我这边生成类、填字段、序列化成字节发出去；下游用同一份描述文件生成的类去反序列化。只要两边都认这份契约，C++ 发的别的语言也能收。如果只是程序内部随便定义个结构体，就没法保证下游能正确解析。

八股补给：

Q1：为什么用 protobuf，不直接把内存里的结构体发出去?

> 直接发结构体踩四个坑：**对齐填充**（编译器塞空字节）、**字节序**（大小端）、**指针**（发过去是个地址没用）、**版本**（加字段后老程序全错位）。protobuf 按**字段编号**认，不认识的编号跳过，加了字段老程序照样能读，而且体积小、跨语言。

Q2：为什么要用 `shared_ptr` 传这一帧，不直接传对象?

> 一是**共享已有消息，不必为每个持有者复制整个对象**；二是**管理生命周期**，需要继续使用消息的一方可以保存一份共享指针，最后一个持有者释放时才销毁对象。复制共享指针会增加引用计数，按 `const shared_ptr&` 传参本身不会增加；这不代表封装视频数据的过程没有复制，也不保证对象内容的并发安全。

Q3：序列化和反序列化是什么?

> **序列化**是把内存里的对象变成一串可传输的字节；**反序列化**是把字节还原成对象。为什么不能直接发内存里的对象——因为那串字节是本机内存布局，换个机器、换个语言就完全看不懂了。序列化约定一个双方都认的格式，把差异抹平。

Q4：为什么要检测"是不是关键帧"、还要解析 SEI 曝光时间戳?

> **关键帧（I 帧）**是能独立解压的完整帧，下游解码必须从它起步，所以要在消息里用 `frame_type` 标出来。**SEI** 是码流里带额外信息的 NALU（类型 6），相机会把**曝光时刻**塞进去，我解析出来填进 `exposure_time`，下游拿到的是"这一帧真实采集的时刻"，比"收到包的本地时间"准——因为网络有抖动，收到的时间会漂，这对后续多传感器时间对齐很关键。

Q2-3 封装好的消息，是怎么发布出去的? [必背] ★★

答： 通过一个**回调**（`sender_`）发出去。驱动本身**不持有发送器、不知道该往哪发**，它只做一件事：把**数据 + 这一路的输出通道名**一起交给回调，由框架去投递。 这个回调是**框架创建设备实例时、通过构造函数参数传进来的**，绑的是框架自己的一个成员函数——所以它一存在就必然可用，不会出现"建好了但还没设回调"的半初始化状态。 调 `sender_` 的时候传两个参数：**输出通道名**（告诉框架往哪个话题发）+ **消息对象**（要发的数据）。因为驱动不认识下游，它必须把通道名交给框架去发，好处是下游增加、删除、换一拨人，驱动一行代码都不用改。

追问：驱动自己拿着发送器直接发不行吗？为什么非要回调绕一圈?

> 不行。驱动是编译成**动态库**的，它根本不知道框架的发送器在哪、什么时候建出来的。更关键的是：一旦驱动直接依赖发送器，它就**再也离不开框架**——没法单独编译、没法单独拿假数据测试。回调的作用就是把"我产出数据"和"数据怎么发出去"两件事切开，驱动只管前者。

八股补给:

Q1：回调函数到底是个什么东西?

> 就是**把一段函数当参数传给别人，让别人在合适的时候回头调**。C 里是**函数指针**（存函数地址）；C++ 里多用 `std::function` 把函数"包"起来，能装普通函数、也能装**成员函数**。一句话：**回调 = 我定"什么时候调"，你定"调了干什么"。**

Q2：为什么成员函数不能直接用裸函数指针当回调?

> 因为成员函数调用时必须带着**对象地址**（那个 `this`），裸函数指针存不下这个。所以要用 `std::function` 或者 `std::bind` 把"对象 + 成员函数"一起打包成一个可调用对象。框架把自己的成员函数绑成回调传进来，就是这么做的。

Q3：写回调要注意什么坑?

> 三个。一是**生命周期**：回调里用到的对象可能在触发时已被释放，一调就崩。二是**别在回调里干重活**——这个回调在拉流线程里被调，做耗时操作这路帧就卡住。三是**线程安全**：多路就是多线程同时调同一个回调，碰了共享数据要加锁。

Q2-4 双层循环是怎么实现断线自动重连的？ [必背] ★★

答： **外层管连接、内层管读数据**，两个循环套在**同一个线程**里。 外层每次进来先 `ConnectRtspStream` 试着连：连不上就睡 `reconnect_interval` 秒，回到外层顶部重试；连上了就置连接标志、进入内层。 内层只要还连着就 `av_read_frame` 不停读包；一旦读失败（断流或出错），就**标记断连、`break` 跳出内层**，出来之后关掉旧连接，睡一会儿，再回到外层顶部重新连。 所以重连不是另起线程，而是**同一个线程里"内层掉出来、外层再连"**。如果一直连不上，它会**无限重试、永不退出**，直到设备整体停止——这是刻意的，因为现场网络抖动是常态，模块不能自己放弃；关键原则是**失败要可见**，每次都打错误日志。

追问：为什么要把"连接"和"读数据"拆成内外两层，而不是一个循环里混着写？

> 因为这是**两种完全不同的状态机**。连接是"有没有连上"，读数据是"这条连接上还有没有数据来"，它们的失败处理完全不同——连不上要重试建连，读到一半断了要先关旧连接再建新连接。拆成两层，各自的状态都很简单；混在一起就得到处用 if 判断"我到底是哪种失败"，代码很快乱掉。

八股补给：

Q1：外层循环判断的是"连接状态"吗？

> 不是，它判断的是**循环自己的退出条件**（停止标志）。真正标记"连没连上"的是另一个独立的状态位，由内层在断流时置掉。把这两件事分开，外层就不用关心连接怎么断的，只看"要不要继续重连"。

Q2：连不上就睡几秒再重试，这算轮询吗？会不会占 CPU？

> 不算，这是**带退避的重试**，不是轮询。轮询是转圈不停检查、CPU 一直耗着；我这个是**真的睡过去**，线程挂起不占 CPU。而且睡一会儿能避免设备刚重启时被疯狂连，把本来能恢复的设备打得更死。

Q3：睡眠那句外面还包着一个判断，你知道吗？

> 包的是**停止标志**，睡前发现要停，就跳过这次等待。但如果线程已经进入睡眠，修改原子标志不会自动唤醒它，仍可能等到睡眠结束；想更快响应停止，需要用可以被通知唤醒的等待方式。

Q4：那个停止标志为什么不用普通 bool，要用原子类型？

> 因为它是**两个线程同时读写**的：工作线程在循环里读，主线程点停止时写。普通 `bool` 会被编译器优化成"只读一次放寄存器"，永远看不到更新。原子类型（`std::atomic`）保证读写原子、且禁止这种优化。

Q2-5 帧间隔统计具体是怎么实现的？ [必背]

答： 每一路流自己维护一组统计字段：**上一帧的发送时间**、累计间隔、最小值、最大值、帧数。 每发出一帧，先算"这一次 - 上一次"的间隔，跟已有 min/max 比一比更新，累加到总和，再把"上一次时间"改成现在。平均值**不是每帧现算**，而是每 100 帧打日志时**现算**（总和除以帧数），省掉每帧做除法的开销。 有两个细节：**第一**

帧不参与统计（因为"上一次时间"初值是 0，第一帧算出的"间隔"其实是"程序启动到收到第一帧"的时间，会污染统计）；**重连后统计不重置**（字段挂在每路状态上，不随连接重建清零）。

追问：这个统计在现场有什么用？为什么要做它？

> **链路诊断。**正常三十帧的视频，相邻包间隔应该在三十几毫秒；某路最大值突然到几百毫秒甚至几秒，说明这条链路有抖动或丢帧。现场套路：画面卡顿 → 翻这路日志 → 最大值大但平均值正常是偶发抖动（网络瞬时问题），平均值也偏大是带宽或相机本身的问题。**不用抓包、不用接显示器，看日志就能定位到哪一路。**

八股补给：

Q1：求最大值/最小值的初值，为什么方向要"相反"着取？

> 这是求极值的标准起手式：求最小值，初值取**可能达到的最大值**，第一个真实数据一定能把它顶下来；求最大值，初值取**可能的最小值**（间隔不为负，取 0）。反过来写就永远得不到正确结果——最小值初值给 0 结果恒为 0。

Q2：为什么平均值不每帧都算？

> 纯省开销。每帧做一次除法，一路三十帧、一小时几十万帧，就是几十万次除法。而平均值只用在打日志的那一刻，所以改成"累计总和 + 帧数，打日志时现除一次"。这是很典型的"**把昂贵操作推迟到真正需要时**"的优化。

Q3：这个统计有什么可以改进的？

> 三点。一是**改滑动窗口**——现在全量累计，跑几天后早期异常会永久留在最大值里，反映不了"当下"。二是**加重连重置**——现在重连后第一帧会带着整段断线时间，污染最大值。三是**加阈值告警**——现在被动等人翻日志，应该超阈值就主动告警甚至触发重启。前两条本质是一件事：**数值要能反映当下、能自动触发使用，而不是等运维想起来去看。**

Q2-6 每路流为什么用独立线程承载？线程是怎么建、怎么安全退出的？ [必背] ★

答：

每路视频流用**独立线程**，因为读包可能阻塞，这样一路等待不会占住其他路的拉流线程。启动时遍历流表，给每路创建线程，分别执行连接和读包循环。退出设计是**先置停止标志，再等待线程收尾，最后清理剩余连接**，避免线程还在使用资源就提前释放。驱动析构也会调用停止逻辑，但当前版本的线程收尾和加锁还有问题，需要完善后才能保证可靠退出。

追问：你们退出时怎么释放拉流资源？

> 驱动提供了 **Stop()**：先设置原子停止标志，再逐路 **join 等待拉流线程结束**，并清理 FFmpeg 输入和上下文，避免资源提前释放。驱动析构会调用它，不过当前代码存在停止路径重复加锁、组件启动线程没有完整收尾的问题，所以**已有清理设计，可靠退出还需要完善。**

> ❗ **口径提醒**：不能说“没有调用停止逻辑，完全依赖进程退出回收”，也不能说“当前已经验证能优雅退出”。原子标志只是在循环检查时通知停止，不会自动打断正在阻塞的读包操作。

追问 2：释放顺序能不能反过来——先释放资源、再 join 线程?

> 不能，反过来就是崩溃：先把连接释放了，线程可能正阻塞在读包里、访问一个已释放的指针。所以铁律是**先停线程，后清资源**，三步缺一不可： ① 置**原子停止标志**——拉流线程两层循环的条件都检查它，最迟当前一轮循环结束就退出（注意：它只是通知，打断不了正在阻塞的读包）； ② **join 卡住等待**——线程真正退出才返回，这之前谁也别动资源； ③ 线程死透了，**才关连接、清上下文。** 标志位用原子类型，主线程写、拉流线程读，保证两边看到同一份值，不会读到撕裂的中间状态。

八股补给:

Q1：`join` 和 `detach` 有什么区别?

> `join` 是**等待线程执行完**，`detach` 是让线程脱离当前线程对象独立运行。这里需要等待拉流线程结束，才能安全销毁它还会访问的驱动和连接。销毁一个仍然 `joinable` 的 `std::thread` 会触发 `std::terminate`，即使线程函数已经返回也一样；`unique_ptr` 不会自动替线程做 `join`。

Q2：每路一个线程，线程多了怎么办?

> 路数增加后，线程栈、调度和网络处理都会有开销，适用上限要看设备性能和实际负载测试。路数很多时可以评估异步接入或其他线程组织方式，不能直接给一个通用上限，也不能把 FFmpeg 的读取接口简单换成 `epoll` 就算解决。

Q3：进程和线程有什么区别？这里为什么用线程?

> 进程通常有各自的地址空间，同一进程里的线程共享地址空间、各自有栈和执行状态。这里多路拉流需要使用同一套驱动对象和发布回调，用线程更方便；但共享数据也要做好同步，不能因为用了线程就默认安全。

Q4：多路线程同时跑，那张流表是怎么保证线程安全的?

> 用互斥锁保护流表的建表、遍历和清理等操作，但要结合完整调用关系检查，不能只看某个函数有没有加锁。流表里的共享指针只管理对象生命周期，**不保证对象成员的并发访问安全**；每路的状态如果还会被停止、查询等其他线程访问，也要分别分析。

> 📌 ***自己复习（面试别说）：`standard_ipcamera.h:48` 的驱动析构调用 `Stop()`；`standard_ipcamera.cc:131` 的停止入口置全局标志，`:169` 起逐路停止，`:180` 等待线程，`:184` 清理仍存在的输入上下文。拉流线程自身也会在 `:348` 关闭输入，不是所有关闭动作都发生在 join 之后。当前静态检查发现三点：① `Stop()` 在 `:137` 持有流表互斥锁，又调用 `CleanupFFmpeg()`，后者在 `:210` 再次申请同一把非递归锁，会自锁；应调整锁范围，避免持锁调用再次加锁的函数。② `ipcamera_component.cc:73` 创建的 `task_` 启动线程没有看到 join/detach；组件默认析构，且该线程对象先于驱动成员销毁，会有 terminate 风险，需要明确收尾顺序。③ 停止标志不能唤醒阻塞的 FFmpeg 调用；配置中的连接超时只看到了读取和日志，未见接入 FFmpeg 中断回调或超时选项，应补齐可中断等待，验证断流、读包阻塞时能否按时停止。以上是代码检查发现的改进点，不写成已经完成修复或现场验证。***

条目三：设备与平台运维（5 题）

> **简历原话** 设备与平台运维：读取 Tegra 系统接口采集 GPU 利用率、显存与温度并周期上报，支撑边缘设备健康度监控；独立开发 GUI 批量部署工具（PyQt5），支持多路口配置批量生成与远程下发：Excel 工参经映射规则生成配置，经 SSH/SFTP 并行下发并软重启生效；耗时操作置于线程池、结果经信号槽回传主线程刷新表格，避免界面卡顿；工具已迭代至 V3 并由我独立维护升级。
>
> ⚠️ **本条是两块拼起来的**：**硬件监控**（参与）+ **部署工具**（我独立做的）。部署工具要重点准备——它是唯一能体现"能独立完成一个东西"的部分。
>
> 🔴 **本条两条红线**
>
> 1. **GPU 数据没有落库**：采集完直接上报，别说是"存进数据库再做字段比对"。
> 2. **主动说用了 AI 辅助**（部署工具），别等被问出来。

技术点 1：硬件指标采集（1 题）

Q1-1 你是怎么拿到 GPU 这些硬件指标的? [必背]

答： 没有调任何厂商的私有接口，就是**读文件**。Linux 会把硬件的状态以**文件**的形式暴露在一个特定的系统目录下，所以想知道 GPU 忙不忙、显存占多少、温度多少度，直接打开对应的文件读出来就行，不需要接任何 SDK。

追问 1：为什么读文件就能拿到硬件信息？原理是什么？

> 这是 Linux 的设计哲学——**一切皆文件**。内核把硬件状态以伪文件的形式挂在一个系统目录下，读这个文件本质上是**触发一次内核查询并返回结果**，不是真的在读磁盘。所以开销极小，随便读。

追问 2：一共采哪几个指标？读法一样吗？

> 四个：**利用率、显存占用、显存占用率、温度**，读法各不相同。有的是一次能读出全部字段，有的要按行解析，有的本身就是个整型值。

追问 3：单位换算上有什么坑？

> 有两个地方必须换算，不做的话报出来的数是荒谬的：一是**字节到 MB** 要除两次 1024，少除一次就是一千倍的误差；二是**温度**有的接口给的是毫摄氏度，得再除 1000。这种错误不会崩，只会让监控图看着很离谱。

追问 4：为什么不直接调厂商的 SDK？读文件会不会不稳定？

> 调 SDK 要引入一整套依赖，还得跟着厂商版本走，边缘设备上多一个依赖就多一分风险。读文件的好处是**零依赖、跨版本稳定**——内核暴露的接口不会随便变。稳定性上做过长时间验证，没出问题。

追问 5：采集完之后数据去哪了？落库了吗？

> **没有落库**，采集完直接走上报。这里要说清：我负责的是"采得到、报得出去"，**后面平台怎么存、怎么展示不是我做的。**

追问 6：如果某个指标读不到怎么办？

> 读不到就**跳过这一个、不影响其他指标**，同时打一条日志。不能因为温度读不到就把利用率也丢了——监控最怕的是"整条数据断了"，而不是"少一个字段"。

八股补给：

Q1：读一个文件，底层到底发生了什么？

> 你写的那句"读文件"，最后会走到一次**系统调用**——就是让操作系统内核替你去干。CPU 会从**用户态切到内核态**：你的代码没有权限直接碰硬件，只有内核有。 所以一次读涉及**两次切换**（进去一次、出来一次）加**一次数据拷贝**。这也是为什么高频读小文件不划算——切换本身的开销比读那几个字节还大。 我这个场景是**几秒采一次、读几个小文件**，开销完全可以忽略。

Q2：打开文件之后拿到的那个"句柄"是什么？

> 是**文件描述符**，本质是一个**整数**。内核给每个进程维护一张表，你打开一个文件它就在这张表里占一行，返回的整数就是行号；后面读写都只报这个数字，内核自己去表里查。 关键坑：**这张表是有上限的**，打开之后不关就会一直占着，占满了之后再打开任何文件都会失败——这就是**句柄泄漏**。读系统文件这种"打开、读一行、关掉"的高频动作尤其容易漏。

Q3：读出来的是字符串，怎么变成数字？

> 文件里存的都是**文本**，读出来是字符串，要转数字。C 里用 `atoi`，C++ 里更推荐 `stoi` 这类。 区别是：`atoi` **转换失败会静默返回 0**，你分不清"文件内容是 0"和"内容根本不是数字"；而 `stoi` 会**抛异常**，你能明确知道这行读坏了。 放到我这个场景，区别就是"温度 0 度"和"温度读失败"——这两件事必须能分开，所以宁可用会报错的那个。

Q4：什么叫"一切皆文件"？

> 就是 Linux 把**硬件、进程信息、网络连接**这些东西，都包装成"文件"的样子给你看：打开、读、关掉，操作的接口完全一样。所以你不需要为每种硬件学一套专门的 API，**会读文件就会读硬件状态。** 但"长得像文件"不代表"是文件"：它们**不占磁盘**、读的时候才由内核**现算**出来，所以每次读到的都是实时值。这也是为什么读它们几乎不花时间——没有磁盘 I/O。

技术点 3：部署工具整体设计（2 题）

Q3-1 这个部署工具是干什么的？为什么要做它？ [必背] ★ 最重要的开场题

答： 现场有大量路侧设备要配参数、升版本、查状态。原来的做法是运维拿着一份 Excel **一台一台 SSH 上去敲命令**——慢，还特别容易漏，一个路口配错了可能很久才发现。 所以我做了这款 Windows 桌面工具，把「看

参数表 → 生成配置 → 远程下发 → 看结果」集中到一个界面。一句话说它的价值：**把"一台一台敲"变成"选表 + 点按钮 + 导出结果"。**

追问 1：为什么做成桌面工具，不做成网页？

> 现场同事在办公电脑上用，桌面工具能**直接读本地的 Excel**，也不用额外部署服务器；而且他们拿到的就是一个 exe，**双击就能跑**，不用装环境、不用配浏览器。

追问 2：配置是怎么从 Excel 变成设备上的配置文件的？

> 中间有一层**映射规则**：Excel 的列名 → 配置项的路径。我改的是映射层那几个文件，主流程完全不用动。这样现场说要加两个字段，我只改映射，不用碰生成逻辑。

追问 3：你怎么保证生成出来的配置是对的？

> 两层。一是**生成时校验**：必填项、取值范围、格式先卡一遍，生成不过就不出文件；二是**下发后比对**：把设备上实际在用的配置读回来跟本地这份对比，不一致会把具体差异项打出来——这才是真正能证明"生效了"的手段。

追问 4：是怎么连到那些设备上去的？

> 用 Python 的 SSH 库连上去，文件传输走另一条通道。连接本身做了超时和重试——现场网络环境差，没有超时的话一个连不上的设备能把整批卡死。

追问 5：几十台一起操作，界面会不会卡死？

> 不会。耗时操作都丢进**线程池**，结果通过信号回传主线程刷新表格。界面线程永远只做刷新，不干重活，所以几百台下去界面照样能拖能点。 👉 深挖见**条目三 Q4-1**。

追问 6：这个工具是你独立做的吗？用了 AI 辅助吗？

> 工具是我**独立开发并维护升级**的，已经迭代到 V3。开发过程中**用了 AI 辅助**——主要是搭界面和写重复的胶水代码，业务逻辑和现场适配是我自己做的。 👉 完整版见**条目三 Q3-8。主动说，别等被问出来。**

八股补给：

Q1：连接设备为什么要设超时？不设会怎样？

> 因为默认的连接会**一直卡在那里等**，直到对端响应或者操作系统自己放弃——后者可能是**几分钟**级别。 放到批量场景里，一台设备网络不通，这个任务就**永久占着线程池的一个位置**；来几台不通的设备，池子就被占满，后面所有的任务都排队等着，**整批处理直接堵死**。设了超时，它最多卡那么久就失败返回，位置立刻还回来。 一句话：**超时不只是"怕慢"，更是防止一个坏节点把整体拖死。**

Q2：SSH 和 SFTP 是什么关系？是两个服务吗？

> 不是两个服务，是**同一条连接上的两套用法**。SSH 那套协议里有个"**开子通道**"的能力，SFTP 就是在这条已经认证过的连接上再开一条通道来传文件。 好处是**只认证一次、只占一个端口**（默认 22），防火墙好开，也省去再连一次的开销。所以工具里是"连一次，命令和传文件都走它"。

Q3：为什么要做"下发后把配置读回来比对"这一步?

> 因为**发出去成功不等于生效了**。文件传上去了、命令也返回成功了，但设备那边可能没重启、可能加载的是旧文件、可能配置里有个字段它不认直接忽略了。 只有**把设备上真正在用的那份读回来跟本地对一遍**，才能证明"生效了"，而且不一致时能把**具体差在哪一项**打出来。这比"返回成功就打勾"可靠得多。

Q4：配置文件为什么一般用 JSON / YAML，不用自己定格式?

> 三个原因。一是**不用自己写解析器**，现成的库直接把文本变成程序里能用的结构，省一大堆解析 bug。二是**层级表达能力强**，嵌套的设备、通道这种结构用缩进或者括号写得很清楚，自己定的"每行一个键值对"根本表达不了嵌套。三是**通用**，别的语言、别的工具、甚至人肉看，都能直接读。 代价是**人写起来容易出错**（少个逗号、缩进错一格），所以才要在生成时先**校验**一遍、生成完再**回读比对**一遍。

Q5：这种批量操作，怎么做才不会"做一半"?

> 核心是**让它可重跑**。每一步都设计成"跑一次和跑三次结果一样"（也就是**幂等**），失败了不用回滚，直接再跑一遍就行。 我这个工具里：传文件是覆盖写、创建目录是"没有才建"、校验不过根本不出文件——都是幂等的。真出问题，现场同事重新点一次按钮就好，不需要我教他怎么清理现场。**给现场用的工具，"重跑一次"必须是最安全的解法。**

Q3-8 这个工具是你独立做的吗？用了 AI 辅助吗？ [保底] ★★ 诚实题，必答

答（照实说，这题诚实比包装值钱）： 工具是我**独立开发并持续维护**的，从第一版迭代到第三版。开发过程中我大量使用了 AI 辅助。 说清楚比较准确：**需求、架构、字段映射规则是我定的**，AI 帮我快速产出界面的骨架和一批样板代码，我再**逐行读、改、调**，最后自己打包分发给现场，后续迭代和维护也都是我在做。 第三版主要解决三个现场反馈的问题：域 ID 和订阅主题改成界面可配；证书目录没写权限导致上传失败；把配置的生成和部署合并成一个按钮，避免有人忘记点其中一步。 能佐证的是：它有完整的开发维护记录、有说明文档、有给现场用的操作手册，产物是现场正在用的东西。

追问：AI 写的代码你怎么保证它是对的?

> 三条。第一是**我逐行读过**——AI 出的东西我不直接用，界面骨架和样板代码我会一行一行看，看不懂的地方就查，看懂了才留。第二是**靠现场验收**：工具是给真人在真设备上用的，配置生成错一个字段，设备那边就起不来，所以每次迭代我都在现场环境上跑一遍全流程。第三是**我自己写的那部分才是核心**——Excel 列名到配置项路径的映射规则、连接和重试、权限兜底，这些是业务最关键也最容易错的地方，是我自己写的。 一句话：**AI 提高的是产出速度，判断对错的责任还在我身上。**

八股补给:

Q1：Python 写的程序，怎么变成现场能双击运行的 exe?

> 靠**打包工具**（比如 PyInstaller）。它做的事是：把你的代码、你依赖的第三方库、还有一个**精简版的 Python 解释器**全塞进一个目录或者一个文件里。所以那个 exe 本质是"自带运行环境的压缩包"，运行时会先把自己解开到一个临时目录，再用自带的解释器去跑你的代码。 这也解释了为什么**只拷那个 exe 跑不起来**——依赖目录没跟着走。

Q2：为什么会有"写到了当前目录"这种坑?

> 因为代码里如果写的是**相对路径**（比如直接写个文件名或者 `./logs`），它是相对**当前工作目录**的，不是相对程序所在目录。从桌面双击启动，工作目录可能是桌面；从命令行 `cd` 到别的地方启动，工作目录又是别处——于是日志和产物就散落在不同的地方。 正确做法是**用程序文件自己的位置去算路径**，然后所有读写都基于它。这个坑在 C++ 里一模一样，也属于"部署到别人机器上才会暴露"的典型问题。

Q3：一个要交给别人长期用的工具，除了"能跑"还该有什么?

> 四样。一是**日志**——现场出问题时你不在旁边，只能靠日志复现，而且日志要分级别，别把调试信息也全量写出去。二是**配置外置**：域 ID、订阅主题这些后来就是改成界面可配的，不然现场每换一个场景就得让我重新打包一次。三是**版本号**，出问题能定位"你用的是哪版"。四是**操作手册**——使用者不是你，他不看代码。

> 💡 **说"用了 AI"不减分，说不清自己贡献了什么才减分。** 主动给出"规则我定 / 代码我读我改 / 迭代我维护"这三点，反而说明你能把 AI 用在刀刃上。

技术点 4：并发与界面不卡（1 题）

Q4-1 几十台设备一起操作，界面会不会卡死? [必背] ★★ 高频

答： 会卡——所以**所有耗时的操作一律不放在界面线程**（也就是主线程）。 具体做法是：点按钮之后，主线程里只干两件轻活——读取参数表、把每台设备的连接信息和要传的文件清单组装成一个**任务对象**；然后按一个并发上限**分批**，**每一台设备一个任务**，投进**线程池**里去跑。 后台线程里连上去、传文件、执行命令，干完之后**发一个信号**把结果对象带出去；主线程那边的**槽函数**收到信号，只负责往表格里写一行、打一条日志，**不做任何耗时的事**。 一句话概括：**重活全在后台线程，主线程永远只刷界面。**

追问：那个并发上限是怎么定的？定多少算合适?

> 是**在现场试出来的**，不是算出来的。先定一个小值，拿一批真实设备跑一遍，看总耗时、看有没有大面积超时；再往上加，加到"再加也不快了"就停。 定这个数的目的不是"越快越好"，而是**给下游留余量**——几十台设备同时被连接、同时被传文件，对网络和目标设备都是冲击，冲得太狠会大面积超时，反而比慢慢来更慢。

八股补给:

Q1：Python 有多线程吗？那个 GIL 到底是什么?

> 有线程，但有个叫 **GIL（全局解释器锁）** 的东西：**同一时刻只允许一个线程执行 Python 字节码**。所以 Python 的多线程**不是真正的多核并行**，是"轮流执行"。 它是为了**简化内存管理**才加的——有了它，引用计数就不用到处加锁，单线程程序也更快。代价就是多核利用率上不去。

Q2：既然有 GIL，那我用线程池还有意义吗?

> **有，而且非常大。** 关键要分清任务是 **I/O 密集**还是 **CPU 密集**。 我这个场景是**连 SSH、传文件、等命令返回**——线程绝大部分时间**卡在等网络上**，这时候它会**把 GIL 放掉**给别人用。所以 30 个线程一起等网络，是真的省了 30 倍时间。 反过来，如果是**大量计算**，那线程全程占着 GIL，多线程就白搭，得换**多进程**才能真正用上多核。

Q3：线程池和"来一个任务就起一个线程"有什么区别?

> 两个区别。一是**开销**：起线程要分配栈、要进内核，任务一多光起线程就耗掉不少；线程池是**预先起好一批反复用**。二是**保护**：线程数不受控的话，几百个任务就是几百个线程，系统会直接被拖垮；线程池把并发数**卡在一个上限**，超出的任务排队等着。 这也是为什么我在工具里要**分批投**，而不是一次性把几百个任务全塞进去。

Q4：信号槽为什么是跨线程安全的？它背后是怎么实现的?

> 原理是**事件队列**。后台线程"发信号"这个动作，本质上是往接收方所在线程的一个**队列里塞了一个事件**；接收方线程在自己的循环里取到这个事件，再执行对应的槽函数。 所以槽函数**是在接收方线程里跑的**，从头到尾只有一个线程碰那些界面控件，自然就不用加锁。这也是它比"自己搞个队列 + 一把锁"省事的地方。

Q5：C++ 里没有信号槽，同样的需求怎么做?

> 思路一样：**后台线程算完，把结果塞进一个加锁的队列，主线程去取。**Qt 的信号槽本质就是把这个模式封装好了。自己写的话要自己管三件事：队列要加锁、要有条件变量通知"有东西来了"、还要有一个"退出"的标志让线程能收尾。

技术点 5：远程下发与现场坑（1 题）

Q5-3 你们传文件遇到过权限问题吗？怎么解决的？ [加分] ★★ 最有含金量的一个坑

答： 遇到过。现象是目标目录属主是 root、我用的普通账号没写权限，传输直接报**权限不足。** 一开始我判断的是"目录在不在，不在就创建"，很快就发现不对：目录**明明存在**，就是写不进去。所以我把判据改成了**"当前用户能不能写"**——这才是真正的成功判据。 解决套路是分级的：可写就直接过；不可写先试普通用户创建；还不行就走**提权**——用 `sudo` 把目录建出来、再把属主改成当前用户，之后同一账号走 SFTP 就能写了。密码走标准输入，**不拼进命令行**（避免转义，也避免被 `ps` 看到）。 还有个坑：部分设备的 sudo 要求必须有终端，第一次提权会失败，所以失败之后**自动带着终端重试一次。**好处是：现场不用在每台设备上手工配免密 sudo，也不用改成 root 登录。

追问：为什么一开始判断"目录在不在"是错的?

> 因为**"存在"和"能写"是两件事。**目录确实在，但它的属主是 root，我用的普通账号只有读和执行权限、没有写权限——所以判据必须换成**"当前用户到底能不能写"**。 这类 bug 的共同点是：**你检查的是成功的前提，不是成功本身。**后来我养成的习惯是，凡是"检查一遍再动手"的代码，都要问一句——我检查的这个条件，是不是就等于能成功?

八股补给:

Q1：Linux 的文件权限是怎么表示的?

> 每个文件挂着三组权限，每组三个字母：**属主、属组、其他人**，每组的三个字母是 **r（读）、w（写）、x（执行）**。用命令看到的是 `rwxr-xr-x` 这种九个字符。 也可以写成数字：r=4、w=2、x=1，加起来。**755** 就是"属主可读可写可执行，其他人可读可执行但不能写"——所以别人创建的文件，我这个"其他人"是写不进去的，这正是我遇到的那个坑。

Q2：`chmod` 和 `chown` 有什么区别?

> `chmod` 改的是**权限位**（那九个字母），`chown` 改的是**属主**（这个文件归谁）。 我那次的处理是**两个都用**：先用提权把目录**建出来**（因为父目录我没权限写），再 `chown` 把它的属主**改成我这个账号**，之后我走普通连接就能写了。只改权限不改属主也行，但把属主改成自己更彻底，后面所有操作都不用再提权。

Q3：`sudo` 凭什么能让普通用户干 root 的事?

> 因为 sudo 这个程序文件上有个特殊的权限位，叫 **setuid**。带这个位的程序，运行时**不是以启动它的用户身份跑，而是以文件属主的身份跑**——sudo 的属主恰好是 root，所以它跑起来就有 root 的权限。 也正因为这么危险，sudo 才会额外要求**输密码**、要求配置白名单（谁能 sudo、能跑哪些命令），有的环境还要求**必须有终端**——我遇到的就是最后这条。

Q4：为什么在命令行里传密码是不安全的?

> 因为**命令行是公开的**。Linux 上任何一个用户都能通过进程列表看到别人进程的完整命令行参数，密码就明晃晃地挂在上面。 正确做法是**走标准输入**（比如给 sudo 加一个"从 stdin 读密码"的参数），这样密码只存在于进程的内存和输入管道里，不出现在命令行中。顺带还解决转义问题：密码里要是有个空格或者特殊字符，拼进命令行就会被 shell 解释成别的意思。

Q5：把用户填的内容拼进 shell 命令，除了注入还有什么坑?

> 最常见的是**空格把命令截断**。比如文件名里有个空格，拼出来就变成两个参数，命令直接执行错。还有引号、通配符这些特殊字符也会被 shell 先解释一遍。 通用解法是**统一走一个转义函数**，用单引号把参数整个包起来，再处理内部的单引号，这样无论填了什么奇怪的字符串，shell 都只把它当成**一个完整的参数**。

条目四：交通参数检测服务（8 题）

启动流程：初始化是谁调用的?

FusionService 宿主调用插件的初始化接口；初始化中第一次调用 `get_instance()` 时，才创建单例工厂。

`FusionService 启动，读取插件和通信配置`
↓
`加载流量插件的 .so`
↓
`调用 TrafficFlowServiceGetApi()`
`拿到插件提供的函数表，并创建 TrafficFlowServiceImpl 对象`
↓
`通过函数表调用插件的初始化入口`
`TrafficFlowService_Init()`
↓
`入口转交给实现对象`
`impl->init(config_path)`
↓
`读取算法配置`
↓
`调用 TrafficFlowFactory::get_instance()`
`第一次调用时，才创建工厂对象`
↓
`工厂构造：`
`初始化队列 → 根据配置创建通道和检测器 → 创建工作线程跑 detect()`
↓
`init() 注册发布回调，再调用 start_running()`
↓
`初始化完成，开始正常接收和处理数据`

> **记忆点**：先加载 `.so`，再初始化，最后正常收数据。工作线程在队列为空时等待；后续每帧回调中的 `get_instance()` 只取已有工厂，不重新创建检测器。 **代码边界**：插件源码能看到初始化入口及后续调用链；宿主内部具体在哪一行加载、调用插件，不在本插件源码中。

> **简历原话** 4. **检测器框架与数据管道**：负责车头时距、流量检测器的实现，参与抽象基类 + 工厂模式的检测器框架搭建，各类复用同一套「逐帧累积、到点结算」多态流程；并以生产者-消费者模型实现数据链路——上游目标流经 DDS 订阅后写入有界阻塞队列，由独立工作线程消费，数据接收与检测计算解耦。 5. **车辆过线判定**：实现基于停止线几何相交的车辆过线判定，并叠加质量门控（位移阈值 + 横穿方向约束）过滤目标抖动与擦边误判，支撑流量与车头时距统计。
>
> 🔴 **本条三条红线**
>
> 1. **输入是结构化目标流，从没见过像素。**解码、检测跟踪、融合是算法团队的活。
> 2. **通信是 Fast-DDS，不是 CyberRT**（跟设备接入那块是两套中间件）。
> 3. **"上报"不是我做的**：口播讲到"独立工作线程消费、逐帧累积、到点结算"为止。
>
> ⚠️ **三条诚实口径**
>
> 1. 检测器实际是 **6 类**（还有排队 / 车速 / 过街 / 溢流），**没有逆行**；简历收敛写成"负责车头时距、流量检测器"。
> 2. **质量门控只对行人和非机动车严格生效，机动车不拦**（详见 Q2-5）。"支撑流量与车头时距统计"站得住，但**绝不能说成"所有过线目标都过了这道门"**。
> 3. "**负责**…检测器的实现"和"**参与**…框架搭建"是**有意分开**的，别说反、别升格。

技术点 1：整体流程（1 题）

Q1-1 这个交通参数检测服务，整体上是怎么跑起来的？ [必背] ★★ 总览题

> **复习卡点：周期与调用入口**
>
> - 周期规则提前写在配置里；初始化按周期建组，每组创建自己的检测器。同一帧更新各组，分别判断是否结算，不是把帧分给某一个周期。
> - 固定周期依据帧时间戳检查；当前配置按整秒/整五分钟对齐，启动不足一窗的部分丢弃。信号周期则看参考相位的真实边界。
> - 宿主调用插件的 Init 和 OnDataAvailable；接收回调负责解析入队。工厂构造时创建工作线程，线程执行 detect，循环 take → update → 检查周期 → calculate_once → 发布。

【文件与对象卡点】 TrafficFlowFactory 不只创建检测器，还管理队列和计算线程。初始化首次 get_instance 创建工厂；每帧回调只取已有工厂。update_new_frame 在接收回调线程执行，detect 在独立工作线程执行，虽在同一 cpp 中也不是同一个线程。详细图见文件调用流程图，也可打开逐步互动动画。

【复习卡点】

- 一次 `update_new_frame()` 处理当前消息的一帧；每来一帧再调用一次。JSON 解析成 `PerceptionResult` 对象后入队，队列里不保留原始 JSON 字符串。
- 加锁保护两个线程共同修改的 deque，不是因为“帧多”。计算在出队后进行，不一直占着队列锁。
- 旧帧指最早入队的队首：`push_back` 入队，超容量 `pop_front` 丢队首，不按时间戳排序。

- 阻塞指队列空时消费者等待；条件变量等待会释放锁，唤醒后重新加锁、检查。满时丢旧帧，不等待空位，但入队仍需获取互斥锁。
- 检测器在初始化时创建并持续复用；逐帧 update，到点 calculate_once。最后生成 JSON 对象，再 dump 成字符串，经宿主发布消息，不是写 JSON 文件。

答： 它跑在边缘设备上，形态是一个**算法插件**（一个动态库，宿主框架按 C 接口加载它）。 数据从**上游融合感知**进来：我通过 **DDS 订阅**收到一帧**结构化目标流**（JSON 报文，里面是一帧所有目标的清单，带 id、车型、经纬度、速度、车道号），在回调里把它**解析成一帧、写进一个有界阻塞队列**，立刻返回。 队列那头是一个**独立的工作线程**在消费：每取出一帧，就**广播给一组检测器各自累加**（流量、车头时距、排队……都继承同一个基类，统一走"喂一帧"的接口），到了配置的周期就**统一结算**出数，再**聚合 + 拼成 JSON 报文**，通过发布回调交给宿主框架。 一句话概括整条链路：**DDS 订阅 → 解析入队 → 工作线程消费 → 逐帧累积 → 到点结算 → 聚合发布**。

八股补给：

Q1：这个插件是怎么被宿主框架加载、调用起来的?

> 通过一套**约定好的 C 接口**：插件编译成动态库，导出几个固定的函数（初始化、接收数据、释放资源），宿主启动时加载它、拿到函数指针后按约定调用。数据到达时，宿主回调插件的"数据到达"函数，把报文递进来。 为什么用 C 接口而不是 C++ 类——因为 **C++ 的 ABI 不稳定**：不同编译器、不同版本编译出的对象布局、符号名不一样，跨模块直接传类会出问题；C 的函数调用约定是稳定的，所以跨模块边界都用 C 风格接口。

Q2：进程和线程在这里是怎么分工的?

> 简单说：**DDS 回调线程**负责"收"（把数据解析后塞进队列），**独立工作线程**负责"算"（出队、累积、结算）。两个线程靠**阻塞队列**衔接，谁也不直接调谁。 为什么拆两个线程——因为"收"和"算"的速度、节奏都不一样：收要快（慢了会堵住 DDS 回调、丢帧），算可以慢一点（有队列缓冲着）。拆开之后各自按自己的节奏跑，互不拖累，这就是简历说的"数据接收与检测计算解耦"。

追问 1：上游给的数据到底是什么？是图像吗?

> **不是图像，是结构化目标流**——一帧里所有目标的清单，每条带 id、车型、经纬度、速度、车道号。解码、目标检测跟踪、多传感器融合那一整套是算法团队做的，我拿到的是他们的结果，**从头到尾没见过像素**。

追问 2：检测器框架是怎么把这么多类检测器统一起来的?

> 一个**抽象基类**规定两个动作"喂帧""出数"，所有检测器（流量/车头时距/排队……）都继承它各自实现；一个**工厂**读配置把它们 new 出来挂到各输出通道。上层主循环只认基类指针，调哪个自动跑到哪个子类。 👉 深挖见**Q2-1**。

追问 3：数据是怎么"收"和"算"解耦的?

> DDS 回调线程只做"解析 + 入队"，独立工作线程负责"出队 + 累积 + 结算"，中间靠一个**有界阻塞队列**衔接。队列满了丢最老一帧，保证接收端永不被堵住。 👉 深挖见**Q2-3**。

追问 4：车辆过线是怎么判定的?

> 两帧连成线段，看它跟虚拟停止线相不相交；再叠两道防抖（位移要够大、方向要够横穿），并且同一辆车周期内不重复计。 👉 深挖见**Q2-4 / Q2-5**。

追问 5：那这几类检测器，你具体做了哪些?

> 我**亲手实现的是流量和车头时距这两个检测器**（分车型计数、PCU 折算、跟车时距），框架是跟 mentor 一起搭的、我出具体模块。其余排队/车速/过街/溢流是别的同事做的。 👉 深挖见**Q2-6**。

技术点 2：七个实现点（7 题）

Q2-1 检测器框架的「抽象基类 + 工厂」是怎么搭的? [必背] ★★

答： **抽象基类**定义一个 `AccumulateDetector`，只规定两个**纯虚函数**：`update(frame)`（逐帧累积）和 `calculate_once()`（到点结算），所有检测器都继承它、各自实现这两个方法。 **工厂**（`TrafficFlowFactory`）在启动时读 JSON 配置，按配置里写的检测器类型，逐个 `new` 出对应的子类（流量、排队、车速、车头时距、过街、溢流），挂到对应的**输出通道**上。所谓"输出通道"= 一种统计周期（比如 5 分钟 / 信号周期）+ 挂在这个周期下的一组检测器。 上层主循环手里只拿**基类指针**，调 `update` / `calculate_once` 时自动跑到具体子类的实现上——这就是多态。**加一类检测器，主循环一行都不用改。**

追问：为什么非要抽象基类 + 工厂？写一堆 if-else 不行吗?

> 不行，因为那会让主循环跟每一类检测器**绑死**。写 if-else 的话，加一类检测器就要打开主循环改一遍，改六类主循环就膨胀成一团，谁都不敢动。 用抽象基类之后，所有检测器在**主循环眼里长得一样**——都是"喂一帧、出一次数"这两个动作，所以主循环一个 for 循环统一调度，具体是哪一类它根本不需要知道。工厂解决的是"该造哪一个"，由配置决定，主循环连这步都不用管。

八股补给:

Q1：为什么要用纯虚函数？普通虚函数不行吗?

> 不行。普通虚函数意味着基类**给了默认实现**，子类漏实现也能编译过，运行时就变成一个"什么都没干"的静默失败。纯虚函数把这件事**逼到编译期**：漏实现就编译不过。而且基类本来也给不出有意义的默认实现——一个"流量检测器"怎么累积、怎么结算，基类怎么会知道。

Q2：通过基类指针销毁对象，要注意什么?

> **析构必须是虚函数。** 否则通过基类指针 delete 的时候，只会调到基类的析构，子类那部分资源就泄漏了。这个基类里就是 `virtual ~AccumulateDetector() = default`，保证能正确析构到派生类。

Q3：工厂里怎么从"配置字符串"找到"要 new 哪个子类"?

> 配置里写的是**类型字符串**（比如 "volume"、"headway"、"queue"），工厂里用一个 **switch/if-else 分支**：读到 "volume" 就 `new AccumulateVolumeDetector`，读到 "headway" 就 `new AccumulateHeadwayDetector`……也就是"字符串 → 具体子类"的映射，靠代码里的分支表实现。这套工厂跟设备接入那套"注册宏自注册"不一样——**这边是手写分支**，所以加一类检测器得在工厂里加一个分支，严格说算不上"零改动"（这也是后面 Q2-6 里改进点之一）。

Q2-2 「逐帧累积、到点结算」这套多态流程是怎么跑的？ [必背] ★★

答：消费一帧就是**两拍**。第一拍**逐帧累积**：工作线程每取出一帧，就用一个 for 循环把它广播给所有通道的所有检测器，每个检测器调自己的 `update(frame)`，只往自己的账本里累加（比如流量检测器数过了几辆车、车头时距检测器记下每辆车的过线时刻），**这一步不出数**。第二拍**到点结算**：用**帧的时间戳**判断这个通道有没有跨过统计周期边界（比如 5 分钟到了没、信号周期翻没翻页），跨过了才让这个通道的每个检测器调 `calculate_once()`，把账本算成结果、**并且算完就清零**；没跨过就继续等下一帧。好处是**窗口干净**：算完即清零，相邻两个周期不重叠、不重复计数。

追问：那"周期"是怎么定的？到没到点怎么判断？

> 周期写在**配置文件**里，每个输出通道各配各的（5 分钟、15 分钟、或信号周期）。判断"到没到点"有两种：**固定周期**用**时间栅格对齐**——用帧时间戳除以周期长度、看商有没有变，变了就说明跨进了新窗口（而不是"上次时间加一个周期"那种会累积误差的算法）；**信号周期**是**由信号灯报文驱动的**——信号相位管理器收到真实的灯态数据、检测到"参考相位从红翻绿"才推进一个边界，没有信号数据这个通道就不出数，**不做定时模拟**。触发逻辑是框架既有能力，我参与的是让服务支持可配置周期。

八股补给：

Q1：判断周期边界，为什么用"时间戳除以周期"而不是"上次时间加周期"？

> 因为后者会**累积误差**：如果某一帧时间戳跳变很大（比如重连后时间跳了一下），用"上次 + 周期"去比，可能**一帧跨好几个周期却只结算一次**，中间几个周期凭空消失。用"除以周期、看商有没有变"，商一变就是跨进新窗口，天然正确处理跨多窗口的情况。

Q2：算完之后为什么要手动清零，不能让它自己重建？

> 可以重建，但**重建要重新分配内存**，清零是**复用同一块**。周期结算是每隔几分钟就来一次的，跑几个月就是成千上万次，每次都重新分配内存对边缘设备是一笔不必要的开销。所以做法是**容器留着、只清空里面的元素**，容量还在，下次接着用。

Q3：为什么平均值/结果不每帧都算？

> 纯省开销。每帧做一次除法或复杂计算，一路目标流一秒几十帧、跑一天就是几百万次。而结果只用在"到点结算"那一刻，所以改成"逐帧只累积、到点才现算"。这是很典型的"**把昂贵操作推迟到真正需要时**"的优化。

Q2-2A 这个项目的多态，在代码里到底是怎么实现的？ [必背] ★★

答： 三个零件配齐：**基类指针容器 + 初始化时的绑定 + 运行时虚表分发。** **容器**：每个输出通道（OutputChannel）里有一个 `vector<unique_ptr<AccumulateDetector>>`，装的全是**基类智能指针**，各自指向不同的子类对象（流量、车头时距、排队……）。 **绑定**：发生在**初始化**。工厂 `build_detectors_from_config()` 读配置，按类型 `make_unique<AccumulateVolumeDetector>(...)`（内部就是 new 出子类对象），`push_back` 进基类容器那一刻，子类指针**隐式转换成基类指针**——"基类指针指向子类对象"就在这一步完成。 **分发**：发生在**运行时**。工作线程 `detect()` 遍历容器调 `detector->update(frame)`，编译期根本不知道调哪个；运行时 CPU 顺着指针找到对象、读**对象开头的 vptr（虚表指针）**、指向该类的 **vtable（虚表）**、按**固定下标**取出函数地址再跳转——指向哪个子类对象，就落到哪个子类的实现。 一句话：**"指针→对象"的绑定在初始化（make_unique 那一刻），"函数→实现"的绑定在运行时（查 vtable），两层配合就是完整的多态。**

追问：绑定的代码到底在哪一行？为什么我翻代码看不到 new？

> 就在工厂读配置建检测器的那几行 `push_back(make_unique<子类>(...))`。两个细节：**代码里看不到裸 new**，因为 `make_unique` 内部就是 new；**也看不到任何"类型转换"字样**，因为子类指针转基类指针是 C++ 隐式做的。另外运行期主循环对这个容器**只读**（不增删检测器），所以没有迭代器失效问题，也不用给容器加锁。

【绑定代码参考】（traffic_flow_factory.cpp，`build_detectors_from_config()` 内，约 L312）

```
// 容器声明（traffic_flow_factory.h，OutputChannel 内）：基类智能指针容器
//   std::vector<std::unique_ptr<AccumulateDetector>> detectors;

// 初始化绑定：new 子类对象 → 隐式转基类指针 → 进容器
channel.detectors.push_back(
    std::make_unique<AccumulateVolumeDetector>(detector_id, coil_info));
//                                  ↑ 绑定就发生在这一刻：子类对象装进基类指针
```

【一帧内的真实动作参考】（traffic_flow_factory.cpp，`detect()` 主循环，L369-373）

```
for (auto& channel : _output_channels) {
    for (auto& detector : channel.detectors) {   // detector 是 unique_ptr<AccumulateDetector>
        detector->update(detected_result);
        //   CPU 实际做的三步：
        //   1. 取 detector 指向的对象开头 → 读 vptr
        //   2. vptr → 指向该子类的 vtable
        //   3. vtable[update 的固定下标] 取函数地址 → 跳转执行
        //       （比如落到 AccumulateVolumeDetector::update）
    }
}
// 容器里有 1 秒通道的、5 分钟通道的、信号周期通道的，volume 的、headway 的……
// 每个指针走同样的三步，但指向的对象不同 → vptr 指向的表不同 → 落到不同实现。
// 这就是"同一段循环，跑出 N 种行为"。
```

八股补给:

Q1：vptr 和 vtable 到底是什么?

> vptr 是**对象里的一个隐藏指针**（通常在对象内存最开头），一个对象一个；vtable 是**每个含虚函数的类一张**的函数地址表，同类 N 个对象共享同一张，表里按声明顺序存该类所有虚函数的地址。`update` 在表里的下标是**编译期定死的**，所以运行时是**按固定下标直接取，O(1)，没有搜索**——虚函数调用只比普通调用多两次解引用，开销很小。

Q2：为什么通过基类指针销毁子类对象不会泄漏?

> 因为基类析构声明成了虚函数（`virtual ~AccumulateDetector() = default`），析构也走 vtable 分发。如果析构不是虚函数，通过基类指针销毁时只调基类析构，子类那部分资源就泄漏了。铁律：**只要打算用基类指针管理子类对象，析构就必须是 virtual。**

Q3：`make_unique` 和裸 new 是什么关系?

> `std::make_unique<T>(args)` 内部就是 `new T(args)`，只是把裸 new 包成了智能指针，析构由 unique_ptr 自动负责、不会泄漏。项目里检测器、工作线程都是这么建的（线程：`make_unique<std::thread>(&detect, this)`），所以翻遍代码看不到一个裸 new。

Q2-3 生产者-消费者数据管道是怎么实现的? [必背] ★★

答： 三个角色：**生产者**是 DDS 回调线程，**消费者**是独立工作线程，中间靠一个**有界阻塞队列**衔接。 生产者（`update_new_frame`）只做两件事：把收到的 JSON **解析成一帧结构化数据**，然后 `put` 进队列，立刻返回——**不做任何检测计算**，避免拖慢 DDS 回调。 消费者（`detect()`）在独立线程里跑一个 while 循环：`take()` 出一帧，广播给检测器累积、到点结算。队列空的时候，`take()` **阻塞等待**，线程睡眠不占 CPU，有数据进来被唤醒。 队列是**有界**的（上线版本容量 60），满了之后 `put` 不阻塞，而是**丢弃最老的一帧**——这叫"丢弃式背压"，保证生产者永不被堵住。

追问：为什么不直接在 DDS 回调里算完，非要加个队列隔一层?

> 因为**回调里做重计算会堵住接收**。感知数据是持续来的，回调线程一旦被计算占住，后面的帧就收不进来，直接丢帧。加队列之后"收"和"算"拆成两段，各自按自己的节奏跑，互不拖累——这就是简历上说的"数据接收与检测计算解耦"。队列还顺带带来三个好处：削峰填谷（上游瞬间密集出目标，队列先缓存着）、背压保护（有界，内存不会无限涨）、线程安全由队列内部的锁保证（生产消费不用各自加锁）。

八股补给:

Q1：阻塞队列内部是怎么实现的?

> 三个零件：**互斥锁**（保护队列本身，生产消费互斥访问）+ **条件变量**（消费方在队列空时等它、生产方 put 后唤醒它）+ 一个**双端队列**（底层存储）。生产方 `put`：加锁 → 压入队尾 → 满了丢最老 → 唤醒消费方。消费方 `take`：加锁 → 空则等在条件变量上 → 有数据被唤醒 → 取队首。

Q2：为什么消费方判断队列空要用 `while`，不能用 `if`?

> 因为存在**虚假唤醒**——线程可能在没有收到任何通知的情况下被唤醒（操作系统层面允许的行为）。用 `if` 的话，被虚假唤醒后就直接往下走去取数据了，而队列其实还是空的，一取就出错。用 `while` 的话，醒来第一件事是**再检查一遍条件**，不成立就继续睡。所以标准写法永远是 `while (queue.empty()) wait()`。

Q3：队列满了，除了丢最旧的还有别的选择吗?

> 四种：① **阻塞等着**（生产者卡住，等有空位再进，不丢但会拖慢上游）；② **丢新的**（新来的扔掉，保住老的）；③ **丢最旧的**（我们这个，适合"新数据更值钱"的实时统计）；④ **扩容**（内存会涨，边缘设备不划算）。判据就一句：**你是更在乎"不丢"，还是更在乎"拿到的是最新的"**。我们选丢最旧，因为交通统计是持续采样，丢一帧老数据比堵住上游、或者内存爆掉都划算。

Q4：为什么队列要设容量上限（有界）?

> 防**内存无限涨**。如果无界，上游出目标的速度一旦超过下游消费速度，队列会越堆越大，最后把边缘设备内存吃光。有界 + 丢最旧，把内存占用**卡死在一个固定上限**，代价只是极端情况下丢几帧老数据——对统计来说这是可接受的。

Q5："有界"具体是什么意思？为什么是 60 帧?

> "有界"就是构造队列时定死一个容量上限，超过就丢最老帧。代码里 `BUFFER_CAPACITY = 60`，构造时 `_input_buffer(BUFFER_CAPACITY)`。 为什么是 60？按 25~30fps 算，60 帧大概是 2~3 秒的数据量。这个量级**够做短时缓冲**（应对上游瞬间密集出目标），又**不会让数据过期太久**（消费者永远处理的是最近 2~3 秒内的帧）。是个工程经验值，太小缓冲不够、太大数据变旧。 选 60 而不是 1000 的根本原因：**交通数据是时效性数据，老帧算出来也没用**。无界队列会让消费者处理"几分钟前的画面"，实时性就没了。

Q6：无界队列 + 不丢帧，能不能同时保证实时?

> **不能**，这是数学约束。假设上游 30fps、消费 10fps，每秒净积压 20 帧，队列只会越来越长，消费者处理的永远是过去的数据。要"不丢帧 + 实时"同时满足，**唯一办法是让消费速度 ≥ 生产速度**：要么加算力并行（不同检测器扔线程池），要么优化算法（比如过线追踪每帧只算一次、结果共享），要么主动降采样（30fps 里每隔 2 帧丢 1 帧，均匀丢而非攒满再丢）。否则只能在"丢帧"和"延迟"里二选一，本项目选丢老帧保实时。

Q7：水位警告会阻断入队吗? `put` 是谁的函数?

> 水位警告**只打日志、不阻断**。`update_new_frame` 里判断队列超过 80% 就 `cerr` 一条 warning，下一行照常 `_input_buffer.put()`。真正的"满了怎么办"在队列内部的 `put` 里处理——直接丢最老帧，不让生产者等。 `put` 是 `BlockingQueue` 类的**成员函数**（`_input_buffer` 的类型就是 `BlockingQueue<PerceptionResult>`），加锁、压队尾、超容丢最老、notify 消费者全封装在里面，工厂不用自己加锁。

Q8：生产者写入队列的完整流程是怎样的?

分两个阶段，别搞混：

- **初始化阶段（只跑一次）**：`init()` 调 `get_instance()` → 工厂构造函数执行（读配置、创建检测器、启动工作线程）。
- **运行阶段（每帧回调都跑）**：DDS 框架自动调 `on_data_available` → 取 JSON 字符串 → `get_instance()` 拿到**已存在**的工厂（不重新构造）→ `update_new_frame(json)`：① 解析 JSON 成结构体；② 水位警告；③ `_input_buffer.put()` 入队，立即返回。 ⚠️ 常见误区：工厂构造不是在回调里调的。`get_instance()` 第一次调用才构造，之后所有调用都直接返回现成实例。如果每次回调都构造，检测器和工作线程全得重建，系统就崩了。

Q9：工厂为什么做成单例？怎么保证单例的线程安全？

为什么单例：整个服务就**一条数据管道**（一个输入队列、一组检测器、一个工作线程、一个发布回调），天然只需要一份。允许多个工厂会导致同一帧被算多遍、结果重复发布。所以 `TrafficFlowFactory` 继承 `Singleton<T>`，只能通过 `get_instance()` 拿，不能自己 new。 线程安全靠 **`std::call_once` + `std::once_flag`**（在 `Singleton::get_instance` 里）。不管多少线程同时调 `get_instance()`，`init` 函数**只执行一次**，其他线程阻塞等它完成再拿同一个实例。这比自己写双检锁（DCLP）简单，且由标准库保证正确性。

Q10：运行期多线程访问工厂成员，怎么判断安不安全？

用**表格法**三步走：① 列出所有成员变量；② 标出每个变量被哪些线程读写；③ 检查"多线程同时写"或"一写多读"有没有保护。 本项目工厂的成员安全设计分三类：

- **"启动期写一次，运行期只读"**（最聪明）：`_output_channels`、`_publish_callback` 只在构造/init 阶段写，服务跑起来后只被工作线程读。**只读不需要锁**，比 mutex 高效且不会死锁。
- **原子量**：`_status` 被 DDS 线程和工作线程同时读写，用 `std::atomic<FactoryStatus>` 保证每次读写原子。
- **靠队列自身的锁**：`_input_buffer` 的 `put`/`take` 内部有 mutex，生产消费天然安全，工厂不用加锁。 判据口诀：**一个变量被多线程访问 → 是否只有一个线程写？是且其他只读 → 不用锁（前提是写完才开始读）；多个线程都写 → 必须 mutex 或 atomic；一写多读 → 写要保护。**

Q11：阻塞队列底层为什么用 `std::deque`，不用 `vector` 或 `list`？

> 先看需求：队列要支持**队尾入队（push_back）+ 队首出队（pop_front）**，而且要快。
>
> - **不用 `vector`**：`vector` 的 `pop_front` 是 **O(n)**——删第一个元素要把后面所有元素往前挪一位。队列频繁出队，这就废了。`vector` 只有尾插尾删是 O(1)。
> - **不用 `list`**：`list`（双向链表）头尾操作都是 O(1)，但每个节点单独 new，**内存不连续、缓存命中率差**，而且 `list` 不支持随机访问。
> - **用 `deque`**：头尾插入删除都是 **O(1) 摊还**，且支持随机访问 O(1)。内部是**多段固定大小的连续数组**（分段连续），比链表缓存友好，又能在两端高效扩缩。
>
> ⚠️ 注意：代码注释里写了"底层环形缓冲"，但 `deque` **不是环形缓冲**。环形缓冲是 `vector` + 头尾指针实现的固定大小循环数组；`deque` 是动态分段数组，按需分配新段。两者都能做到头尾 O(1)，但 `deque` 不需要提前分配整块连续内存，更适合容量可变的队列。
>
> 选型口诀：**只在尾部操作 → vector；头尾都操作 → deque；需要中间插入删除 → list；需要键值查找 → map/unordered_map。**

Q12：`take()` 不做阻塞等待、改成轮询，会有什么现象？

> 三个坏处：① **CPU 忙等打满**——队列空时线程不停"循环→加锁→判断→解锁"，白烧一个核，还跟 `put` 抢同一把锁，反而拖慢生产方，**忙等是负收益**；② **返回值语义被破坏**——`T take()` 没法表达"没取到"，要么改签名，要么取出未初始化帧，timestamp=0 送进结算逻辑会误触发窗口判定；③ 折中成"轮询 + sleep(1ms)"也不行，引入固定延迟和抖动。 所以用条件变量阻塞等待：队列空时线程真正睡眠、零 CPU，`put` 里 notify 精准唤醒，事件驱动、延迟最小；`wait` 包在 `while` 里防虚假唤醒。

Q13：不设置"满了丢最老帧"，会有什么现象？

> 只有两种替代，都不可行： ① **满时阻塞生产方**——生产方是 DDS 回调线程，一阻塞整条感知链路的订阅回调全卡住，背压反向传到上游造成丢包/缓冲溢出。我们只是旁路统计模块，**不能因为自己算不过来把主链路拖垮**； ② **无界、永不丢**——生产快于消费时内存线性增长最终 OOM；队列只增不减，消费端永远在追历史数据，输出时间戳离真实时刻越来越远——**"假正常"，最危险的失效模式。** 设了丢最老：生产方恒定 O(1) 永不阻塞；队列 ≤ 60 帧，端到端延迟封顶 2~3 秒；内存有界。代价是静默丢帧：车头时距采样变稀疏，但过线判定影响小（移动线段变长、仍能与停止线相交）。

Q14：最初的笨方法是什么？怎么发现问题、怎么优化的？[加分] ★★ 叙事题

> **笨方法第一版**：`std::queue` + mutex，`put` 直接 push 不设上限，`take` 轮询——队列空就直接进下一轮循环。两个独立缺陷：take 忙等 + 队列无界。 **三个症状**：单核 CPU 打满、内存持续上涨、输出报文时间戳离真实时刻越来越远。 **排查四步**： ① **先定层**：拿帧自带 timestamp 和进模块时的本地时间对比——进模块的数据是新鲜的，发布出去的报文却落后好几分钟 → 延迟产生在模块内部，先排除上游； ② **区分"算得慢"还是"空转"**：CPU 接近 100% 但单帧耗时打点只有几 ms——几 ms 的计算不可能打满一个核 → 抓线程栈，采样点停在 take 的循环判断上 → 确认是忙等； ③ **为什么会积压**：单帧耗时随目标数增长（每个目标 × 每条停止线求交），高峰期从几 ms 涨到 30ms+，帧率恒定但消费能力波动 → 队列开始堆； ④ **确认内存**：工作集阶梯式只涨不降，和单调上涨的 queue.size() 曲线一致 → 无界队列在持有帧。 **优化**：消费端轮询 → 条件变量阻塞等待（wait 包 while 防虚假唤醒）；生产端无界 → 有界 60 帧 + 满则丢最老。**为什么丢而不是等**：生产方是 DDS 回调线程，阻塞它背压会传到感知主链路，丢几帧统计的代价远低于拖垮主链路；**为什么丢最老**：队尾永远是最新数据，延迟有界、不越积越旧。 **验证**：CPU 从 100% 降到个位数；内存稳定在固定水位；报文时间戳与真实时刻差值稳定在 2~3 秒内、不再随时间恶化。 **改完引入的新问题**：退出时进程偶尔卡住——析构只改退出标志、没有 notify，工作线程永远睡在空队列的条件变量上，join 卡死。修复是改用带超时的 take，或退出时塞哨兵 + notify_all（退出唤醒是条件变量的通用设计要求）。 **一句话方法论**：先定层（隔离上游）→ 再定性（用单帧耗时区分算得慢/空转）→ 再定量（队列水位 + 延迟 + 内存曲线三指标交叉验证）→ 最后决策（谁能被阻塞、谁不能）。**队列满经常只是受害者，真凶可能在消费端的计算里。**

Q2-4 车辆过线判定，几何相交是怎么算的？ [必背] ★★

> **数据来源卡点**：停止线来自初始化读取的 ROI/车道配置（当前实现取两条边界各自的首点）；每帧目标的位置、id、类型来自上游实时 JSON。按 id 从内存中的 `_prev_positions`（`std::map`）取上次位置，处理后更新为当前位置。`Segment2D(P, Q)` 保存两个端点并计算方向、长度；求交不需要画图或生成中间点。目标类型由上游 `inner_type` 等字段给出，本模块按枚举分类；`_crossed_ids` 只保存本窗口已认可过线的 id，不是所有见过的 id。

> **复述纠错（过线判定）**：按同一个目标 id 找到上次记录的位置，与当前位置构造 `Segment2D`，再调用 `has_intersect()`。首次出现只记位置；漏帧时“上次位置”不一定来自紧邻上一帧。`_crossed_ids` 记录本统计窗口已计数的 id，结算时清空。质量门控只严格检查行人、非机动车，机动车直接通过；位移是 **≥ 0.5 m**，方向条件是 **|cosθ| ≥ 0.5**（θ 为移动方向与停止线法向的夹角），两侧穿越都可通过，不代表识别合法行驶方向。

答： 核心是**两帧连成一条线段、看它跟停止线相不相交。** 记每辆车**上一帧的位置**，本帧到了，就把"上一帧位置 → 当前帧位置"连成一条**移动线段**，然后用线段相交算法判断这条移动线段跟**虚拟停止线**（配置里给这条检测断面画的一条线段）相不相交。相交后还要检查该目标在当前统计窗口内是否已计数；质量门控按类型分流：行人、非机动车再检查位移和横穿方向，机动车直接通过门控。 **为什么用两帧连线段、不直接看当前帧位置**——因为只看单帧会**漏掉高速车**：30 帧的视频两帧间隔几十毫秒，快车一帧能挪出去一米多，很可能上一帧还在线这边、下一帧已经跳到线那边，两帧的位置都不在线上，单帧判断就认为它压根没过线。连成线段才能覆盖这种"跨帧跳过去"的情况。

追问：相交是怎么用代码算出来的?

> 当前调用的是工程自带 `Segment2D::has_intersect()`，原理是**线段参数方程 + 二维叉积**：两条线段写成 `A + r(B-A)`、`C + s(D-C)`，解出 r、s，二者都在 [0,1] 才判相交，排除延长线上的交点。本函数用方向叉积作分母；分母为 0 时直接返回不相交（包括平行、共线情况）。另一接口 `get_intersect()` 才使用跨立等检查，不要混讲成当前调用的实现。我调用已有求交接口，没有改这个几何库。

八股补给：

Q1：叉积为什么能判断"点在线的哪一侧"？

> 二维叉积（两个向量构成的二阶行列式）的**符号**告诉你"这个点在有向线的左边还是右边"。判两条线段相交，就是看：第一条线段的两个端点，是否分别落在第二条线段的两侧；反过来再看一次。两边都"分居两侧"，才是真相交。好处是**全是乘法和减法、没有除法**，所以又快又不会除零。

Q2：浮点数为什么不能直接用 `==` 比较？

> 因为**浮点数的表示本身就不精确**（比如 0.1 存不准），两次不同计算路径算出来的"同一个值"，末位可能差一点点，`==` 就判不相等。标准做法是**比差值**：看两个数差的绝对值是否小于一个很小的容差（代码里叫 `EPSILON`，1e-9）。几何计算里到处是浮点，判断"是不是零""是不是相等"都必须用这个方式。

Q3："每辆车上一帧的位置"在代码里用什么存？

> 当前实现用 `std::map<std::string, PrevState>`：key 是目标 id，value 是上次位置和最后出现时间。每帧按 id 查历史位置，判定后再更新为当前位置。位置记录超过 10 秒未更新会被清理；同一目标首次出现时只记位置、不判过线。

Q2-5 质量门控（防抖两道闸）是怎么实现的？ [必背] ★★

答： 两道闸，滤掉"停在停止线附近抖一下就算过线"的误判。 **第一道闸是位移阈值**：上次记录位置到当前位置的距离要 ≥ 0.5 米，小于半米直接不算。**第二道闸是方向**：移动方向与停止线法向的夹角满足 |cosθ| ≥ 0.5，过滤贴着停止线平移的情况；取绝对值，所以两侧穿越都可能通过，不区分合法行驶方向。这两道检查只对行人、非机动车严格执行。 还有一个**去重**：每辆车判过一次就记进一个集合（`_crossed_ids`），**同一个统计周期内不再重复计**，防止同一辆车连续几帧跨着线被计好几次。

追问：这两道闸对**所有**目标都生效吗？

> **不，只对行人和非机动车严格生效，机动车不拦。**代码里是按目标类型决定要不要走这道门控的：行人和非机动车 `strict=true` 走严格门控，机动车 `strict=false` 直接放行。 **按当前代码说明范围即可**：流量、车头时距检测器都复用这套过线追踪器，但机动车不会被上述位移、方向门限过滤。代码没有交代按类型分流的业务依据，不把推测说成已验证结论。

八股补给：

Q1：判断方向为什么用余弦，不直接把角度算出来？

> 点积归一化后就能直接比较余弦阈值，不必再调用 `acos` 求角度。本代码取了绝对值，检查的是 **|cosθ| ≥ 0.5**：θ ≤ 60° 或 θ ≥ 120° 都可通过，因此过滤的是沿线擦边，并不区分两个穿越方向。

Q2：余弦怎么算出来的？点积是什么？

> 两个向量的**点积**（对应分量相乘再相加）等于"两个向量的长度相乘、再乘夹角的余弦"。所以点积除以两个长度，就得到余弦。几何意义很直白：**方向越一致点积越大**，垂直是 0，相反是负的。代码里就是算"移动向量在停止线法向上的投影占比"，越接近垂直穿越占比越大。

Q3：这种"写死的阈值常量"（0.5 米、60°）该怎么定义？为什么不该用 `#define`？

> 用 `constexpr`。三个写法优先级：`constexpr`（编译期确定、有类型）> `const`（运行期只读）> `#define`（宏，**不推荐**——没有类型、不参与作用域、预处理阶段被简单替换，容易出各种怪问题）。这些阈值提成常量主要是为了**好找、好改**，它本质上是个经验值，你不可能推导出"为什么正好半米"。

Q2-6 流量、车头时距这两个检测器，具体是怎么算的？ [必背] ★★

答：**流量检测器**（`AccumulateVolumeDetector`）：每帧调用过线追踪器拿到"本帧刚过线"的目标，按车型分类（行人 / 非机动车 / 小车 / 面包车 / 大车）分别计数，同时累加 **PCU**（标准车当量——小车系数 1.0、面包车 1.25、大车 1.8，把不同车折算成统一的"标准车数"）。到点结算时，把机动车三类（小车+面包车+大车）加起来就是协议里的"机动车总流量"（Volume），再输出 Volume1~Volume5 的分车型明细。 **车头时距检测器**（`AccumulateHeadwayDetector`）：记录每辆车**过线的时刻**，同一车道前后两辆车过线时刻的差值就是"车头时距"（跟车时间间隔）。到点结算时算出这一周期的平均车头时距。 两者都复用过线追踪器，只是"累积"和"结算"的逻辑不同——这正是"统一接口 + 各自实现"多态的价值：**上层主循环不用知道下面是流量还是车头时距，喂帧就行。**

追问：那 PCU 折算系数是怎么定的？为什么大车要乘 1.8？

> 这是交通工程里的**标准折算**，不是代码随便写的：一辆大车占用的道路资源、对通行能力的影响，比一辆小车大，所以要折算成"等效小车数"才能公平比较。系数是行业约定值（小车 1.0、面包车 1.25、大车 1.8），代码里写死成一张表，按车型查。这属于"**业务常量**"，跟"质量门控的 0.5 米"一样，是领域知识，不是算法推导出来的。

八股补给：

Q1：为什么分类计数要用 `switch` 而不是一串 `if-else`？

> 如果判断的是**枚举或整数**，用 `switch` 更合适——一眼看出"一共就这几种情况"，可读性好。而且编译器可能把它编译成**跳转表**，一次比较都不用。如果判断的是字符串、浮点数或范围，那只能用 `if-else`。判据：**分支依据是离散整数值 → switch，否则 if-else**。代码里按 `inner_type` 分类就是 switch。

Q2：为什么"算机动车总流量"是"三个分类相加"，而不是单独再记一个计数器？

> 因为**别重复计数**。分车型明细（Volume1~Volume5）是必须出的，机动车总流量其实就是"小车 + 面包车 + 大车"这三个分类之和——如果单独再维护一个计数器，两套数字就得保证同步，任何一处漏加就对不上。**能由已有数据推导出来的，就不要单独存**，避免数据不一致。

Q3：`unordered_map` 和 `map` 怎么选？

> 就看**要不要有序**。只按 key 查（比如"按 id 查上一帧位置""按车道 tag 查累加器"），用 `unordered_map`（哈希表，查找 O(1) 平均）；需要按 key 顺序遍历（比如输出时要按车道号排序），用 `map`（红黑树，有序，但查找 O(log n)）。代码里过线追踪的位置表是纯按 id 查，用哈希表；车道累加器也是按 tag 查，同样哈希表。

深入：流量 & 车头时距 代码细节问答（Q2-6 展开）

> 下面这组问答是把 Q2-6 里"怎么算的"落到**代码行**上，面试官如果追问"那 update 里具体写了啥""counts 是哪来的"，就用这里的答案。

Q1：流量检测器的 `update()` 里到底干了哪几件事?

> 三步：① 调过线追踪器 `_tracker.process_frame(frame)` 拿到"本帧刚过线"的目标列表；② 对每个目标 `classify_volume_category()` 分类（行人/非机动车/小车/面包车/大车，对应枚举 1~5），然后 `acc.counts[分类-1]++`——五个桶，下标 0~4 对应 Volume1~Volume5；③ 按车型查 PCU 系数表（小车 1.0、面包车 1.25、大车 1.8，行人和非机动车 0）累加到 `acc.pcu`。**update 只记账，不出数、不清零。**

Q2：为什么五个分类用一个数组 `counts[5]`，而不是五个独立变量?

> 因为车型是**离散枚举**（1~5），天然可以当数组下标用——`counts[(int)category - 1]` 一行搞定，不用写五行 `if`。而且结算清零时一句 `counts.fill(0)` 就全清了，五个变量要写五行。**能用下标索引的连续枚举，就用数组，别用散落的变量。**

Q3：`counts` 里存的是"上一帧的数量"还是"整个周期的累计"?

> 是**整个统计周期的累计**，不是上一帧。检测器对象是长生命周期的（启动时创建，一直活到服务退出），`counts` 是它的成员变量，每帧 `update` 往里加，一直加到 `calculate_once` 被调用才读出来、清零。所以它是一个"周期内累加器"。

Q4：`calculate_once()` 里做了什么？会再次累加吗?

> **不会累加**，只做三件事：① 读——把 `counts` 里的数取出来组装成输出字段，`Volume1~Volume5` 直接对应五个桶，`Volume`（机动车总流量）= 后三个桶之和（小车+面包车+大车），`PCU` 取累计值；② 清——`counts.fill(0)`、`pcu = 0`、过线追踪器状态清空；③ 返回结果。**"结算" = 读 + 组装 + 清零，不做任何加法**，加法全在 `update` 里。这样"逐帧累积、到点结算"在代码上就是严格分开的两个函数。

Q5：机动车总流量为什么是"后三个桶相加"，不单独维护一个计数器?

> 避免**双源计数**。分车型明细 Volume1~Volume5 是协议要求必须出的，机动车总量本来就是其中三类之和——如果再单独维护一个 `motor_count`，就得保证每次加车型桶时也同步加 `motor_count`，漏一次两边就对不上。**能由已有数据推导出来的量，就不要单独存**，这是数据一致性的基本做法。

Q6：车头时距 `update()` 里，一辆车过线时记了哪些东西?

> 记两个时刻：`last_arrival`（这辆车**车头**过线时刻 = `frame.timestamp`）和 `last_tail`（这辆车**车尾**离开停止线时刻 = 车头时刻 + 车长/(速度/3.6)×1000，单位毫秒）。第一辆过线的车只存这两个值、不参与统计；从第二辆开始，才算跟前车的间隔。

Q7：车头时距的"平均"是怎么除的？为什么除以 `count - 1`？

> N 辆车过线，相邻两车之间有 **N-1 个间隔**。所以平均车头时距 = 所有相邻间隔之和 ÷ (N-1)。如果只有 0 或 1 辆车，没有间隔，输出 0。这跟"算平均速度要除以段数"是一个道理——**N 个点之间只有 N-1 段距离**。

Q8：跨帧的"上一辆车"信息是怎么记住的？

> 靠**成员变量**：`last_arrival`、`last_tail`、`count`、`headway_sum`、`gap_sum` 都是检测器对象的成员。`update` 每帧被调用时，先读这些成员（拿到上一辆车的信息），算完再写回去。因为对象一直活着，所以这些值能**跨帧链式传递**——这也是为什么检测器必须是"长生命周期对象"而不能是每次 new 一个临时变量。

Q9：如果同一帧里有多辆车过线，它们之间的车头时距是多少？

> 是 **0**。因为同一帧里的所有目标共用同一个 `frame.timestamp`，所以相邻两辆车的 `current_time - last_arrival` = 0。这意味着车头时距的**时间分辨率等于帧间隔**（比如 25fps 就是 40ms），同一帧内无法区分先后。这是感知结果的固有局限，不是 bug。

Q10：`Gap` 和 `Headway` 有什么区别？

> - **Headway（车头时距）**：前车**车头**过线 → 后车**车头**过线 的时间差，包含了前车自身通过停止线的时间。
> - **Gap（净间距）**：前车**车尾**离开停止线 → 后车**车头**过线 的时间差，是两车之间真正的"空隙"。
>
> 代码里 `gap_sum` 只在 `current_time > last_tail`（后车到的时候前车已经走了）才累加，否则 gap 记 0。**Headway = 前车通过停止线的时间 + Gap**。

Q11：这两个检测器都要判断"车过线了没"，它们各自实现了一套吗？

> 没有，**都复用同一个 `StopLineCrossingTracker`**。流量和车头时距的 `update` 第一行都是 `_tracker.process_frame(frame)`，拿到"本帧刚过线"的目标列表后，再各自做分类计数或时间差计算。过线判定（几何相交 + 位移阈值 + 方向约束 + 去重）只在追踪器里写了一份，检测器不重复实现——**这就是"统一过线逻辑 + 各自统计逻辑"的分层**，改过线规则只动一个文件。

附录：通用追问（4 题）

Q1 你在这个项目里具体是什么角色？哪些是别人做的？ [必背] ★★ 最重要的一题

答： 我是实习生，主要做了两块：**设备接入与数据链路**，以及**交通参数检测服务**。 具体的：**接口怎么抽象、工厂和自注册怎么设计、多路拉流这一块的实现和调试、GPU 指标的采集，还有交通参数这边的检测器框架、数据管道和车辆过线判定**，这几块是我在 mentor 指导下负责开发和调试的。整体架构是 mentor 定的，我负责把具体模块落地。 部署工具是我独立完成的。中间的解码、目标检测跟踪、多传感器融合那部分是算法团队做的，我不参与——我拿到的是他们的结构化结果。

> 💡 这题的打分点在于**边界清晰**。说"我参与了 X、Y，Z 不是我做的"，比含糊地说"这个项目我都接触过"靠谱得多。

八股补给：

Q1：多人协作写代码，怎么避免互相改坏？

> 靠两样：**分支**和**评审**。 **分支**：每个人在自己的分支上改，改完再合并回主线。这样你写一半的东西不会影响别人，也不会出现"我这边正在改、他那边也在改同一个文件然后互相覆盖"。 **评审**（Code Review）：合并之前必须有人看过你这版改动。它抓的不只是 bug，还有"这段以后别人能不能看懂""这个做法跟现有的一致不一致"。

Q2：评审的时候到底看什么？

> 按优先级：第一**对不对**（边界条件、空指针、资源有没有释放、多线程有没有加锁）；第二**好不好改**（命名、重复代码、有没有把常量写死在逻辑中间）；第三**跟现有的一致不一致**（同一个项目里两套风格，后面的人最痛苦）。 一个很实用的经验：**一次评审的改动别太大**。几百行的改动没人能认真看完，评审就变成走形式了。

Q3：怎么知道某一行代码是谁、什么时候、为什么改的？

> 用版本管理工具的追溯功能（Git 里的 `blame`），它能显示**每一行的最后一次修改是哪个提交、谁提的**。配合提交信息，就能知道当时为什么这么改。 这在实际排查里特别有用：看到一行奇怪的代码，先查它被改的历史，往往注释里没有的原因就出来了。**所以提交信息要写"为什么改"，而不是只写"改了什么"**——后者看代码就知道，前者看不出来。

Q2 你遇到过最难的调试是什么？怎么定位的？ [加分] ★

答（重点是"怎么查"而不是"多难"）： 印象最深的是加载失败那个。现象是某个设备类型在目标机器上就是注册不上，但**在我自己的开发机上是好的**——这种"环境相关的失败"最难查。 我的排查三步：先确认不是代码问题（同一份代码在开发机跑得通）；再缩小到环境差异（是不是依赖库缺失）；最后用查依赖的工具对比两台机器，发现目标机上少了一个间接依赖。解决办法是**把依赖检查前置**——加载之前先确认依赖都在。 教训是：**凡是"在某台机器上不行"的问题，八成是环境的锅，先把环境差异列出来比对，比闷头看代码有效得多。**

八股补给：

Q1：程序崩溃之后怎么查？那个 dump 文件是什么？

> 程序崩的时候，操作系统可以把**崩那一刻整个进程的内存和寄存器状态**存成一个文件，这就是 **core dump**（核心转储）。 有了它，就能用调试器**把它打开、还原出崩的时候的调用栈**——看到是哪一个函数、哪一行、当时的变量是什么值。没有它的话，就只能靠猜。 前提是**要打开这个开关**（系统默认往往是关的，还要放开文件大小限制），这属于"出事前就该准备好"的东西。

Q2：调试器看调用栈，到底在看什么？

> 调用栈就是"**我是怎么一路走到这一行的**"——谁调了谁、谁又调了谁，从最外层一直排到崩的那一句。 有了它，即使是别人的代码崩了，你也能看出**是不是你的参数传错了**。这会比"看着代码想"高效得多。 有个前提：**编译的时候要带上调试符号**（`-g` 那个选项）。发布版本一般会去掉符号，文件小很多，但崩了之后栈里只剩地址、看不出函数名——所以**关键环境要留一份带符号的版本**。

Q3：内存一直涨，怎么确认是不是泄漏？

> 三步。第一步先**确认它真的在涨**：隔一段时间记一次内存占用，画个曲线，看是不是单调上升（程序正常时占用会上下波动，不会一直涨）。 第二步**缩小范围**：把功能一个个关掉，看关到哪个之后不涨了，就定位到那个模块了。 第三步**定位到代码**：用专门的内存检测工具跑一遍，它会告诉你是哪一行分配的内存最后没释放。 我那次是靠前两步定位到"读包那个分支没归还"的——**大多数时候不需要一上来就上工具，先能稳定复现、再二分缩小范围就够了。**

Q3 你的视频流用的是什么协议？为什么选它？ [必背]

答： 用的是 RTSP，实时流传输协议。选它主要有三个原因：第一它**支持双向控制**，除了拉流还可以发暂停、跳转这类控制指令；第二它**延迟比较低**，适合实时监控这种对时效敏感的场景；第三它是**安防监控领域的行业标准**，市面上绝大部分网络相机都原生支持，我们不用为了接某个厂商去做私有适配。 它的工作方式是：先通过协议完成握手和参数协商，然后视频数据才通过另一个通道持续过来，控制和数据是分开的。

追问：那跟 HTTP-FLV 有什么区别？

> HTTP-FLV 基于 HTTP，穿透性更好（走 80 端口基本不会被防火墙拦），但它只能单向拉流、没有控制能力，而且延迟相对高一些。路侧监控看重低延迟和控制能力，而且设备本身原生支持 RTSP，所以选它比较自然。

八股补给：

Q1：RTSP、RTP、RTCP 这三个到底是什么关系？

> 分工不同：**RTSP 管控制**——握手、协商参数、发播放暂停这些指令，它本身**不传视频数据**。真正传数据的是 **RTP**，它是另一条通道。 **RTCP** 是给 RTP 打辅助的，负责汇报"发出去多少包、丢了几个、网络抖不抖"，让你能知道链路质量。 一句话记：**RTSP 是遥控器，RTP 是水管，RTCP 是水表。**

Q2：控制走 TCP，数据走 UDP，为什么分开？

> 因为**要求不一样**。控制指令**一个都不能错、一个都不能丢**，所以用 **TCP**——它保证可靠、保证顺序，代价是有确认和重传，会慢一点。 视频数据**量大、实时性要求高，偶尔丢一包只是花一下**，如果用 TCP，一个包丢了后面的全得等着重传，画面就卡住了。所以用 **UDP**——不管到不到，到了就播。 这就是典型的**按需求选协议**：要可靠就 TCP，要实时就 UDP。

Q3：TCP 和 UDP 最本质的区别是什么？

> 四个。① **要不要先连**：TCP 要三次握手建立连接，UDP 直接发。② **保不保证到**：TCP 丢了会重传，UDP 丢了就丢了。③ **保不保证顺序**：TCP 保证按发送顺序收到，UDP 不保证（后发的可能先到）。④ **有没有流控**：TCP 会根据网络情况自动降速，UDP 不管，有多少发多少。 所以 TCP 像**打电话**（先接通、说清楚、没听清让你重复），UDP 像**喊话**（喊完就完，听见多少算多少）。

Q4：那视频用 UDP 丢了包，画面不会花吗？

> 会，但**可控**。视频是压缩过的，一个包往往不是一个完整画面（可能只是画面的一部分，也就是一个分片），丢一个只是**那一块花一下**，下一帧就补回来了。 而且关键帧（完整的那一帧）会周期性重发，最多卡一下就恢复。真正不能丢的是**关键帧和参数信息**（解码必需的那几个），这些一般会走可靠的通道单独保证。

Q5 你觉得自己这个项目里有哪些做得不够好的地方？ [加分] ★ 必问收尾

答： 坦诚说有几点。第一**可观测性不够**：加载成功与否、注册了哪些设备、每路流当前健康状态，这些都应该在启动时一目了然地打出来，现在是出事了才去翻日志。 第二**统计是全量累计而不是滑动窗口**：像帧间隔这种，跑久了早期数据会一直留着，反映不了"当下"的状态。 第三**缺少主动告警**：现在基本都是"采集了、记录了、等人来看"，理想情况应该是超阈值就主动推送。 第四**测试不足**：当时的验证主要靠实际部署环境和手工构造场景，**没有系统性的单元测试**——有些边界条件如果当时有单测，根本不会拖到现场才发现。

八股补给：

Q1：单元测试到底测什么？为什么大家都说要有？

> 测的是**最小的那一个单元**（一般是一个函数或者一个类），给它一个输入，断言输出是不是你预期的。 最大的价值不是"现在能不能跑通"，而是**以后改代码时有人替你把关**——你改了一处，跑一遍测试就知道有没有把别的弄坏。这在边界条件多的代码上尤其明显：周期正好卡在某一帧、目标突然消失，这些**靠手跑很难稳定复现，但写成测试就是几行的事。**

Q2：C++ 里写单元测试一般用什么？

> 常用的是 **GoogleTest**（gtest）配 **GoogleMock**（gmock）。前者管断言和用例组织，后者用来造"假的对象"——比如你想测一个依赖网络的函数，就用假对象把网络那层替掉，测试就不用真的连网。 配合 CMake 的话，用它的测试模块注册一下，就能一条命令跑完全部用例并出报告。

Q3：什么叫 CI？为什么要有它？

> CI 就是**每次提交代码，自动帮你编译 + 跑测试**的一套流水线。 没有它，"能不能编过、测试过不过"全靠每个人自觉跑一遍——忙起来就有人忘，坏了的代码就悄悄进去了。有了它，**提交之后几分钟就知道结果**，坏了立刻发现，改的成本最低。 对长期项目来说，它的价值不在"高级"，而在**把"应该做但会忘的事"变成自动的**。

Q4：日志为什么要分级别？一般有哪几级？

> 因为**全打太多、不打又查不了**。分了级别就能**按场景开关**：现场正常跑只看警告和错误，出问题了再把调试级打开。 常见的几级，从粗到细：**错误**（出问题了，必须看）、**警告**（不正常但还能跑）、**信息**（关键节点，比如"启动完成""连上了哪台设备"）、**调试**（排查用的细节，正常时不打）。 还有一条容易忽略的：**日志要带上下文**——哪一路流、哪个设备 id、第几帧。光打一句"出错了"，几十路流同时跑的时候根本没法查。

> 💡 主动承认问题 + 给出改进方案 = 有反思能力。比"我觉得挺好的"强一百倍。

补充：

1.项目中的相机怎么连接的？有线+TCP建立相机和mec

2.数据咋来的？读配置文件中的RTSP地址

2.TCP的分层？

```
应用层 传输层 网络层 链路层 物理层
```

3.RTSP协议是啥？还知道什么协议？

【复习易错】 名称是“实时流传输协议”；DESCRIBE 获取媒体描述；SETUP 协商传输参数，UDP 和 TCP 都需要协商；PLAY 请求开始发送。

是什么：

RTSP 叫**实时流传输协议**，主要负责音视频会话的建立和控制。可以把它理解成摄像头的“遥控器”：我们的程序是客户端，摄像头是服务端，我们通过 RTSP 告诉它要哪路视频、怎么传、什么时候开始和停止。

为什么需要它：

光知道摄像头的 IP 还不够，还要和它约定视频信息、传输方式和接收端口。典型交互是：`DESCRIBE` 询问音视频信息，`SETUP` 协商传输方式和端口，`PLAY` 请求开始发送，`TEARDOWN` 结束会话。**RTSP 管控制，实际视频数据通常交给 RTP 承载。**

项目里怎么体现：

我们从配置文件读取摄像头的 `rtsp://...` 地址，调用 FFmpeg 的 `avformat_open_input()` 打开输入，由 FFmpeg 处理这些协议交互和媒体接收准备；随后用 `avformat_find_stream_info()` 获取流信息，再循环调用 `av_read_frame()` 读取压缩视频数据包。**这部分做的是拉流和拆包，没有解码成图像。**

如果追问“还知道什么协议”：

我还了解 RTP、TCP、UDP，以及 HTTP、SSH、SFTP。结合项目来说，**RTSP 控制会话，RTP 承载媒体，TCP/UDP 提供底层传输**；部署工具里通过 SSH 执行远程命令、通过 SFTP 传文件。H.264 是视频编码格式，Protobuf 是消息定义和序列化工具，要和这些网络协议分开理解。

4.**RTP是啥?**

【复习卡点】 没写 RTP 连接代码，核心原因是 FFmpeg 已封装会话协商和底层接收。UDP 不握手，但仍要准备 socket、端口；TCP 可与 RTSP 共用连接。协商发生在建立会话的过程中，当前源码不能断言一定走 UDP。

是什么：

RTP 叫**实时传输协议**，通常用来承载音视频数据。比如摄像头把画面编码成 H.264，再按 RTP 规则打包发送。RTP 包里除了媒体数据，还有**序号、时间戳**等信息，接收端可以据此处理顺序和播放时序，但它本身不保证可靠送达。

为什么需要它：

RTSP 只约定“传什么、怎么传、何时开始”，实际发送的视频还需要打包规则。**H.264 决定画面怎么压缩，RTP 决定压缩数据怎么装进媒体传输包，TCP/UDP 决定这些包怎么在网络上传。** RTP 可以走 UDP，也可以通过 TCP 传输，不能把 RTP 和 UDP 当成同一个东西。

项目里怎么体现：

我们没有自己写 RTP 收包逻辑，而是交给 FFmpeg。调用 `av_read_frame()` 时，FFmpeg 在内部完成底层接收、RTP 解析和必要的重组，交给我们的是 `AVPacket` **压缩媒体数据包，不是原始网络 RTP 包**。我们取其中的视频数据，再封装成 Protobuf 消息发布到 CyberRT。

如果追问“为什么代码里没单独连接 RTP”：

因为这部分由 FFmpeg 处理。**走 UDP 时不需要 TCP 那样的三次握手，但仍要创建 socket、准备接收端口，并通过 RTSP 的 SETUP 协商；走 TCP 时，常见方式是让 RTP 数据和 RTSP 控制消息共用同一条 TCP 连接。** 当前这段代码没有显式设置 `rtsp_transport`，所以不能只凭源码就说现场一定走 UDP。

参考：FFmpeg RTSP 传输选项、FFmpeg 读取媒体数据包接口。

5.H.264 是啥？为什么要编码？H.265 和它有什么区别？还有哪些编码？

【复习卡点】 ①原始视频是像素多、帧多导致总数据量大，不是“单个像素太大”。②帧内压缩利用一张图里的相似信息；帧间压缩参考其他帧，记录运动和预测差异，比如路口背景基本不变、汽车在移动。③“压缩效率高”是同等画质下通常用更少数据，不是编码更快。记：H.265 通常更省带宽和存储，但编解码更复杂；H.264 兼容性成熟。

是什么：

H.264 是一种**视频压缩编码标准**，也叫 AVC，规定怎样把视频画面压缩成更少的数据。**编码**是把图像压缩成码流，**解码**是根据码流重建图像。它和前面的协议分工不同：H.264 管画面怎么压缩，RTP 管媒体数据怎么打包，RTSP 管会话怎么建立和控制。

为什么需要它：

视频是一张张连续图像，如果把每张图的像素原样传出去，带宽和存储开销会很大。比如摄像头拍路口，前后两张画面里道路和楼房基本没变，只有汽车移动了一点，没有必要每次把重复信息都重新传一遍。

H.264 主要利用两类重复信息：**帧内压缩**利用同一张图里相邻区域的相似性；**帧间压缩**参考其他帧，记录运动和预测后的差异。可以先理解成“相似的部分少传、变化的部分重点传”，实际还包含预测、残差等处理，不是简

单比较像素有没有变化。监控场景通常采用有损压缩，所以需要平衡画质、带宽和存储。

项目里怎么用：

在我们这种网络摄像头场景里，**画面的编码通常在相机内部完成。**相机先采集画面，再编码成 H.264 码流，通过网络发送；我们的程序用 FFmpeg 拉流、拆包，通过 `av_read_frame()` 获取压缩视频数据包，再封装成 Protobuf 消息发布到 CyberRT。下游模块收到后，再解码成图像并进行感知处理。**我这部分没有给画面做编码，也没有解码成图像。**

```
相机采集画面 → 相机内部编码成 H.264 → 网络传输
    → 我们用 FFmpeg 拉流、拆包 → 封装 Protobuf、发布 CyberRT
    → 下游解码成图像 → 感知处理
```

如果追问“用什么编码是不是相机内部决定的”：

要看**设备能力 + 视频流配置。**相机的芯片和固件决定它支持哪些编码；如果同时支持 H.264 和 H.265，通常可以通过相机管理页面或设备接口选择具体格式，并设置分辨率、帧率、码率。不同视频流也可能使用不同配置。我们的这段拉流代码没有下发切换编码的命令，而是接收当前 RTSP 地址对应的码流。

代码里的 `compressed_image->set_format("h264")` **只是写格式标签，不会把数据编码或转换成 H.264。**当前驱动按 H.264 处理；如果相机改发 H.265，还要适配格式标记、关键帧判断、SEI 解析，并确认下游解码器支持，不能只改一个字符串。

H.264 和 H.265 怎么对比：

H.265 也是视频压缩编码标准，也叫 HEVC。**它通常能用更少的数据保留相近画质，但编解码更复杂，也要考虑硬件支持和兼容性。**

比较点	H.264 / AVC	H.265 / HEVC
压缩效率	相对较低	通常更高
相同画质下的码率	通常较高	通常较低，更省带宽和存储
编解码计算	相对简单	通常更复杂，需确认算力和硬件加速支持
兼容性	支持范围广	要确认相机、接收程序和下游解码器支持
常见应用场景	通用视频监控、视频会议、直播，以及需要兼容旧设备或多种客户端的系统	高分辨率监控、多路视频集中存储，以及带宽或存储成本敏感、接收端支持已确认的系统
选择时优先考虑	希望接入方便、兼容成熟设备与现有软件链路	希望同等画质下减少数据量，并且整条链路有相应支持

应用场景怎么理解：

- **H.264：优先考虑兼容性。** 比如相机、录像设备和客户端来自不同厂商，还要兼容已有软件，用支持范围广的编码通常更容易接通。视频会议、直播等实时场景也会使用 H.264，但实时性还取决于编码参数、缓存和网络，不能说用了 H.264 就一定低延迟。
- **H.265：优先考虑带宽和存储效率。** 比如大量高分辨率相机需要持续传输、录像保留时间长，每路节省的码率叠加起来能减少网络和磁盘开销；前提是相机编码器、接入处理和下游解码器支持，并满足处理性能要

求。

两者的用途并不按分辨率硬划分：**H.264 也能做 4K，H.265 也能做 1080p。**选型要结合画质、码率、设备支持、计算开销和时延要求。

码率就是每秒传输多少比特，常用 Mbps 表示。比较时不能只说“H.265 更清晰”，更准确的是：**同等画质下通常更省码率，同等码率下通常能保留更多细节。** 实际效果还受场景、编码器和参数影响，不能保证固定节省多少，也不能只凭编码名称判断延迟大小。

如果追问“项目里为什么用 H.264，不用 H.265”：

先把事实和选型分析分开：**当前源码能确认的是，这版驱动按 H.264 适配；不能据此判断相机只支持 H.264，也不能证明团队做过 H.264/H.265 对比测试。**

面试可以这样说：

> 我们这版接入链路是按 H.264 适配的，我负责的是通过 FFmpeg 接收、拆包和转发。编码选型要看相机能力和整条链路的支持情况。从工程角度讲，H.264 的优势是兼容性比较成熟；H.265 虽然更省码率，但也要确认接入逻辑、下游解码器和硬件加速都支持。我们代码里有按 H.264 写的格式标记、关键帧判断和 SEI 解析，所以切换到 H.265 还需要适配和验证，不能只改相机配置。

如果继续追问“当初是不是因为带宽足够，才选 H.264”，而自己没有参与或没有记录，就回答：

> 具体的初始选型依据我没有记录，不能说是我对比测试后决定的。我能确认的是当前驱动按 H.264 处理；如果重新评估，我会结合实际场景比较画质、码率、处理负载和端到端延迟，再决定是否切换。

不要直接说“H.265 太慢”“相机不支持 H.265”或“我们测过 H.264 更好”，这些都需要具体设备信息或测试记录支持。

还有什么视频编码：

编码	怎么理解
MJPEG	每一帧分别压成 JPEG，帧之间不做预测，通常比较占带宽
H.264 / AVC	压缩效率与兼容性之间比较成熟的选择
H.265 / HEVC	进一步提高压缩效率，同时需要考虑计算开销与兼容性
VP9	另一种视频编码标准，常见于互联网视频
AV1	追求较高压缩效率的视频编码标准，也已被部分网络摄像头采用

别混淆：MP4、MKV 是封装格式，不是视频编码。 可以把它们理解成装视频、音频等内容的容器；同一个 MP4 文件里的视频可以采用 H.264，也可以采用 H.265。H.264/H.265 也不是分辨率，不能把 H.264 等同于 1080p、H.265 等同于 4K。

面试口述版：

> H.264 是一种视频压缩编码标准，主要是减少视频传输和存储的数据量，它会利用单帧内部和连续帧之间的相似信息来压缩。我们项目里的编码由网络摄像头完成，具体用什么格式取决于设备能力和视频流配置。我这部分通过 FFmpeg 拉取已经编码好的数据，拆包后封装发布，没有参与编码和解码。H.265 相比 H.264 通常压缩效率更高，同等画质下更省带宽和存储，但计算更复杂，也要确认下游支持。另外还有 MJPEG、VP9、AV1 等编码。

参考：视频压缩原理与编码比较、相机视频流编码配置示例、AV1 在网络摄像头中的应用。

6.Protobuf 是什么？为什么用？和 JSON 有什么区别？

【复习易错】 Protobuf 不要求使用 shared_ptr，普通对象也能填字段。自动生成的是消息类、字段读写和序列化代码，不是整个业务程序。“紧凑”是二进制通常用字段编号等信息减少表示开销，不是“给字段取名字更省时间”。

【追问 shared_ptr】 项目用它管理消息生命周期，并匹配回调、Send / Write 接口。同进程代码复制共享指针可共同持有一个对象，最后一个持有者释放后才销毁；按 const 引用传参本身不增加引用计数。跨进程订阅者不和发送者共用普通 shared_ptr 的引用计数，不能说“所有下游都是同一个指针的主人”。它也不自动保证消息内容的线程安全或全链路零拷贝。**引用计数安全不等于对象内容安全**：不同 shared_ptr 实例可安全地共同管理生命周期；多个线程并发读写同一消息仍需锁等同步，或发布后只读。不能说“shared_ptr 的引用计数不安全”。

面试口述版：

> Protobuf 是一种定义消息结构和序列化数据的工具。上下游要交换数据，先用 .proto 约定有哪些字段、各是什么类型，再用 protoc 生成 C++ 等语言的消息类。程序创建对象、填好字段后，可以把对象里的数据序列化成字节，接收端按相同定义解析。我们项目用它把压缩视频、编码格式、编号和时间戳组织成消息，交给 CyberRT 发布。它的好处是格式统一、自动生成读写代码、支持跨语言，而且二进制表示通常比较紧凑。

记忆线：先定格式 → 生成代码 → 填消息 → 转成字节 → 对方解析。

- **说明书** `.proto`：写字段、类型和编号；`string format = 3` 只规定“有个格式字段”，本次值是 `"h264"` 要在运行时填写。
- **生成代码** `.pb.h / .pb.cc`：构建时由 `protoc` 生成类和读写方法；不是 `.proto` 自己执行，也不必手写打包解析逻辑。
- **消息对象**：程序创建类的对象，调用 `set_format()`、`set_data()` 填本次数据。类是模板，对象才装具体值。
- **序列化后的字节**：转换的是“对象里的字段数据”，不是 C++ 源码或类的代码。可以存在 `std::string` 里，但它是二进制字节，不是可读文字；接收端反序列化后读取字段。
- `bytes data`：像一个装任意字节的袋子，这里装 FFmpeg 读出的压缩视频内容，不会再压缩或解码视频。我们驱动把消息对象交给 CyberRT，传输所需的序列化由框架相应路径处理。

好处记三句：格式先说好，代码自动造，二进制通常更省。

对应含义：①上下游字段统一；②工具生成读写和序列化代码，还支持多种语言；③结构化字段通常少些表示开销，bytes 能直接装视频字节。“更省”不代表把 H.264 再压小，也不保证网络一定传得更快。

追问：和 JSON 有什么区别，为什么这里用它？

> 两者都能表示和传递数据。JSON 是文本，打开就能看懂，方便查看和修改；Protobuf 把消息结构写在 .proto 里，再生成代码，方便程序按统一格式读写，二进制表示通常也更紧凑。我们还可以用 bytes 直接装视频数据，不必先转成 Base64 文字。所以这里用 Protobuf 组织视频消息，配置文件和交通参数插件里也会用 JSON。

易混点：`.proto` 说明书本身也是可读文本；不方便直接看的是“序列化后的二进制消息”，不是数据藏在代码里。JSON 通常直接写出字段名和值。两者都需约定含义，也都能做数据校验。

追问：.proto 怎么写?

> 我们用的是 proto2。先声明语法版本和包名，再用 message 定义消息；字段写清规则、类型、名字和唯一编号。需要引用别的文件就加 import。写完通过 protoc 生成代码。

```
syntax = "proto2";
package os.v2x.device.ipcamera;

message CompressedImage {
    optional string format = 3;
    optional bytes data = 4;
}
```

这是项目字段的摘简示例：`package` 对应 C++ 命名空间；`message` 定义消息；`optional` 表示可选；`string` 装文本、`bytes` 装二进制；`3/4` 是字段编号，不是字段值，已有编号不能随便改或复用。

补充词：`repeated` 是列表，`enum` 是枚举；`set_format("h264")` 是填字段，`format()` 是读字段；手工序列化 / 解析常用 `SerializeToString()` / `ParseFromString()`。

7.DDS 是什么？QoS 是什么?

面试口述版：

> DDS 全称数据分发服务，是一种数据分发中间件标准，提供发布订阅、通信端点发现和 QoS 等能力。发布者发布某类数据，订阅者订阅对应数据，不需要业务代码自己维护每个接收者的连接。Fast DDS、Cyclone DDS 是它的具体实现。我们交通参数服务由 FusionService 宿主封装 DDS，插件通过回调拿到 JSON 数据，再计算指标、发布结果。

记忆：DDS 定标准，Fast DDS 等软件做实现；“发布订阅”是通信模式，DDS 不是 TCP/UDP 这样的底层传输协议。

追问：QoS 是什么?

【复习卡点】 可靠性先记两种：Reliable（可靠）通过确认、重传等机制处理传输丢失；Best Effort（尽力而为）尽力发送，不提供这种可靠性恢复保证。例子：实时传感器数据若更重视新鲜度，可考虑 Best Effort；要求可靠交付的数据可考虑 Reliable。不是 Reliable 就一定用 TCP，也不保证任何故障下都不丢。

> QoS 就是服务质量策略，用来规定消息传递和保留的要求，比如是否要求可靠传输、保留多少条历史数据、晚来的订阅者能否收到保留的数据。发布端和订阅端的策略要兼容，否则可能无法通信。

- **Reliability 可靠性**：Reliable 使用确认、重传等机制；Best Effort 尽力发送，不提供这种可靠性保证。Reliable 不等于任何故障下都绝不丢数据。
- **History / Depth 历史与深度**：例如 Keep Last、Depth 10，保留最近最多 10 条历史样本。
- **Durability 持久性**：Volatile 不向晚加入者补发先前数据；Transient Local 可向匹配的晚加入者提供发布端保留的数据。

别混淆：RTPS 是 DDS 体系中的互操作通信协议；前面摄像头视频用的 RTP 是另一回事。

8.CyberRT、ROS 2 是什么？三者有什么关系?

CyberRT 的定义：

【复习表述】 具体说“AirOS 设备接入链路使用 CyberRT”，交通参数插件另由 FusionService 托管。组件组织管理的是各个业务组件，不是“每个框架”；通信接口叫 Writer / Reader，通道叫 Channel。调度器选择就绪任务交给工作线程执行，不是每个任务都新建一个线程。

> CyberRT 是 Apollo 面向自动驾驶设计的运行时框架，提供模块组织、消息通信和任务调度等能力。模块可以组织成 Component，通过 Writer 向指定 Channel 发布消息，Reader 订阅接收。我们 AirOS 在它外面做了封装，相机组件通过 Send 把 Protobuf 视频消息发布到对应 Channel。

ROS 2 的定义：

> ROS 2 是机器人软件开发框架和工具集，提供节点、Topic、Service、Action、参数和启动管理等能力，让不同功能模块协作。我在导览机器人中用它组织导览、语音等节点，通过 Topic 传状态、通过 Action 下发导航目标。它运行在 Linux 等系统上，不是替代 Linux 的操作系统内核。

两者与 DDS 的关系：

> CyberRT 和 ROS 2 都可以放在开发框架这一层理解，DDS 是通信基础这一层的标准。ROS 2 通常通过 RMW 适配层使用 Fast DDS 等实现，也支持其他通信实现；CyberRT 的 RTPS 通信路径使用了 Fast DDS / Fast RTPS 相关实现，同时还有进程内和共享内存通信路径，所以不能说它每条消息都经过 DDS 网络传输。

记忆：框架组织模块，Topic / Channel 区分消息通道，底层通信负责传递，QoS 规定传递要求。

CyberRT 和 ROS 2 有什么区别：

【复习易错】 ROS 2 的消息通道叫 Topic，CyberRT 叫 Channel。两者都提供模块组织和通信能力；ROS 2 也有组件组合、Executor 等执行机制，不能说“只有 CyberRT 能组织组件和调度任务”。区别重点说 CyberRT 针对自动驾驶集中计算链路设计，ROS 2 的机器人软件包和工具生态更广；ROS 2 也能用于自动驾驶。

比较点	CyberRT	ROS 2
主要定位	面向自动驾驶的运行时框架，关注任务调度和数据处理链路	机器人开发框架与工具生态，也可用于自动驾驶
发布订阅接口	Writer / Reader，消息通道叫 Channel	Publisher / Subscription，消息通道叫 Topic
常见消息定义	Protobuf 的 `.proto`	`.msg`、`.srv`、`.action` 等接口定义
项目中的用法	AirOS 相机消息的发布与组件运行	导览机器人节点协作、导航业务集成

表达要准确：说“通过 CyberRT 发布到某个 Channel”，就像“通过 ROS 2 发布到某个 Topic”。Channel / Topic 是逻辑消息通道，不是另外一台服务器。

两者都有自己的接口和消息约定，不能因为都用到 DDS/RTPS，就认为能直接互通；跨框架对接通常需要相应适配。

追问：ROS 1 和 ROS 2 有什么区别?

> 最大区别是通信架构。ROS 1 通常依赖 ROS Master 做注册和发现，节点之间主要通过 TCPROS 等方式传消息；ROS 2 通常基于 DDS 实现发现和通信，不需要 ROS Master，并提供更丰富的 QoS 策略。此外，ROS 2 的构建工具、节点执行和生命周期管理等机制也有变化。

比较点	ROS 1	ROS 2
注册与发现	通常需要 ROS Master	无需 ROS Master；常见 DDS 实现支持分布式发现，也可采用发现服务器等配置
消息通信	常见 TCPROS，也有 UDPROS	经 RMW 适配层，通常使用 DDS/RTPS 等实现
通信策略	队列、latching 等能力	更丰富的 QoS：可靠性、历史、持久性等
构建与执行	常见 catkin，使用 spin / spinner 等处理回调	常见 ament 包构建、colcon 工作区构建，使用 Executor 等机制组织回调执行
生命周期	没有 ROS 2 这种标准化的托管节点生命周期机制	支持 Lifecycle 节点，显式配置、激活和停用；普通节点不必都采用它

两点别背错：① ROS 1 的普通话题数据通常由节点直接传，不是所有数据都经 Master 中转。② ROS 2 对实时系统开发有更多支持，但不等于用了它就自动满足硬实时；也不是把 ROS 1 版本号升级一下就能直接运行原代码。

最短背法：CyberRT 和 ROS 2 是两套框架；DDS 是通信中间件标准；ROS 1 / ROS 2 先答“Master 与通信架构变化”，再补 QoS 和开发机制。

参考：CyberRT 官方说明、ROS 2 通信实现、ROS 1 / ROS 2 设计变化。

9.FFmpeg 是什么？项目里怎么用？标准解码流程和关键函数是什么?

【函数名易错】 拉流顺序：`avformat_open_input()` 打开输入 → `avformat_find_stream_info()` 探测、补充流信息 → `av_read_frame()` 读取压缩包。信息查询函数是 find_stream_info，不是 av_read_stream_info；打开函数末尾是完整的 input。这些属于拉流读取，不是解码。

【复习补充】 解码流程记完整：找到并初始化解码器 → send_packet 送压缩包 → 循环 receive_frame 取图像 → 按需转换格式 → 输入正常结束时排空缓存、释放资源。“归还关闭”说成“排空并释放资源”更准确；输入包和输出帧不保证一一对应。相机完成编码，我们接入层只拉流、拆包、封装发布。

一、对 FFmpeg 的理解：是什么、为什么用、项目怎么用

是什么：

FFmpeg 是一套**开源音视频处理框架**，既提供命令行工具，也提供程序可以直接调用的库。它能处理网络流、封装与解封装、编码与解码、图像转换等工作。可以理解成音视频处理的工具箱，**它本身不是 RTSP 协议，也不是 H.264 这样的编码标准，更不只是一个播放器。**

组成	主要作用
`ffmpeg` 命令行工具	按命令进行音视频转换、转码等处理
`ffprobe` 命令行工具	查看编码、分辨率、帧率等媒体信息
`ffplay` 命令行工具	简单播放音视频
`libavformat` 库	处理媒体输入输出、封装与解封装，配合协议实现读取 RTSP 等输入
`libavcodec` 库	对压缩音视频进行编码、解码
`libswscale` 库	图像缩放、像素格式与颜色空间转换

为什么用：

因为我们需要连接网络摄像头、读取视频数据，还要处理底层协议和媒体格式。直接用 FFmpeg 的接口，就不用自己从头实现 RTSP 交互、RTP 解析和媒体数据读取。它也提供成熟的编解码能力，但**使用 FFmpeg 不代表一定进行了编解码，要看实际调用了哪些接口。**

项目里怎么用：

我们是在 C++ 代码中**直接调用 FFmpeg 的库接口**，不是启动一个命令行程序来拉流。当前驱动通过 `avformat_open_input()` 打开摄像头 RTSP 输入，调用 `avformat_find_stream_info()` 获取流信息，再用 `av_read_frame()` 循环读取视频压缩包，提取数据后封装成 Protobuf 消息发布。读包后调用 `av_packet_unref()` 释放当前包引用，连接结束时通过 `avformat_close_input()` 关闭输入。**这段链路负责拉流和拆包，没有解码成图像。**

二、解码是什么：输入压缩数据，输出图像

解码就是把 H.264、H.265 等压缩码流**重建成图像帧**，供显示或算法处理。以 FFmpeg 的常见软件解码流程为例，先记住这条主线：

```
准备解码器 → 获取压缩包 → 送入解码器 → 循环取出图像
          → 按需转换格式 → 输入结束时排空、释放资源
```

对象	可以怎么理解
`AVPacket`	装压缩媒体数据的包，还没有解码成像素
`AVFrame`	解码输出的图像帧，包含图像数据及尺寸、格式等信息
`AVCodecContext`	解码器的工作空间，保存配置和解码状态

三、标准解码流程：一般分这六步

第 1 步：确定编码类型，找到并打开解码器。

先确认收到的是 H.264 还是 H.265，找到对应解码器、创建上下文，并准备必要的编码参数，然后打开解码器。如果直接从媒体输入读取，通常可以从流信息中复制参数；如果从消息接收码流，就要从配置或码流中获得必要参数，不能假设任何一段字节都能直接解码。

常见顺序：`avcodec_find_decoder()` → `avcodec_alloc_context3()` → `avcodec_parameters_to_context()` → `avcodec_open2()`。其中复制参数这一步适用于已经有 `AVCodecParameters` 的输入路径，不是所有码流接入方式都必须照搬。

第 2 步：获取压缩视频数据包。

直接读取文件或 RTSP 输入时，用 `av_read_frame()` 获取压缩包，筛出视频流的数据。**在我们拆分后的链路里，下游也可以从 Protobuf 消息里取出压缩码流，按解码器要求组织输入，不需要为了调用解码器再连接一次摄像头。** 如果接收的是任意长度字节块，还需要先解析、组包，确定可送入的压缩数据边界。

第 3 步：把压缩包送入解码器。

调用 `avcodec_send_packet()`，把 `AVPacket` 交给解码器处理。可以理解为“给解码器喂一包数据”。成功送入不代表马上就能拿到图像，因为它可能还需要参考数据或者内部缓存。

第 4 步：循环取出解码后的图像。

调用 `avcodec_receive_frame()`，成功时得到 `AVFrame`。**输入一个包和输出一帧，不保证严格一一对应**，所以成功送包后通常要循环接收当前能输出的图像。接收返回 `EAGAIN` 表示需要继续输入；发送返回 `EAGAIN` 则表示要先接收已有输出，再重试同一个包，不能直接把这个包丢掉。其他错误要按实际情况处理。

第 5 步：按下游要求转换图像。

软件解码结果常见为 YUV 类像素格式，不一定是 RGB。如果下游需要 RGB/BGR 或指定尺寸，可以用 `libswscale` 做颜色、像素格式转换或缩放，常见接口是 `sws_getContext()` 和 `sws_scale()`。**转换是解码后的可选处理，不是每次解码都必须做。** 硬件解码还可能输出硬件表面或设备内存帧，需要按硬件接口处理，不能假设都能直接按 CPU 图像访问。

第 6 步：正常结束时排空缓存，再释放资源。

解码器可能缓存了尚未输出的图像。输入正常结束后，用 `avcodec_send_packet(ctx, nullptr)` 通知没有后续输入，再持续调用 `avcodec_receive_frame()`，直到返回 `AVERROR_EOF`，取完剩余图像。最后释放包、图像和解码器上下文；如果还打开了媒体输入或格式转换上下文，也要分别关闭、释放。

这里的**排空（drain）**是“把剩余输出取完”，`avcodec_flush_buffers()` 是“清除解码器内部缓存状态”，常用于跳转等需要重置状态的情况，**不能用后者代替正常结束时的排空。**

四、关键函数单独记：函数名 + 干什么

阶段	关键函数	作用
打开输入	`avformat_open_input()`	打开文件或 RTSP 等媒体输入；当前接入代码已使用
获取流信息	`avformat_find_stream_info()`	探测、补充音视频流信息；当前接入代码已使用
读取压缩包	`av_read_frame()`	得到压缩媒体包；名字里有 frame，也不是已经解码的图像
查找解码器	`avcodec_find_decoder()`	按编码 ID 找对应解码器
创建上下文	`avcodec_alloc_context3()`	创建并初始化解码器上下文的默认字段
复制编码参数	`avcodec_parameters_to_context()`	把已有流编码参数复制到解码器上下文
打开解码器	`avcodec_open2()`	按配置初始化解码器
准备包、帧对象	`av_packet_alloc()` / `av_frame_alloc()`	分配包和帧的容器对象，分配对象不等于已经填好输入数据

阶段	关键函数	作用
送入压缩包	`avcodec_send_packet()`	**给解码器输入压缩数据；传空包可进入排空阶段**
取出图像	`avcodec_receive_frame()`	**接收解码后的图像帧**
可选格式转换	`sws_getContext()` / `sws_scale()`	创建转换上下文，执行缩放、颜色或像素格式转换
释放当前内容引用	`av_packet_unref()` / `av_frame_unref()`	释放当前数据引用，容器对象可继续复用
释放解码资源	`av_packet_free()` / `av_frame_free()` / `avcodec_free_context()`	释放包对象、帧对象和解码器上下文
释放其他资源	`sws_freeContext()` / `avformat_close_input()`	释放转换上下文、关闭已打开的媒体输入

优先记住这六个解码函数：找、建、配、开、送、取。

```
找：avcodec_find_decoder
建：avcodec_alloc_context3
配：avcodec_parameters_to_context（有流参数时）
开：avcodec_open2
送：avcodec_send_packet
取：avcodec_receive_frame
```

其中**最核心的一对就是** `send_packet` **和** `receive_frame`**：送进去的是压缩包，取出来的是图像帧。** `avformat_open_input()`、`av_read_frame()` 属于输入和读取阶段，不是把压缩数据变成图像的解码函数。

五、面试口述版

问“你怎么理解 FFmpeg”：

> FFmpeg 是一套开源音视频处理框架，既有命令行工具，也有可以在 C++ 里直接调用的库，能做拉流、封装与解封装、编解码和图像转换。我们项目主要用它打开摄像头的 RTSP 输入、获取流信息和循环读取压缩视频包，再封装成 Protobuf 发布。我使用的是拉流和拆包这部分，没有参与解码实现。

问“那你了解解码流程吗”：

> 解码这部分我没有参与实现，但了解 FFmpeg 常见流程。先根据编码类型找到并初始化解码器，然后获取压缩包，通过 `avcodec_send_packet()` 送进去，再循环调用 `avcodec_receive_frame()` 取出图像。拿到图像后按下游要求做颜色转换或缩放，正常结束时排空缓存、释放资源。输入包和输出帧不是严格一一对应的，解码器内部可能有缓存和帧重排序。

问“你们实际用的是软解还是硬解”：

> 我负责的是接入层，具体下游采用哪种解码方式，需要核对对应模块和部署配置。我了解软解主要由 CPU 执行，硬解使用专用硬件解码单元；不能只因为设备带 GPU 或者项目用了 FFmpeg，就认定一定用了硬解。

参考：FFmpeg 官方介绍与库分工、FFmpeg 读取与解码示例、发送、接收与排空机制、图像缩放与格式转换。

10.为啥要单独的拆包 单独的解码?

接入层是系统里唯一认识 URL 的地方，它拉流拆包后把 H.264 封装成 protobuf 上总线；解码器只认压缩包不认 URL，从消息里取数据直接解。这样拆开是因为拆包是 I/O 密集、解码是计算密集，而且一份流能被感知、存储多个下游复用，还能屏蔽不同厂商相机的接入差异。

"解码器不认 URL"只是接口层面的表现，拆开有更深的三层原因：

1. 工种不同 ： 拆包是 I/O 密集型 （网络收发、断线重连、每路一个线程）； 解码是 计算密集型 （吃 CPU/GPU）。两类活放在一起互相拖累，拆开各跑各的线程/进程
2. 复用 ： 一份码流可能被多个下游消费——感知要用、录像存储要用、预览转发也要用。接入只拉一次流，发上总线，谁要谁订阅
3. 屏蔽差异 ： 路口有海康、有大华、有 RTSP 有厂商 SDK，接入层统一消化掉这些差异，下游看到的永远是同一种 protobuf 消息——这就是你简历上"设备抽象"的意义

11.I/O的理解?

I/O = Input/Output，程序与"外部世界"之间的数据搬运

你的程序（CPU + 内存）是一个世界， 凡是跨出这个世界边界的数据交换，都叫 I/O ：

外部世界 I/O 例子 磁盘 读写文件（你想的那个） 网卡 socket 收发网络数据（你们项目这个） 键盘/显示器 你敲键盘、printf 上屏 外设 读串口、读写设备

所以"拉流拆包"为什么算 I/O——数据从 网卡 进内存，程序在收网络数据，跟从磁盘读文件是同一性质的事，只是外部对象从硬盘换成了网卡。

为什么要区分"I/O 密集"和"计算密集"?

关键在 速度差 ： CPU 算一次加法不到 1 纳秒，而从网络收一个包、从磁盘读一块数据，动辄微秒到毫秒级 —— 慢成千上万倍 。于是程序呈现两种完全不同的"忙法"：

- I/O 密集型 ： CPU 大部分时间在 等 外部数据到达，真正干活的时间很少。像厨师一直站在门口等外卖送菜——不是忙，是等
- 计算密集型 ： 数据都齐了，CPU 一直满负荷运算。像菜全备好了，厨师不停切菜炒菜 放回你们项目：拆包线程的大部分时间阻塞在 `av_read_frame()` 上 等网卡收 RTP 包 （I/O 密集）； 解码/检测线程拿到包后疯狂算矩阵（计算密集）。这就是它俩要拆成不同线程的底层逻辑—— 等的人专心等，算的人专心算 ，谁也不耽误谁。

文末新增：专业技能与 Jetson 基础追问（30 题）

> **这部分怎么用** 补的是简历「专业技能」里容易被顺手问到的小点，难度按**实习岗基础问答**控制。每题一段口述答案，加一条顺着答案来的追问；先过 `[必背]`，再看 `[加分]` 和 `[保底]`。 下面是独立新增题组，编号统一用 **Q补**，不占前面原有题号，也不改变原文标题中的 33 题口径。
>
> 🔴 **复习时分清三个边界**
>
> 1. **epoll / Reactor / mmap** 属于简历里的网络服务经历；AirOS 相机接入用的是独立拉流线程，别把两套实现混着讲。网络服务源码不在这次检查的目录里，下面讲通用原理，不编具体压测数字和代码细节。
> 2. **机器人、科研和 AirOS 分开举例。**ROS 2 / TF2 对应机器人项目，UPPAAL 对应科研项目；不会的底层实现就讲到自己实际用过的层次。
> 3. **Jetson 不等于自己做过 CUDA 开发。**能部署程序、读 GPU 指标，不代表写过推理引擎或 GPU 算子；具体板卡型号、JetPack 版本按现场记录说，不靠猜。

补充一：C / C++ 与资源管理（7 题）

> **简历原话 编程语言：**熟悉 C++，能用 RAII 与智能指针管理资源、用移动语义与多态组织代码、用 STL 处理数据；能用 C 做指针/内存管理与系统调用编程；会用 Python 写脚本，用过 PyQt5 做图形界面、Paramiko 做 SSH 远程操作。
>
> **本组重点**：资源谁释放 → 对象能不能复制 → 容器怎么选。Python 和桌面工具见 Q补3-1、Q补3-2。

Q补1-1 简历写了 RAII，能用一个例子讲清楚吗？ [必背] ★★

答：

RAII 就是把**资源的申请和释放交给对象管理**，对象创建时接管资源，销毁时自动释放。比如锁交给一个局部的锁管理对象，函数结束或者中途抛异常，它的析构函数都会负责解锁。这样不用在每个返回分支里都手写清理，少漏一处就少一个隐患。资源不只指内存，也包括文件、锁和网络连接。

追问：既然会自动释放，那对象一直不销毁呢？

> 资源也会一直被占着，所以还得把对象的**生命周期**安排好。RAII 管的是「销毁时怎么收尾」，不替我决定「什么时候该结束」。

> 📌 *自己复习：流量模块 `src/base/BlockingQueue.h:23` 的局部锁管理对象；AirOS `middleware/device_service/modules/ipcamera/standard_ipcamera/standard_ipcamera.h:48` 的析构收尾。线程对象由智能指针管理，也仍然需要正确停止并等待线程退出。*

Q补1-2 unique_ptr 和 shared_ptr 怎么选？weak_ptr 又是干什么的？ [必背] ★★

答：

我先看资源**归谁管**：只有一个持有者，就选独占智能指针；确实需要多个地方共同持有，再选共享智能指针。独占指针不能复制，但可以转移所有权；共享指针靠引用计数，最后一个持有者离开时释放对象。弱指针只观察对象，不增加强引用计数，可以用来打破共享指针之间的循环引用。

追问：用了共享指针，多线程访问对象就安全了吗？

> 不是。不同共享指针实例的引用计数管理有线程安全保证，但它们指向的**对象内容**仍然可能被同时修改，该加锁还是得加锁。

> 📌 *自己复习：`unique_ptr` / `shared_ptr` / `weak_ptr` / `lock()`；AirOS `base/device_connect/ipcamera/device_factory.h:48`、`:56` 提供独占和共享两种设备实例接口，`:76` 是构造函数注册表。*

Q补1-3 移动语义到底省了什么？用了 std::move 就一定不复制吗？ [必背] ★★

答：

移动语义主要是把已有资源的**管理权转过去**，避免再复制一份大对象里的数据。比如一个对象里有很大的动态数组，支持移动时可以接管那块内存，不用重新申请再逐项复制。但「允许移动」不等于「一定发生移动」，还要看这个类型有没有合适的移动操作；移动后也不能假定原对象还保留原来的内容。

追问：那 std::move 本身做了什么？

> 它本身只是把表达式转换成可以参与移动操作的形式，**不负责搬数据**。真正怎么转移资源，由类型的移动构造或移动赋值决定；没有合适的移动操作时，也可能还是复制。

> 📌 *自己复习：流量模块 `src/base/BlockingQueue.h:33` 的右值入队，`:48` 的出队移动。当前接收路径传入的是具名对象，别把「队列支持移动」说成「每次入队都一定零拷贝」。*

Q补1-4 你在项目里怎么用多态？为什么基类析构通常要写成虚函数？ [必背]

答：

项目里用的是**基类指针调用不同子类的实现**。比如设备有统一的初始化和启动接口，框架只认识基类，具体驱动各自实现；检测器也是同一个调用入口，各自完成不同统计。这样上层不用到处判断具体类型。如果要通过基类指针删除子类对象，基类析构要是虚函数，否则这种删除会产生未定义行为。

追问：有虚函数就一定会发生你说的这种多态吗？

> 还要看调用方式。要通过**基类指针或引用**指向实际的子类对象，并发生虚函数调用；如果把子类按值复制成基类对象，子类那部分会被切掉。

📌 *自己复习：AirOS `base/device_connect/ipcamera/device_base.h:60` 的虚析构；流量模块 `src/traffic_flow/traffic_flow_detector.h` 的公共检测接口。主问题的设备接入细节回看前文条目一。*

Q补1-5 你用过哪些 STL 容器？vector、map、unordered_map 怎么选？ [必背] ★

答：

按顺序保存一批数据，我通常先考虑**连续数组容器**，遍历方便，也能按下标访问。按编号查对象，就考虑键值容器：需要按键有序时用有序映射，查找通常是对数复杂度；不需要顺序时可以用哈希映射，平均查找接近常数时间，但最坏情况会退化。选哪个先看访问方式，不是名字带「哈希」就一定更快。

追问：连续数组容器扩容以后，之前保存的元素指针还能用吗？

> 如果扩容引起**重新分配内存**，之前指向元素的指针、引用和迭代器都会失效。可以提前预留容量减少扩容，但不能把旧地址永远有效当成前提。

📌 *自己复习：`vector / map / unordered_map / reserve()`；AirOS 的设备工厂用 `map` 按名字查构造函数，流量工厂用 `vector<unique_ptr<AccumulateDetector>>` 管检测器列表。*

Q补1-6 C 里面用指针和动态内存，最容易错在哪里？ [必背]

答：

我会先确认指针**指向哪里、能访问多长、谁负责释放**。常见错误是没初始化就用、数组越界、释放后继续访问，以及同一块内存释放两次。申请和释放还得配套，C 的分配方式和 C++ 的对象创建方式不能随便混用。写字符串时尤其要留意缓冲区长度和结尾的零字符，指针本身不会告诉我容量。

追问：释放后把指针设成空，就彻底解决了吗？

> 只能避免继续误用**这一个指针变量**。如果还有其他指针指向同一块已释放内存，它们照样悬空，所以关键还是把所有权理清楚。

📌 *自己复习：`malloc/free`、`new/delete`、`new[]/delete[]` 必须配套；栈对象、动态内存、空指针、悬空指针、越界。*

Q补1-7 你说会系统调用编程，系统调用和普通函数调用有什么区别？ [必背]

答：

系统调用就是让**操作系统内核替程序办事**，比如读文件、收网络数据、创建连接。普通函数主要在当前程序里执行，系统调用则要经过内核提供的入口，涉及权限检查，也可能阻塞等待资源。使用时我会检查返回值，分清成功、暂时没数据和真正失败。文件描述符用完要关闭，不能只顾着读写。

追问：库函数和系统调用是一一对应的吗？

> 不是。库函数可能在内部缓存数据、做格式转换，必要时才发起系统调用；也有普通函数完全不需要进入内核。不能只看函数长得像读文件，就断言每调用一次一定读一次磁盘。

> 📌 *自己复习：`open/read/write/close`、`socket/recv/send`、`errno`。这里讲通用原理，不补造某一次具体排障经历。*

补充二：网络与并发（8 题）

> **简历原话 网络与并发**：理解 TCP/IP 与 Socket 编程，用 epoll（边缘触发）与 Reactor 模型写过主从式高并发网络服务；用线程池、互斥锁/条件变量实现生产者-消费者模型，用 mmap 优化 IO，用 GDB 调试定位问题。
>
> **本组重点**：连接怎么建 → 消息怎么拆 → 多连接怎么处理 → 线程怎么协作。网络服务的具体实现和压测结果，以自己的源码及记录为准。

Q补2-1 Socket 是什么？用 TCP 写服务端，大致经过哪几步？ [必背]

答：

Socket 是程序使用网络通信的一套**接口**，可以在它上面用 TCP，也可以用 UDP。TCP 服务端大致是创建套接字、绑定地址和端口、开始监听，再接收连接并收发数据；客户端则主动发起连接。监听用的套接字和接收连接后得到的套接字要分清，前者接新连接，后者和某个客户端交换数据。

追问：TCP 和 UDP 的区别怎么用一句话讲？

> TCP 提供**可靠、有序的字节流**，UDP 按报文发送，本身不保证到达和顺序。选择还要看业务能不能容忍丢失、是否需要自己补可靠性，不能只背「一个快、一个慢」。

> 📌 *自己复习：`socket → bind → listen → accept → recv/send → close`；监听描述符和已连接描述符。TCP/UDP 的展开见前文附录 Q3 的八股补给。*

Q补2-2 TCP 不是可靠传输吗，为什么还会有粘包、半包？ [必背] ★★

答：

因为 TCP 保证的是**字节顺序**，不负责保留应用消息的边界。发送方发两条消息，接收方一次可能拿到一条半，也可能两条一起拿到，这不代表 TCP 出错。常见处理是规定消息头里的长度，先攒够头部，再按长度取完整消息。剩余字节留在接收缓冲区，下次继续拼，不能把一次接收就当成一条业务消息。

追问：那发送一次，数据就一定全部发完吗？

> 也不一定，特别是**非阻塞发送**可能只发送一部分。要记录已经发了多少，把剩余数据保留下来，等可写时继续发送。

> 📌 *自己复习：定长报文、分隔符、长度头三种边界方案；接收缓冲区、发送缓冲区、部分读写。*

Q补2-3 epoll 是干什么的？边缘触发最重要的注意点是什么？ [必背] ★★

答：

epoll 帮我同时等很多连接，哪些连接可以读写了，再交给程序处理，不用给每个连接都安排一个阻塞线程。**边缘触发**关注就绪状态的变化，收到通知后通常要配合非阻塞读写，把当前可处理的数据读完或写到暂时不能继续。否则缓冲区里虽然还有旧数据，也可能一直等不到下一次通知，表现成连接卡住。

追问：你怎么知道这次已经读完了？

> 非阻塞读取返回**暂时无数据**时，就停止这轮读取、继续等事件。还得分清对端关闭、被信号中断和真正的错误，它们的处理不一样。

> 📌 *自己复习：`EPOLLET`、`O_NONBLOCK`、`EAGAIN/EWOULDBLOCK`、`EINTR`；`recv` 返回 0 表示流式连接对端有序关闭。参考 Linux epoll 手册。*

Q补2-4 Reactor 和主从 Reactor，分别是什么意思？ [必背] ★★

答：

Reactor 就是**等事件，再调用对应处理函数**：连接可读了就读，可写了就处理发送，新连接来了就接入。主从 Reactor 的常见分工是主循环接新连接，再把连接分给多个从循环，由从循环处理读写。这样接入和已连接客户端的通信可以分开组织。真正耗时的业务计算还可以交给工作线程，避免把事件循环堵住。

追问：从循环和工作线程是一回事吗？

> 不一定。从循环负责**网络事件和连接读写**，工作线程主要处理耗时业务；具体有没有额外线程池，要看实现，不能把所有线程都叫工作线程。

> 📌 *自己复习：主 Reactor 接连接、从 Reactor 处理连接事件、业务线程池处理计算是常见分工，不等于本人的代码必然三层齐全。面试前回自己的网络服务源码核准。*

Q补2-5 为什么用线程池？什么任务不适合直接塞进共享线程池？ [必背]

答：

线程池是预先准备一组线程，任务来了排队，由空闲线程取走执行，避免每次都创建和销毁线程。它还能**限制并发数**，防止任务一多把机器拖垮。但如果任务长期不返回，比如一直阻塞着等视频流，就会长期占住线程，池子里的其他任务可能没机会执行。项目里的常驻拉流和有结束点的批量部署任务，要分开考虑。

追问：线程数是不是开得越多越快？

> 不是。计算任务受 CPU 核数限制，I/O 任务还受网络、设备和服务端承载能力限制；线程太多反而增加**调度和资源开销**，要结合任务类型设置上限。

> 📌 *自己复习：AirOS 每路常驻拉流线程与 PyQt 批量部署线程池分开举例；有界任务队列、超时、退出和任务收尾。*

Q补2-6 生产者消费者为什么同时需要互斥锁和条件变量? [必背] ★★

答:

互斥锁负责保护**共享队列**，防止两个线程同时改坏它；条件变量负责在队列为空时让消费者睡眠，有数据以后再通知它。等待时会释放锁，醒来重新拿锁，所以生产者才有机会入队。醒来以后还要重新检查队列，满足条件才能取数据。退出时也要把停止状态放进等待条件并唤醒线程，否则可能一直等下去。

追问：为什么重新检查要用循环，不能只判断一次?

> 可能发生**虚假唤醒**，也可能数据先被另一个消费者取走。通知只代表「该起来检查了」，不代表「轮到你时一定还有数据」。

> 📌 *自己复习：流量模块 `src/base/BlockingQueue.h:42` 起的取数据流程；本项目队列满时丢最旧帧，阻塞的是空队列上的消费者，不是把生产者堵住。退出唤醒是通用设计要求，不把它说成当前队列已有的功能。*

Q补2-7 mmap 为什么可能优化 I/O? 是不是用了就一定快? [必背] ★

答:

mmap 可以把文件映射进进程的**虚拟地址空间**，程序通过访问这段地址读取文件内容，实际页面由内核按需准备。相比反复读取到自己的缓冲区，有些场景能减少拷贝和系统调用，适合考虑用于文件访问。但它仍然可能缺页、仍然有磁盘 I/O，也有建立映射的开销，所以要结合文件大小和访问方式比较，不能直接说成完全零拷贝。

追问：映射建立以后，文件关闭了，这段地址还能访问吗?

> 在 Linux 上，关闭原文件描述符**不会自动撤销已有映射**，映射还可以保留。用完要单独解除映射，并且不能越过有效文件范围访问。

> 📌 *自己复习：`mmap/munmap`、`MAP_FAILED`、`MAP_PRIVATE/MAP_SHARED`；原网络项目的具体映射用途和性能数据需回源码确认。参考 Linux mmap 手册。*

Q补2-8 你说用 GDB 定位问题，程序崩溃时你会怎么查? [必背]

答:

我会先保留**崩溃日志、对应程序和调试符号**，能复现就调试运行，现场问题也可以分析 core 文件。进入调试器后先看调用栈，找到出错位置和调用来源，再检查参数、指针和对象状态。多线程问题还要看其他线程在等什么，区分崩溃和卡住。改完重新跑触发场景，不能只因为这次没崩就认为修好了。

追问：如果程序没崩，只是卡着不动，也能用 GDB 吗?

> 可以附加到正在运行的进程，看**所有线程的调用栈**。这样能区分是在正常等网络、等条件变量，还是多个线程互相等锁；线上附加会暂停目标进程，需要考虑现场影响。

> 📌 ***自己复习：`-g`、`bt`、`frame`、`print`、`break`、`thread apply all bt`、`gdb 程序 core文件`。这些是排查步骤，不把通用答案编成自己一定遇到过的具体事故。***

补充三：通信、脚本与开发工具（12 题）

> **简历原话 通信与工具：**会用 ROS 2 的 Topic/Action 通信与 Launch 编排启动、理解 TF2 坐标变换；用 Protobuf 定义消息、YAML/JSON 做配置；了解 Docker，会在容器里运行代码与搭建环境；用 CMake/Bazel 编译构建、Git 管理版本、SSH/SFTP 传输文件；用 UPPAAL 做时间自动机建模与安全性验证；CET-6，可阅读英文技术文档；会用 Claude、Codex、WorkBuddy 等 AI 编程助手辅助代码阅读、调试与脚本编写。
>
> **本组重点**：讲清「拿它做了什么」，不把会用工具升级成实现过工具底层。CET-6 和 AI 辅助也留了保底题，不需要背成技术演讲。

Q补3-1 Python 和 PyQt5 在你的工作里分别做什么？界面为什么不能直接执行远程任务？ [必背]

答：

Python 用来把配置处理、批量操作这些重复工作写成脚本，PyQt5 则给工具加上**图形界面**，方便现场人员使用。远程连接和传文件可能等很久，如果放在界面线程里，按钮、拖动和刷新都会一起停住。所以耗时任务交给后台线程，结果通过信号槽回到主线程，再更新表格和状态。界面主线程主要负责响应用户和显示结果。

追问：后台线程拿到结果后，为什么不直接改界面？

> Qt 的界面对象应由**主线程**操作，跨线程直接修改容易出问题。后台负责干活，通过排队的信号槽把结果交回主线程处理。

> 📌 ***自己复习：PyQt5、信号槽、工作线程 / 线程池；具体部署工具设计回看前文条目三 Q3-1、Q4-1。***

Q补3-2 Paramiko、SSH 和 SFTP，三个是什么关系？ [必背]

答：

SSH 是远程安全连接协议，可以登录设备、执行命令；SFTP 是基于 SSH 的文件传输协议。**Paramiko 是 Python 库**，让我在程序里完成这些操作，不用人工一台台登录。批量部署时，通过它连接设备、执行命令，再用 SFTP 上传配置。每台设备的任务都要有超时和错误结果，最后及时关闭连接，避免失败任务一直占资源。

追问：SFTP 和 FTP 只是名字差不多吗？

> 它们是不同协议。SFTP 通过 SSH 通道传输，不能当成「给普通 FTP 开个加密选项」，服务端也得支持对应协议。

> 📌 *自己复习：连接、认证、命令执行、文件传输各阶段的超时和返回状态；传输结束后核对文件并不等于服务已经加载新配置。参考 Paramiko SSHClient 文档。*

Q补3-3 ROS 2 的 Topic 和 Action 怎么选？导航为什么适合用 Action？ [必背] ★★

答：

Topic 适合**持续发布的数据**，比如激光、里程计或者节点状态，订阅者按需接收。Action 适合需要持续一段时间的任务，客户端提交目标，执行中可以接收反馈，最后拿到结果，也可以请求取消。导航就是这种任务，不能下发以后就不管了。我在导览业务里要根据导航结果决定继续讲解还是跳过展点，所以会用 Action 对接导航。

追问：请求取消，就能保证机器人立刻停下来吗？

> 不能。取消需要服务端处理和响应，客户端还要确认任务的最终状态；实际停车还受**控制执行和安全逻辑**影响，不能把发出取消当成已经停稳。

> 📌 *自己复习：Topic、Service、Action；Action 的目标、反馈、结果、取消。机器人用途来自简历，不补造当前目录没有的导航接口代码和参数。*

Q补3-4 Launch 和 YAML 各自干什么？JSON 配置和它有什么区别？ [必背]

答：

Launch 主要负责**怎么启动系统**，比如启动哪些节点、传哪些参数、话题怎么重映射。YAML 常用来保存参数和业务配置，让速度、展点这些值不用写死在代码里；JSON 也能保存配置，只是语法和工具支持不同。程序读完配置还要检查字段、类型和取值范围，文件格式合法不代表业务值就合理。启动编排和参数内容要分开理解。

追问：把配置文件改了，运行中的程序会自动生效吗？

> 不一定。要看程序有没有实现**重新加载或动态参数更新**；只在启动时读一次的配置，通常要重启相应服务才能生效。

> 📌 *自己复习：Launch 编排不等于默认等待上一个节点完全就绪；YAML 缩进、JSON 语法、默认值和合法范围检查；ROS 2 动态参数与任意业务配置文件不是同一个机制。*

Q补3-5 TF2 是干什么的？map、odom、base_link 为什么要分开？ [必背] ★★

答：

TF2 管的是**不同坐标系之间的关系**，让同一个位置能在传感器、机器人本体和地图坐标下转换。机器人本体坐标跟着车走，里程计坐标追求局部连续，但长期会漂；地图坐标提供全局参考，定位修正可能让两者关系发生变化。把这些坐标分开，既能保留局部运动的连续性，也能接受全局定位修正。传感器外参则把传感器挂到机器人本体上。

追问：坐标关系明明有，为什么还会转换失败？

> 还可能是**时间对不上**。TF2 查询的是某个时刻的变换，消息时间超出已有变换范围、时钟不一致或者变换尚未发布，都可能导致失败。

> 📌 *自己复习：`map → odom → base_link → 传感器`；静态外参和动态位姿；TF2 坐标变换与 AMCL 定位算法职责不同。*

Q补3-6 Protobuf、DDS、CyberRT 是一回事吗？你为什么同时提到它们？ [必背] ★★

答：

它们分工不同。**Protobuf 管消息怎么定义和序列化**，DDS 和 CyberRT 提供消息通信等能力，不能把格式和中间件当成同一层。我们设备接入这边把数据封成 Protobuf，经 CyberRT 发布；交通流量服务则由 FusionService 封装 DDS，交给插件的是 JSON 内容。发布订阅是交换消息的方式，底层仍然可能使用 UDP 等传输。

追问：那为什么不直接把 C++ 结构体的内存发过去？

> 里面可能有**指针、内存对齐和平台差异**，对方收到字节也不能直接当成自己的对象。序列化是按约定编码字段，让接收方重建数据，而不是照搬进程里的地址。

> 📌 *自己复习：AirOS `middleware/runtime/src/air_middleware_common.h:21` 起封装 CyberRT；流量服务 `config/traffic_flow/traffic_flow_service.json:10` 起配置 Topic、Domain ID、UDPv4。Protobuf 不是压缩算法，JSON 也不一定落成磁盘文件；新增 Protobuf 字段要使用新编号，不复用已经删除字段的编号。*

Q补3-7 Docker 镜像和容器是什么？用了容器就能在任何机器上跑吗？ [必背]

答：

镜像是程序和依赖环境的**打包模板**，容器是从镜像启动出来的运行实例。它能减少现场机器缺库、版本不一致的问题，也方便部署和替换版本。但容器共享宿主机内核，CPU 架构、GPU 驱动和设备访问仍然受宿主机约束，不能保证换任何硬件都能直接跑。配置和日志通常通过目录挂载管理，避免跟运行实例绑死。

追问：服务没收到网络消息，你会只查容器里的代码吗？

> 不会，还要查**网络模式、端口、绑定地址和防火墙**。服务端口开着不代表对方能到达它；用了主机网络，也不等于所有链路天然连通。

> 📌 *自己复习：image、container、volume、network；流量工程 `docker-compose.yaml` 使用主机网络并挂载配置目录。Jetson 的架构与 GPU 兼容性见 Q补4-1、Q补4-2。*

Q补3-8 CMake、Bazel 和编译器分别负责什么？你用到什么程度？ [必背]

答：

编译器负责把源代码编译成目标文件，链接器再把目标文件和库组成程序。**CMake 和 Bazel 管构建关系**，告诉工具要编哪些文件、依赖哪些库、产出哪些目标，不能把它们叫成 C++ 编译器。我主要是按项目已有规则构建、补模块依赖和排查编译链接错误。工程里 AirOS 用 Bazel，交通流量模块用 CMake，这两套要分开说。

追问：找不到头文件和链接时报未定义符号，是同一种问题吗？

> 不是。前者通常查**头文件路径和依赖安装**，后者要查函数有没有实现、对应库有没有链接，以及版本和符号是否匹配，先分清发生在编译还是链接阶段。

> 📌 *自己复习：AirOS 根目录 `WORKSPACE / BUILD`；交通流量根目录 `CMakeLists.txt:45` 起设置 C99、C++17。CMake 常生成 Ninja/Make 等构建文件，Bazel 按目标和声明依赖组织构建。*

Q补3-9 Git 平时怎么用？合并冲突时你会怎么处理？ [必背]

答：

Git 用来记录代码变化和管理分支，让一项修改可以单独开发、检查，再合并回去。提交前我会先看**差异**，确认只包含这次需要的改动，再写清修改目的。发生冲突时，要理解两边各自要保留什么，整理出正确结果后重新编译或运行相关场景。不能为了让冲突标记消失，就直接把某一边全部覆盖掉。

追问：已经提交的修改有问题，怎么撤回？

> 先看提交有没有**共享给别人**。共享分支通常用一个新提交反向撤销，保留历史；只在自己本地的提交才考虑整理历史，并先确认工作区改动不会丢。

> 📌 *自己复习：`status / diff / log / branch / merge / revert`；`reset` 会影响分支或工作区状态，不能把所有撤销场景都背成一条硬重置命令。*

Q补3-10 UPPAAL 是干什么的？时间自动机比普通状态机多了什么？ [加分]

答：

UPPAAL 是用来建模和验证**带时间约束的状态系统**的工具。时间自动机在状态和转移之外加入时钟，可以表达「等待多久以后才允许切换」或者「这个状态最多停留多久」。我的科研项目用它表示车辆和障碍物行为，再检查碰撞等安全性质。验证结论只对模型及其假设成立，不能直接说证明了真实道路上绝对安全。

追问：能找到一次碰撞，和证明一定会碰撞，是一回事吗？

> 不是。**存在一条路径**能碰撞，只说明有这种可能；「所有允许的路径最终都碰撞」是更强的性质，要用不同的查询表达，不能把一条危险轨迹说成必然发生。

> 📌 *自己复习：状态、时钟、守卫、重置、不变式；`E<> collision` 表示存在路径到达碰撞，`A<> collision` 表示所有路径最终到达碰撞，`A[] not collision` 表示所有可达状态都不碰撞。参考 UPPAAL 查询语义。*

Q补3-11 你写 CET-6，英文文档能读到什么程度？ [保底]

答：

我能结合代码阅读**英文技术文档和接口说明**，重点看功能、参数、返回值、使用条件和示例。遇到不熟悉的术语会查资料或辅助翻译，但最后要回到原文确认，特别是超时、单位和版本限制。我的优势是能带着具体问题去找答案，不会把六级成绩直接说成口语交流或者所有文档都没有障碍。

追问：读一个没用过的库，你先看哪里？

> 先看**最小使用示例和接口约定**，把输入输出、资源释放和错误处理弄清楚，再补需要的章节。先解决眼前问题，不要求把整份文档从头翻完。

Q补3-12 你会用 AI 编程助手，那哪些工作是你自己完成的？ [保底] ★★

答：

我会主动说明用了 **AI 辅助**，主要帮我读陌生代码、整理排查方向和写重复脚本。需求怎么拆、接口是否对得上、修改能不能运行，仍然需要我自己确认；生成的代码也要看资源释放、异常路径和线程边界。部署工具是我独立开发维护的，但独立负责不等于每一行都不借助工具。我能讲清自己负责的模块和验证过程，讲不清的部分不会认成自己掌握了。

追问：怎么防止 AI 给出的代码看着对、实际有问题？

> 先核对**真实接口和项目上下文**，再编译、运行最小场景，补查失败分支。一次只让它改清楚一件事，结果要验证，不能把「它说通过了」当作证据。

📌 ***自己复习：不把使用 Claude / Codex / WorkBuddy 说成训练过模型；部署工具的个人职责口径与前文条目三 Q3-8 保持一致。***

补充四：Jetson 与边缘设备小点（3 题）

> **简历对应位置** 实习技术栈写了 **Jetson Linux**，工作内容包含读取 **Tegra 系统接口**采集 GPU 指标。
>
> ❗ **名字先念对**：写作 **Jetson**，不是 Jeston；MEC 是边缘计算设备这一角色，Jetson 是可能采用的硬件平台，两者不是同一个产品名称。

Q补4-1 Jetson 是什么？和普通电脑、MEC 是什么关系？ [必背] ★★

答：

Jetson 是 NVIDIA 面向机器人、视觉等边缘场景的**嵌入式计算平台**，常见型号集成了 ARM CPU 和 GPU。MEC 说的是边缘计算角色，可以采用 Jetson，也可以采用其他硬件。我接触它主要是运行 Linux 软件、调数据链路和采集硬件状态，算法推理和底层驱动不是我负责。具体型号按现场记录说明。

追问：那电脑上编好的程序，可以直接复制过去运行吗？

> 不一定。普通开发电脑常见是 **x86-64**，Jetson 常见是 **ARM64**，二进制和依赖库要对应架构；可以在目标平台编译，也可以配置交叉编译，不能只看两边都叫 Linux。

> 📌 *自己复习：ARM64 / AArch64、x86-64、交叉编译、目标平台依赖；流量 README 标注 ARM NVIDIA AGX ORIN，但不能据此断言所有现场机器都是这个型号。参考 NVIDIA Jetson 软件文档。*

Q补4-2 Jetson Linux、JetPack、CUDA、TensorRT 怎么区分？ [加分]

答：

Jetson Linux 是 Jetson 的**系统和板级软件基础**，JetPack 是配套的软件开发栈，包含系统与计算组件。CUDA 提供 GPU 通用计算能力，TensorRT 用于模型推理优化和运行，它们都不等于 AirOS。我使用的是搭好的项目环境，能说明程序和依赖怎么部署，但不会因为环境里有这些库，就说自己写过 GPU 算子或推理引擎。

追问：用了 Docker，GPU 环境版本还需要匹配吗？

> 需要。容器仍然依赖**宿主机驱动、设备和运行时支持**，镜像也要对应硬件架构及兼容的软件版本。容器能固定很多用户态依赖，不能替换整台机器的底层环境。

> 📌 *自己复习：只记分工，不硬背现场未确认的 JetPack / CUDA 版本；CUDA 也不等于视频解码。参考 NVIDIA JetPack 软件栈说明。*

Q补4-3 Jetson 的「显存」和电脑独显一样吗？GPU 指标读不到怎么查？ [必背] ★

答：

我接触的 Tegra 集成 GPU 场景里，CPU 和 GPU 使用同一套**物理内存**，所以监控里的「显存占用」必须结合接口统计口径理解，不能照搬独显概念。采集程序读系统暴露的负载、内存和温度信息，再做单位换算。读不到时先查接口路径、权限和系统版本，不能直接把失败填成零；这项采集失败也不应让整个服务崩掉。

追问：GPU 利用率高，能直接说明程序有问题吗？

> 不能。还要结合**温度、内存、输出频率和延迟**一起看，利用率高也可能只是正在正常处理任务。先确认异常表现，再定位瓶颈，不能凭一个数字下结论。

> 📌 *自己复习：AirOS `middleware/protocol/om_common/performence_utils.cc:976` 起读 GPU 负载，`:1000` 起解析 nvmap 信息，`:1067` 起查找温度节点。不同系统版本、权限和统计接口可能不同，不背「读文件就跨版本完全稳定」。共享物理内存也不等于任意 CPU 指针都能直接给 GPU 用，参考 NVIDIA CUDA for Tegra 内存说明。*

> 📍 **简历术语 → 新增题号**
>
> - **RAII / 智能指针 / 移动语义 / 多态 / STL** → Q补1-1 ~ Q补1-5。
> - **C 指针与内存 / 系统调用** → Q补1-6、Q补1-7。
> - **TCP/IP / Socket / epoll 边缘触发 / 主从 Reactor** → Q补2-1 ~ Q补2-4。
> - **线程池 / 互斥锁与条件变量 / 生产者消费者** → Q补2-5、Q补2-6。
> - **mmap / GDB** → Q补2-7、Q补2-8。
> - **Python / PyQt5 / Paramiko / SSH / SFTP** → Q补3-1、Q补3-2。
> - **ROS 2 Topic / Action / Launch / YAML / JSON / TF2** → Q补3-3 ~ Q补3-5。
> - **Protobuf / DDS / CyberRT / FusionService** → Q补3-6。
> - **Docker / CMake / Bazel / Git** → Q补3-7 ~ Q补3-9。
> - **UPPAAL / 时间自动机 / 安全性验证** → Q补3-10。
> - **CET-6 / 英文文档 / AI 编程助手** → Q补3-11、Q补3-12。
> - **Jetson / ARM / Tegra / JetPack / CUDA / TensorRT / GPU 状态** → Q补4-1 ~ Q补4-3。
>
> **突击顺序**：先过 Q补1-1 ~ Q补1-3、Q补2-3、Q补2-6、Q补3-3、Q补3-6、Q补4-1；其余按简历被追问到的词补上，不用一次全背。